



Universitatea
Transilvania
din Brașov
FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR

Programare orientată pe obiecte

–

Programarea calculatoarelor și limbaje de programare – modulul 3

Îndrumar de laborator

prof. dr. ing. Dominic Mircea KRISTÁLY
drd. ing. Mădălina NIȚOI

BRAȘOV, 2026

Cuprins

1.	Introducere în C# și .NET	7
1.1.	Limbajul de programare C#	7
1.2.	Platforma .NET.....	8
2.	Crearea și rularea programelor C# cu ajutorul mediului integrat de dezvoltare Visual Studio	8
2.1.	Instalarea și configurarea mediului de lucru VS Community	9
2.2.	Crearea unui proiect în Visual Studio.....	10
2.3.	Crearea unei aplicații Windows Forms utilizând C#	12
2.3.1.	Dezvoltarea interfeței utilizator	12
2.3.2.	Adăugarea codului	14
2.3.3.	Rularea aplicației.....	15
2.4.	Structura proiectului Windows Forms	15
2.4.1.	Form1.cs	15
2.4.2.	Form1.Designer.cs.....	15
2.4.3.	Form1.resx	17
2.4.4.	Fereastra Designer	17
3.	Proprietățile și evenimentele unui control Windows Forms	18
3.1.	Controale Windows Forms.....	18
3.1.1.	Common Controls (Controale comune).....	18
3.1.2.	Containers (Controale container).....	20
3.2.	Proprietățile unui control	21
3.3.	Evenimentele unui control.....	22
4.	Tipuri de date.....	22
4.1.	CTS.....	23
4.2.	Tipuri întregi.....	23
4.2.1.	Tipurile byte și sbyte	24
4.2.2.	Tipurile short și ushort.....	24
4.2.3.	Tipurile int și uint.....	24
4.2.4.	Tipurile long și ulong.....	24
4.3.	Tipuri reale.....	24
4.4.	Tipul caracter.....	24
4.5.	Tipul logic.....	25
4.6.	Conversia valorilor între diferite tipuri de date	25
4.6.1.	Parse.....	25

4.6.2.	Convert.....	25
4.6.3.	TryParse.....	25
4.6.4.	ToString.....	26
4.7.	Variabile.....	26
4.8.	Constante.....	26
4.9.	Operatori.....	27
4.9.1.	Operatori aritmetici	27
4.9.2.	Operatori de atribuire.....	27
4.9.3.	Operatori relaționali.....	27
4.9.4.	Operatori logici.....	28
4.9.5.	Operatori bitwise.....	28
4.9.6.	Operatori speciali	29
4.10.	Aplicație cu operatori aritmetici (Calculator)	29
4.11.	Exerciții.....	37
4.11.1.	Calculul ariei unui dreptunghi.....	37
4.11.2.	Conversia temperaturii.....	37
5.	Structuri de control.....	39
5.1.	Instrucțiuni de decizie / selecție.....	39
5.1.1.	Instrucțiunea if.....	39
5.1.1.1.	Clauza <i>else</i>	40
5.1.1.2.	Clauza <i>else if</i>	41
5.1.1.3.	Instrucțiuni <i>if</i> imbricate.....	42
5.1.1.4.	Utilizarea instrucțiunii <i>if</i>	43
5.1.2.	Instrucțiunea switch.....	45
5.2.	Instrucțiuni iterative.....	47
5.2.1.	Instrucțiunea for.....	48
5.2.1.1.	Cicluri <i>for</i> imbricate.....	49
5.2.1.2.	Utilizarea instrucțiunii <i>for</i>	50
5.2.1.3.	Instrucțiunea <i>foreach</i>	54
5.3.	Exerciții.....	55
5.3.1.	Aplicație Windows Forms pentru calculul mediei ponderate.....	55
5.4.	Instrucțiunea while	57
5.4.1.	Utilizarea instrucțiunii while.....	58
5.5.	Instrucțiunea do..while	61
5.6.	Instrucțiuni de salt.....	62
5.6.1.	Instrucțiunea break.....	62
5.6.2.	Instrucțiunea continue.....	63

5.6.3.	Instrucțiunea return.....	63
5.6.4.	Instrucțiunea goto.....	64
5.7.	Utilizarea instrucțiunilor do..while, break, continue și goto	65
5.8.	Exerciții.....	68
5.8.1.	Extinderea aplicației de simulare a depozitului bancar	68
6.	Tablouri	70
6.1.1.	Declararea unui tablou.....	70
6.1.2.	Instanțierea.....	70
6.1.3.	Inițializarea.....	71
7.	Colecții.....	71
7.1.	Liste.....	71
7.1.1.	Declararea listei.....	72
7.1.2.	Constructorii listei	72
7.1.3.	Parcurgerea listei cu foreach	72
7.1.4.	Metode ale colecției List<T>.....	73
7.1.4.1.	Metoda Add(T).....	76
7.1.4.2.	Metoda Remove(T)	76
7.1.4.3.	Metoda IndexOf(T).....	77
7.1.4.4.	Metoda LastIndexOf(T).....	78
7.1.4.5.	Metoda Sort().....	79
7.1.4.6.	Metoda BinarySearch(T).....	79
7.1.4.7.	Metoda CopyTo(T[]).....	80
7.1.5.	Clasa Enumerable	81
7.1.5.1.	Metoda Sum().....	82
7.1.5.2.	Metoda Average().....	82
7.1.5.3.	Metoda Min().....	83
7.1.5.4.	Metoda Max()	83
7.1.5.5.	Metoda Count().....	84
7.1.5.6.	Metoda Aggregate().....	84
7.2.	Aplicație cu liste – Manager de note pentru studenți	86
7.2.1.	Structura proiectului:.....	86
7.2.2.	Design grafic – Form1	87
7.2.3.	Proprietăți și evenimente pentru controale	87
7.2.4.	Cod sursă.....	88
7.3.	Exerciții.....	93
7.3.1.	Exercițiul 1 – Extinderea aplicației Manager de note.....	93
8.	Fluxuri de intrare / ieșire (fișiere).....	95

8.1.	Fișiere în C#.....	95
8.2.	Clasa File în C#	95
8.2.1.	Metode pentru informații despre fișier.....	96
8.2.2.	Metode de creare și scriere a unui fișier	97
8.2.3.	Metode de citire dintr-un fișier	99
8.2.4.	Metode de copiere, mutare și ștergere a unui fișier.....	101
8.3.	Aplicație cu fișiere – TextFile Manager	103
8.3.1.	Structura proiectului:.....	103
8.3.2.	Design grafic – Form1	104
8.3.3.	Proprietăți și evenimente pentru controale	104
8.3.4.	Cod sursă.....	105
8.4.	Exerciții.....	109
8.4.1.	Exercițiul 1.....	109
9.	Programare în rețea	111
9.1.	Modelul client-server.....	111
9.2.	Adrese IP.....	113
9.3.	DNS.....	113
9.4.	Porturi și socket-uri.....	114
9.5.	Exemplu cu IP și DNS.....	115
9.6.	Protocoale de nivel transport (L4).....	116
9.6.1.	TCP	116
9.6.2.	UDP	119
9.7.	Protocoale de nivel aplicație (L7)	120
9.7.1.	HTTP / HTTPS	120
9.7.1.1.	HTTP	120
9.7.1.2.	HTTPS.....	120
9.7.2.	Programare HTTP în C# – clasa HttpClient	122
9.7.2.1.	GetAsync.....	123
9.7.2.2.	SendAsync.....	124
9.7.2.3.	Deserializare JSON cu System.Text.Json	125
9.8.	Exerciții.....	126
9.8.1.	Downloader paralel de imagini cu HttpClient.....	126
9.8.2.	Client REST pentru un API public de vreme	127
10.	Programare orientată pe obiecte în C#.....	128
10.1.	Clase și obiecte	128
10.1.1.	Declararea unei clase	128
10.1.2.	Instanțierea obiectelor.....	129

10.1.3.	Câmpuri.....	130
10.1.4.	Proprietăți	130
10.1.5.	Metode	131
10.1.6.	Constructori.....	132
10.1.7.	Modificatori de acces	133
10.2.	Încapsulare.....	133
10.3.	Moștenire	134
10.3.1.	Cuvântul cheie base.....	135
10.4.	Polimorfism	136
10.4.1.	Metode virtuale și override.....	136
10.4.2.	Conversii între tipurile din ierarhie	137
10.5.	Clase abstracte	137
10.6.	Interfețe	138
10.7.	Aplicație – Gestionar de figuri geometrice.....	140
10.7.1.	Structura proiectului.....	140
10.7.2.	Design grafic – Form1	140
10.7.3.	Proprietăți și evenimente pentru controale	141
10.7.4.	Cod sursă.....	142
10.8.	Exerciții.....	147
10.8.1.	Exercițiul 1 – Extinderea ierarhiei de figuri	147
10.8.2.	Exercițiul 2 – Interfața IScalabil	148
11.	Tratarea excepțiilor și depanarea aplicațiilor	149
11.1.	Tipuri de erori în aplicații	149
11.2.	Ierarhia claselor de excepții în .NET	150
11.3.	Instrucțiunea try-catch.....	151
11.3.1.	Mai multe blocuri catch	151
11.3.2.	Accesarea detaliilor excepției	152
11.4.	Clauza finally.....	152
11.5.	Instrucțiunea using și interfața IDisposable.....	153
11.6.	Aruncarea excepțiilor – instrucțiunea throw.....	154
11.6.1.	Re-aruncarea unei excepții: throw; vs throw ex;	155
11.7.	Filtre de excepții – clauza when.....	155
11.8.	Excepții personalizate.....	156
12.	Depanarea aplicațiilor în Visual Studio.....	157
12.1.	Breakpoints.....	157
12.2.	Rularea pas cu pas.....	158
12.3.	Ferestrele Locals, Watch, Immediate	158

12.4.	Fereastra Call Stack.....	158
12.4.1.	Setări pentru oprirea la excepții	158
13.	Aplicație – Cont bancar cu jurnal de tranzacții.....	159
13.1.	Structura proiectului.....	159
13.1.1.	Design grafic – Form1	159
13.1.2.	Proprietăți și evenimente pentru controale	160
13.1.3.	Cod sursă.....	161
13.2.	Exerciții.....	167
13.2.1.	Exercițiul 1 – Extinderea jurnalului.....	167
13.2.2.	Exercițiul 2 – Excepție personalizată nouă.....	167
13.2.3.	Exercițiul 3 – Depanare în Visual Studio.....	167
BIBLIOGRAFIE		169

LABORATORUL 1

1. Introducere în C# și .NET

1.1. Limbajul de programare C#

C# (pronunțat în limba engleză *C sharp*) este un limbaj de programare cu scop general, sigur pentru tipuri, orientat pe obiecte. Scopul limbajului este productivitatea programatorului. În acest scop, C# echilibrează simplitatea, expresivitatea și performanța. Arhitectul principal al limbajului încă de la prima versiune este Anders Hejlsberg (creatorul Turbo Pascal și arhitectul Delphi). [1]

C# este o implementare bogată a paradigmei orientării pe obiect, care include încapsularea, moștenirea și polimorfismul. Încapsularea înseamnă crearea unei granițe în jurul unui obiect pentru a separa comportamentul său extern (public) de detaliile sale interne (private) de implementare [1]. C# prezintă următoarele caracteristici: [1]

- Sistem unificat de tipuri pe obiecte – Elementul fundamental în C# este o unitate încapsulată de date și funcții numită tip. C# are un sistem unificat de tipuri în care toate tipurile împărtășesc, în cele din urmă, un tip de bază comun. Aceasta înseamnă că toate tipurile, fie că reprezintă obiecte de business sau sunt tipuri primitive precum numerele, împărtășesc aceeași funcționalitate de bază;
- Clase și interfețe – În C#, pe lângă clase există și interfețe. O interfață este ca o clasă care nu poate stoca date. Aceasta înseamnă că poate defini doar comportamentul (și nu starea), ceea ce permite moștenirea multiplă, precum și o separare între specificație și implementare;
- Proprietăți, metode și evenimente – În paradigma pur orientată pe obiecte, toate funcțiile sunt metode. În C#, metodele sunt doar un tip de membru al funcției, care include și proprietăți și evenimente. Proprietățile sunt membri ai funcției care încapsulează o parte din starea unui obiect, cum ar fi culoarea unui buton sau textul unei etichete. Evenimentele sunt membri ai funcției care simplifică acțiunea asupra schimbărilor de stare a obiectului.

Domenii de utilizare:

- Aplicații desktop: Windows Forms, WPF;
- Aplicații web: ASP.NET Core;
- Aplicații mobile: Xamarin / .NET MAUI;
- Jocuri: Unity;
- Servicii și API-uri backend: microservicii, servicii web.

1.2. Platforma .NET

.NET este o platformă dezvoltată de Microsoft care oferă un mediu pentru rularea aplicațiilor și include un set de biblioteci și un runtime performant.

Componentele arhitecturii .NET sunt următoarele:

1. CLR (*Common Language Runtime*) – Este motorul de execuție al .NET care:
 - Codul C# este compilat de compilatorul *Roslyn* în cod intermediar (IL – *Intermediate Language*). CLR utilizează un compilator *Just-In-Time* (JIT) pentru a transforma IL în cod nativ executabil pe platforma curentă.
 - Eliberează memoria (*Garbage collection*), oferă securitate, gestionează excepțiile și administrează thread-uri.
2. FCL (*Framework Class Library*) – conține mii de clase pentru a oferi acces la API-ul Windows și funcții comune precum manipularea șirurilor, structuri de date comune, I/O, fluxuri, fire de execuție, securitate, programare de rețea, programare Windows, programare web și acces la date. [2]

Figura 1 ilustrează etapele arhitecturii .NET și fluxul de execuție al unei aplicații.

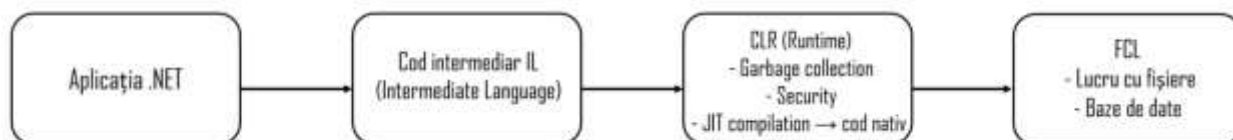


Figura 1 - Arhitectura platformei .NET

2. Crearea și rularea programelor C# cu ajutorul mediului integrat de dezvoltare Visual Studio

Visual Studio este un mediu de dezvoltare profesional (IDE – *Integrated Development Environment*) creat de Microsoft pentru construirea aplicațiilor software moderne.

Visual Studio funcționează pe bază de proiecte și soluții, unde proiectele conțin aplicațiile propriu-zise (fișiere, cod, resurse), iar soluția poate conține mai multe proiecte (ex: aplicație + bibliotecă + API).

Visual Studio este un mediu integrat de dezvoltare complet care gestionează întreg procesul de dezvoltare:

- Scrierea codului;
- Organizarea proiectului;
- Compilarea;
- Testarea;
- Depanarea (debugging);
- Publicarea aplicației.

Visual Studio Community este un mediu de dezvoltare gratuit pentru studenți, dezvoltatori individuali și proiecte open-source.

La momentul editării acestei lucrări, cea mai recentă versiune, *Visual Studio 2026*, poate fi descărcată de pe site-ul oficial <https://visualstudio.microsoft.com/downloads/>.

2.1. Instalarea și configurarea mediului de lucru VS Community

După instalarea aplicației Visual Studio Community, se va deschide fereastra de configurare a componentelor (Figura 2). În această fereastră, se va bifa căsuța corespunzătoare pentru .NET Desktop Development.

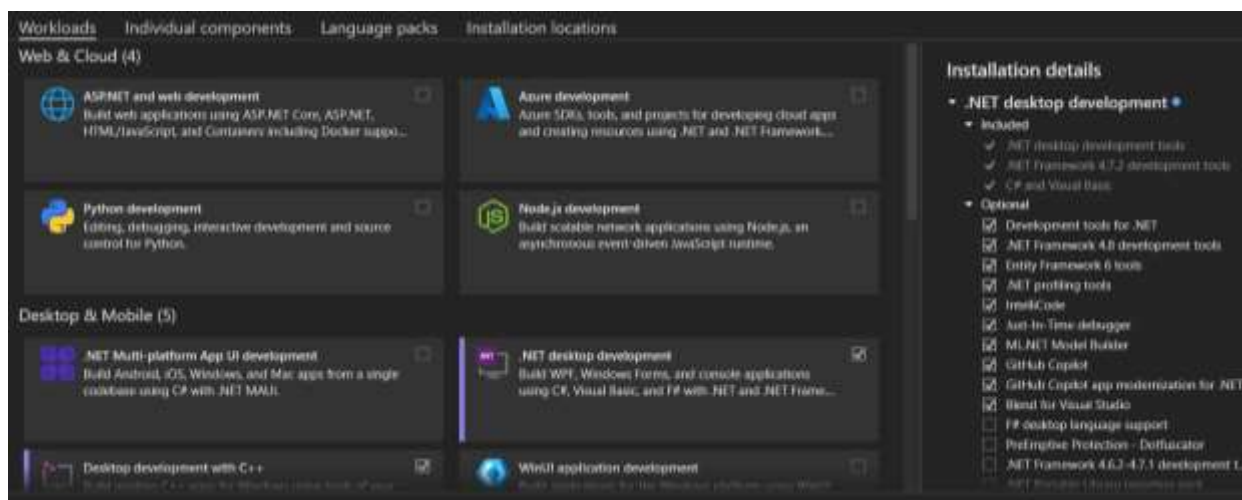


Figura 2 - Selectarea componentei .NET Desktop Development

După instalare, la prima deschidere, aplicația va solicita autentificarea într-un cont Microsoft. (Figura 3). Acest pas poate fi omis apăsând pe textul "Skip and add accounts later".

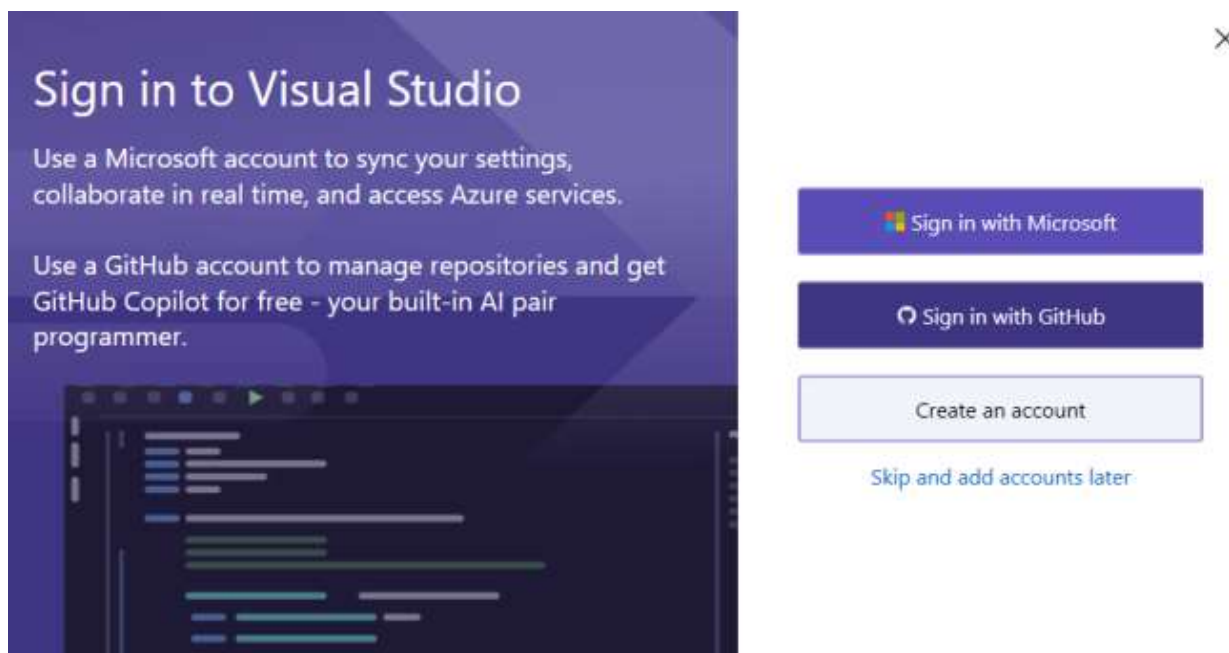


Figura 3 - Omiterea pasului de autentificare într-un cont Microsoft. [3]

După care se poate alege setarea pentru tipul de dezvoltare și alegerea culorii temei pentru IDE (Figura 4).

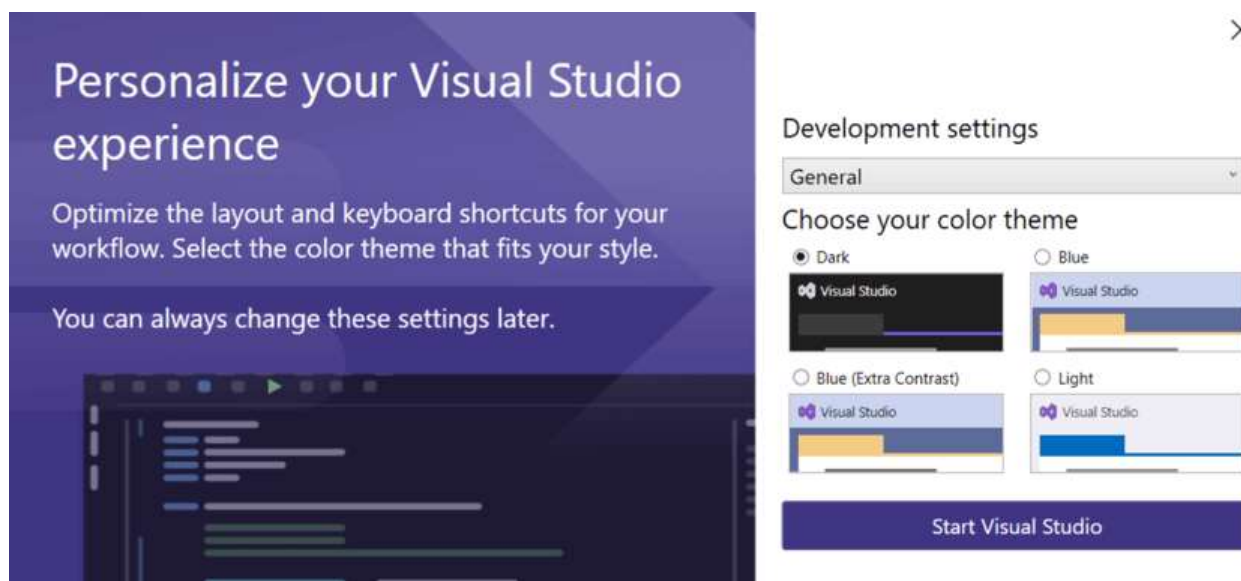


Figura 4 - Setarea pentru tipul de dezvoltare și alegerea culorii temei pentru IDE. [3]

2.2. Crearea unui proiect în Visual Studio

Pentru a crea un Windows Forms App, se selectează "Create a new project" sau se accesează File → Create a new project. Se alege limbajul C#, platforma poate rămâne "All platforms", iar din lista afișată de șabloane se selectează "Windows Forms App" (Figura 5).

În noua fereastră care apare, se completează numele proiectului, care, la fel ca în cazul limbajului Java, ar fi indicat să înceapă cu literă mare (Figura 6). După care se poate selecta locația în care se dorește salvarea proiectului.

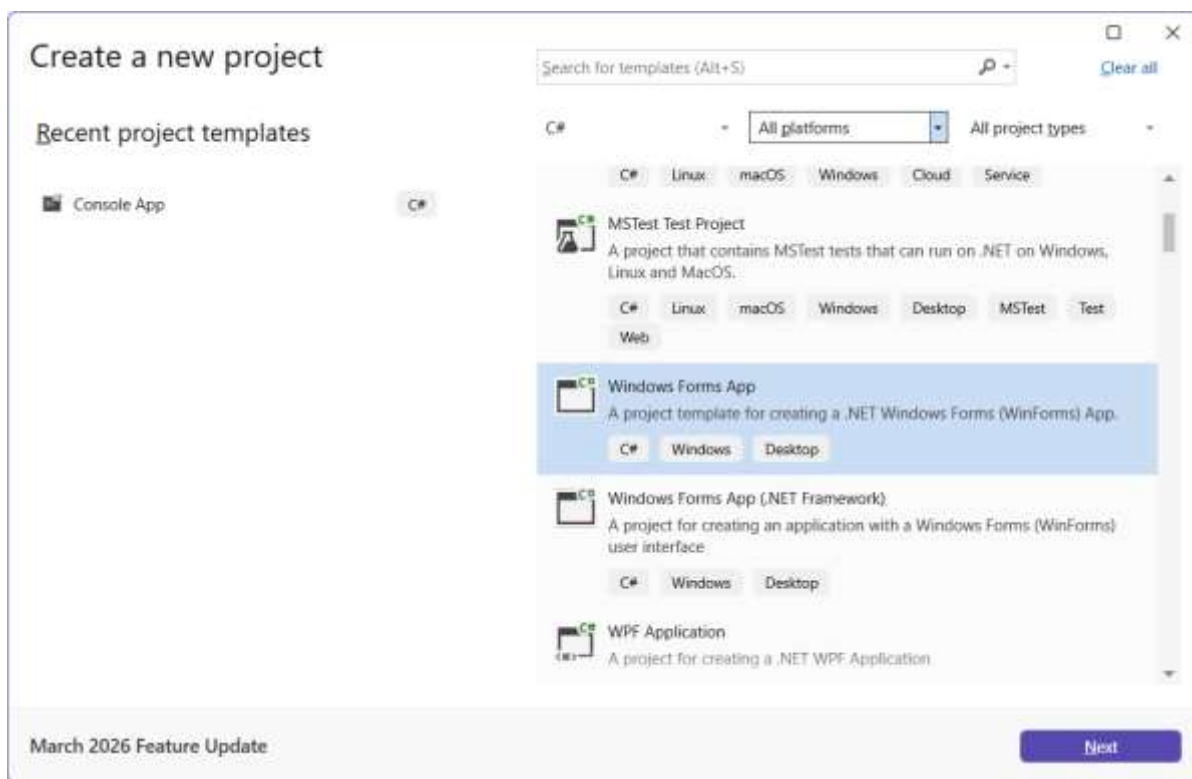


Figura 5 - Crearea unui Windows Forms App în Visual Studio

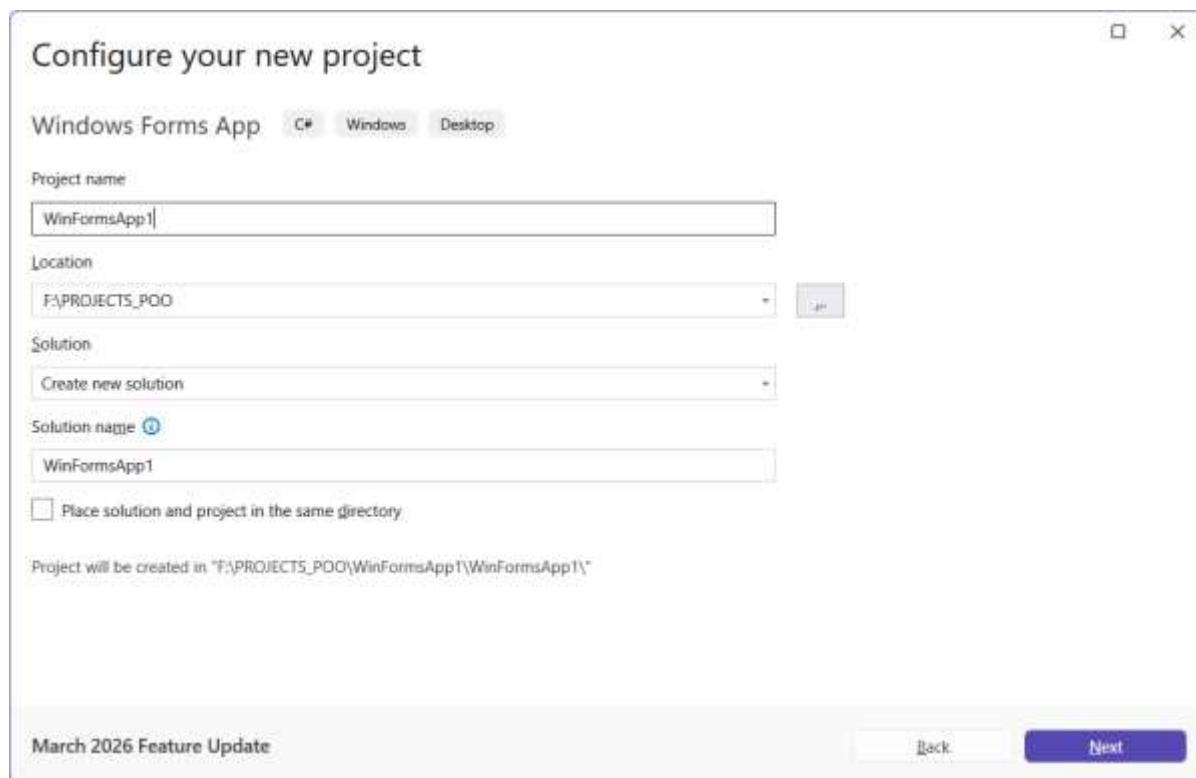


Figura 6 - Configurarea proiectului dezvoltat

Urmează alegerea versiunii .Net. Este recomandat să se folosească numai versiuni LTS (*Long term support*) (Figura 7).

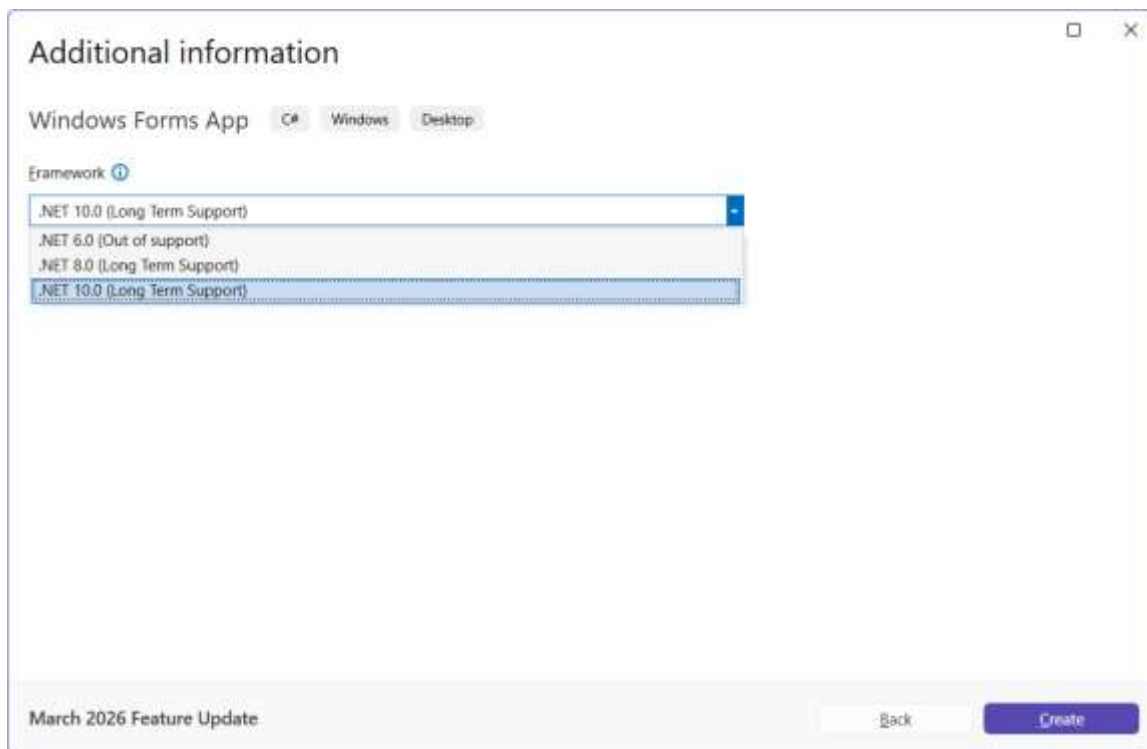


Figura 7 - Alegerea versiunii .Net pentru aplicație

În cadrul unei aplicații Windows Forms, Visual Studio pune la dispoziție următoarele componente principale: *Solution Explorer*, *Form Designer*, *Toolbar* și *Output Window* (Figura 8). **Solution Explorer** afișează proiectele din spațiul de lucru, fișierele acestora și referințele la biblioteci. **Toolbar-ul** este bara de unelte situată în partea de sus a IDE-ului, care include butoane precum Build, Save și Start/Debug. **Form Designer** reprezintă zona de creare a interfeței grafice (GUI), unde se pot plasa controale precum butoane, etichete, căsuțe de text și se pot modifica proprietățile acestora (culoare, text, font etc.). **Output Window** este fereastra în care sunt afișate rezultatele rulării aplicației și mesajele generate în timpul build-ului.

2.3. Crearea unei aplicații Windows Forms utilizând C#

2.3.1. Dezvoltarea interfeței utilizator

Se deschide fereastra cu controale (Toolbox). Dacă opțiunea nu este vizibilă, aceasta poate fi accesată din bara de meniu, View → Toolbox. Se extinde secțiunea "Common Controls" și se selectează componenta de tip `Button`, iar apoi acesta este adăugat în formular (Figura 9).

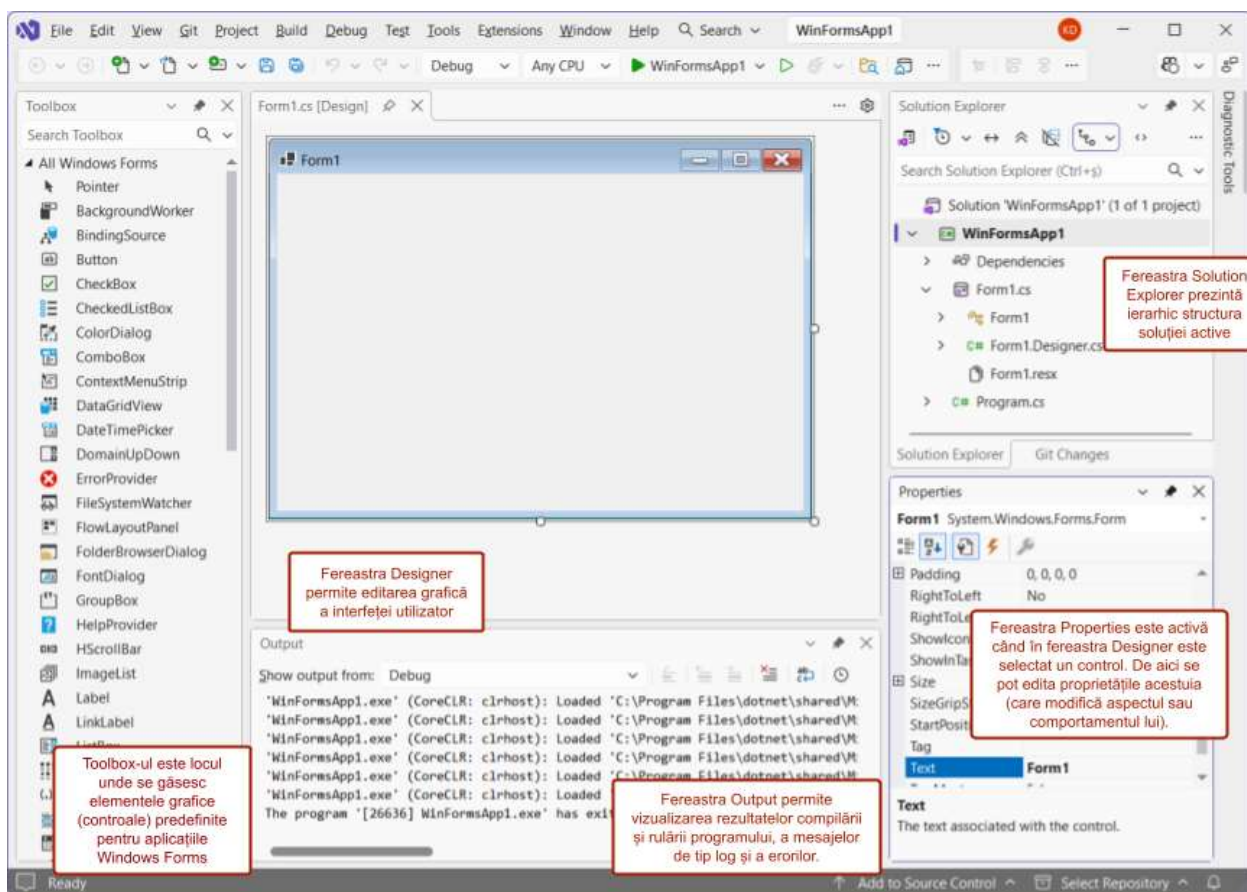


Figura 8 - Componentele IDE-ului Visual Studio pentru Windows Forms App

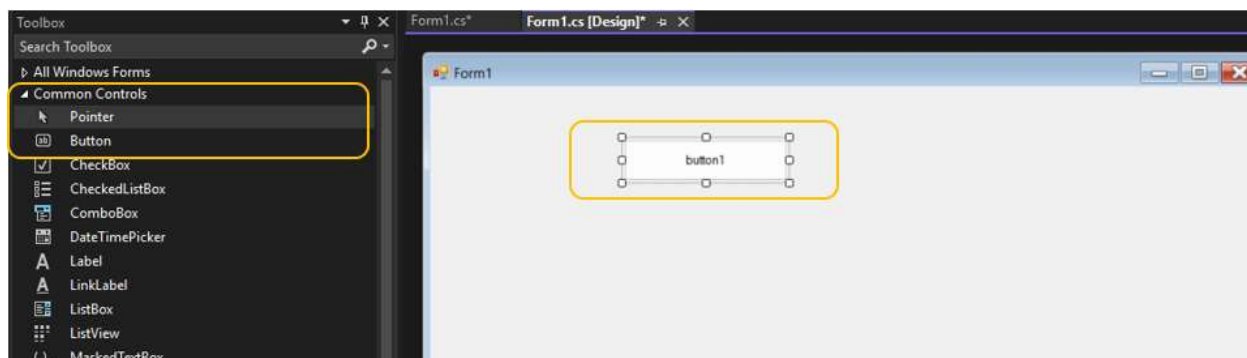


Figura 9 - Selectarea componentei de tip buton din Toolbox și adăugarea acesteia

În fereastra de proprietăți, se identifică proprietatea `Text`, iar valoarea acesteia se modifică din `button1` în `Click this`, după care se apasă Enter (Figura 10). Dacă fereastra de proprietăți nu este vizibilă, aceasta poate fi deschisă din View → Properties sau click dreapta pe buton și selectare `Properties`.

TabStop	True
Tag	
Text	Click this
TextAlign	MiddleCenter
TextImageRelation	Overlay

Figura 10 - Modificarea textului pentru buton

Proprietatea (Name) se modifică din "button1" în "btnClickThis" (Figura 11).

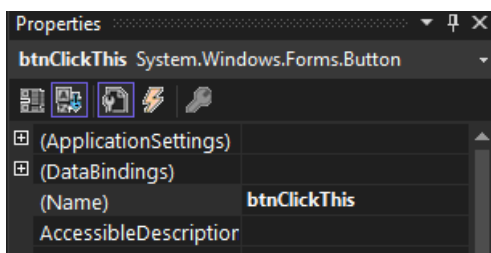


Figura 11 - Modificarea proprietății Name a butonului

După adăugarea butonului, se introduce un control de etichetă (Label) pentru afișarea mesajului (Figura 12). Se selectează eticheta din Toolbox și se adaugă în formular, sub buton. În fereastra de proprietăți, proprietatea "Name" se modifică din "label1" în "lblHelloWorld".



Figura 12 - Adăugarea etichetei în formular

2.3.2. Adăugarea codului

Se efectuează dublu click pe butonul creat sau din Solution Explorer, se poate deschide fișierul "Form1.cs". În metoda generată automat pentru evenimentul Click al butonului, se introduce următoarea instrucțiune (Figura 13):

```
lblHelloWorld.Text = "Hello World!";
```

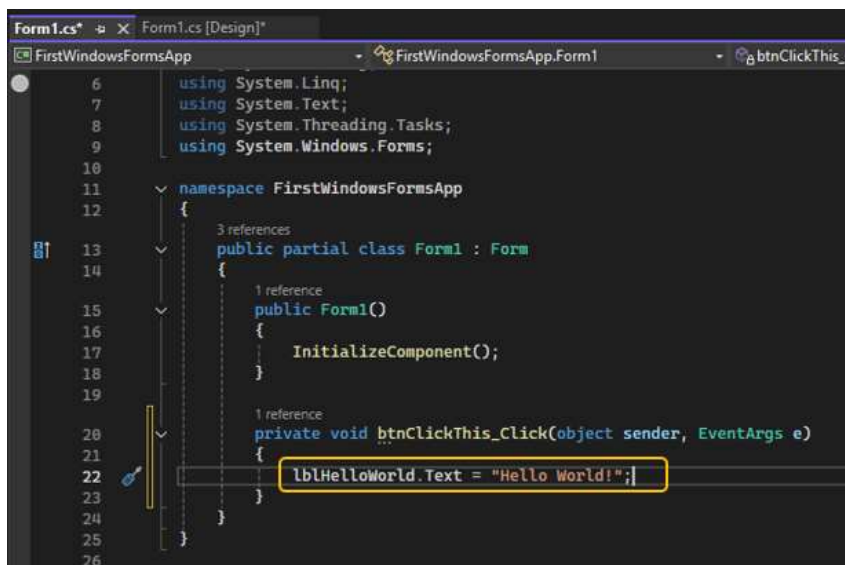


Figura 13 - Adăugarea unei instrucțiuni pentru evenimentul butonului

2.3.3. Rularea aplicației

Se selectează butonul de start (din Toolbar) sau se apasă tasta rapidă F5 pentru a rula aplicația. La apăsarea butonului, textul etichetei se va modifica din "label1" în "Hello World!" (Figura 14).

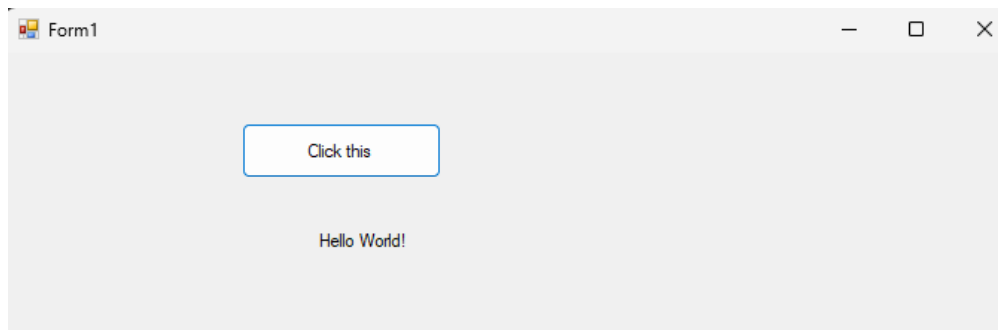


Figura 14 - Rezultatul rulării aplicației

2.4. Structura proiectului Windows Forms

Aplicația Windows Forms este alcătuită din următoarele componente:

- Fișierul `Form1.cs` – reprezintă fișierul principal al formularului și conține logica aplicației;
- Fișierul `Form1.Designer.cs` – fișier generat automat, responsabil pentru definirea și inițializarea interfeței grafice;
- `Form1.resx` – reprezintă fișierul de resurse.

2.4.1. Form1.cs

În "Form1.cs" se implementează comportamentul aplicației, respectiv acțiunile executate la apăsarea unui buton, introducerea unui text sau declanșarea altor evenimente (Figura 15).

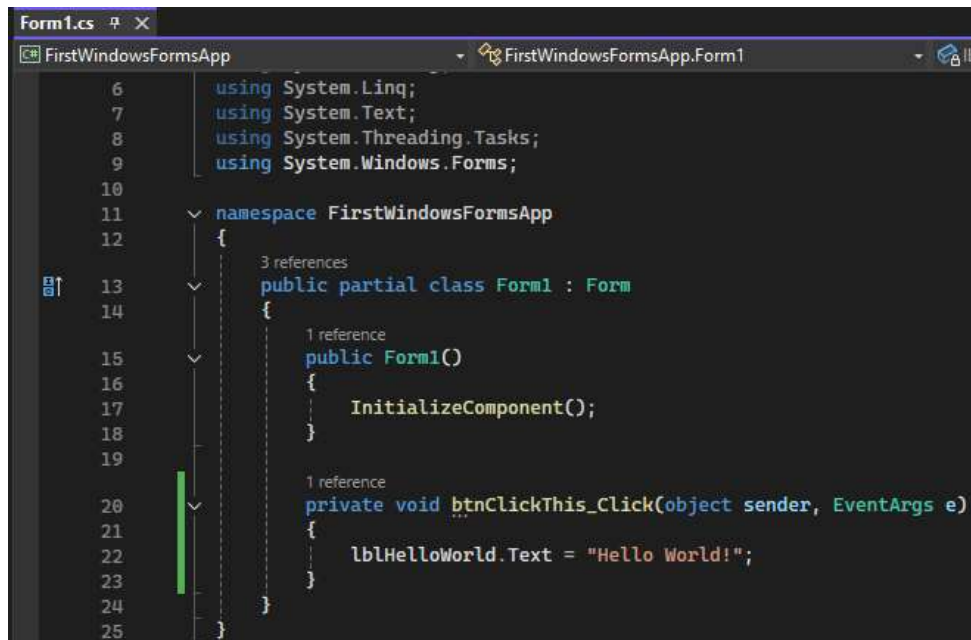
Acesta include:

- Definiția clasei formularului (`public partial class Form1 : Form`);
- Codul scris;
- Metodele asociate evenimentelor;
- Logica de procesare a datelor și interacțiunea cu utilizatorul.

2.4.2. Form1.Designer.cs

Fișierul "Form1.Designer.cs" este generat automat de mediul de dezvoltare și conține codul necesar pentru definirea și inițializarea interfeței grafice (Figura 16). Acesta include:

- Crearea controalelor (Button, Label, CheckBox etc.);
- Setarea numelor de variabilă asociate, a poziției, a dimensiunilor și a altor proprietăți modificate din fereastra Properties.

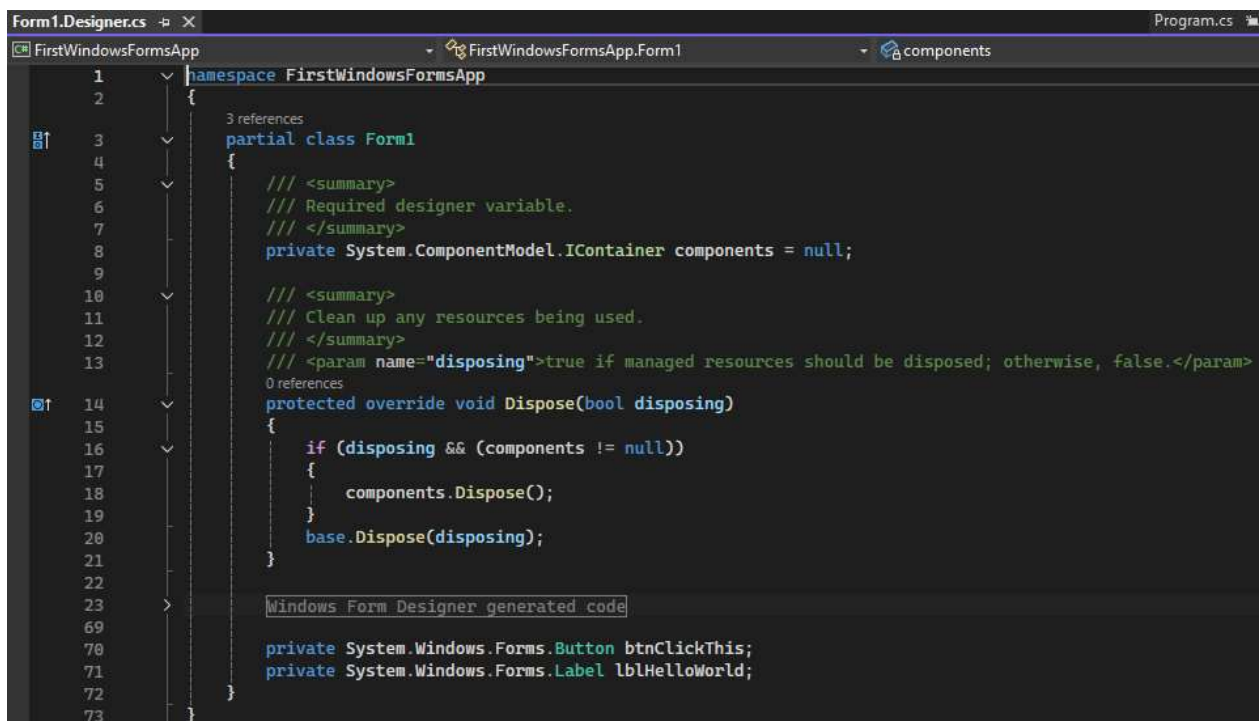


```
Form1.cs
FirstWindowsFormsApp
FirstWindowsFormsApp.Form1
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace FirstWindowsFormsApp
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void btnClickThis_Click(object sender, EventArgs e)
        {
            lblHelloWorld.Text = "Hello World!";
        }
    }
}
```

Figura 15 - Form1.cs



```
Form1.Designer.cs
FirstWindowsFormsApp
FirstWindowsFormsApp.Form1
namespace FirstWindowsFormsApp
{
    partial class Form1
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        Windows Form Designer generated code

        private System.Windows.Forms.Button btnClickThis;
        private System.Windows.Forms.Label lblHelloWorld;
    }
}
```

Figura 16 - Form1.Designer.cs

2.4.3. Form1.resx

Fișierul "Form1.resx" este un fișier de resurse asociat formularului "Form1" într-un proiect Windows Form. Acesta este utilizat pentru stocarea elementelor auxiliare necesare interfeței grafice. Fișierul poate conține:

- Texte utilizate în interfață;
- Imagini (icon-uri, logo-uri);
- Fișiere audio sau alte resurse multimedia.

2.4.4. Fereastra Designer

Prin intermediul "Form Designer" (Figura 17) se pot:

- Adăuga controale din Toolbox;
- Poziționa și redimensiona elementele grafice;
- Configura proprietățile vizuale și comportamentale ale controalelor.

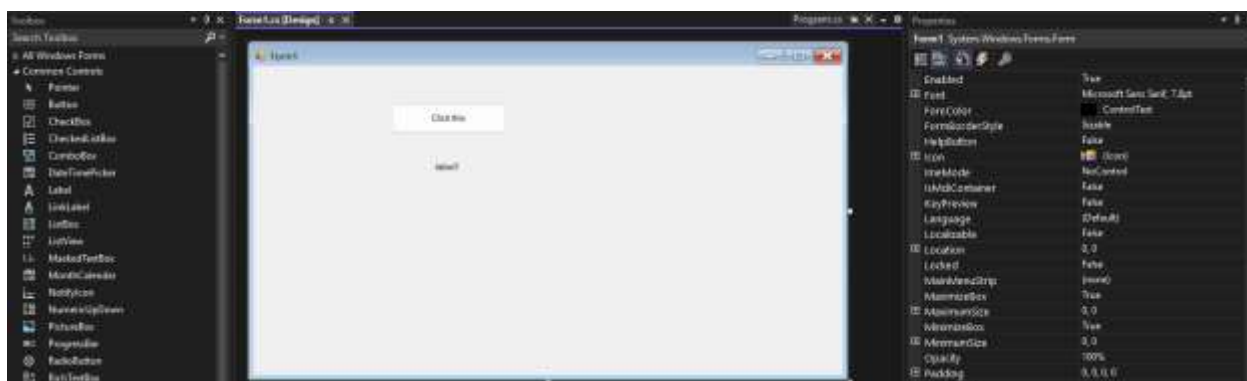


Figura 17 - Form Designer

LABORATORUL 2

3. Proprietățile și evenimentele unui control Windows Forms

3.1. Controale Windows Forms

Panoul de controale (*Toolbox*) din Visual Studio conține mai multe categorii de controale (se mai pot folosi termenii de controale UI, controale grafice sau controale Windows Forms).

Un **control** reprezintă un element grafic al interfeței prin intermediul căruia utilizatorul interacționează.

Panoul de controale grupează controalele în următoarele categorii:

- *Common controls* (Controale comune) - cuprinde controalele cele mai frecvent utilizate;
- *Containers* (Controale container) - cuprinde controalele de organizare a interfeței;
- *Menu&Toolbars* (Meniuri și bare de instrumente) - cuprinde controalele de tip meniu sau bară de unelte;
- *Data* (Date) - cuprinde controalele pentru afișarea și manipularea datelor;
- *Components* (Componente) – cuprinde controalele non-vizuale, cele care nu apar direct pe formular, dar care oferă funcționalități suplimentare;
- *Printing* (Tipărire) - cuprinde controalele care ajută la tipărirea documentelor sau elementelor;
- *Dialogs* (Ferestre de dialog standard) - cuprinde ferestre predefinite care permit interacțiuni rapide cu utilizatorul.

3.1.1. Common Controls (Controale comune)

Tabel 1. Controale comune

Tip de control	Descriere
Button (Buton)	Permite utilizatorului să execute o acțiune prin click.
CheckBox (Casetă de bifare)	Permite utilizatorului selectarea sau deselectarea unei opțiuni.

Tip de control	Descriere
CheckedListBox (Listă cu casete de bifare)	Afișează o listă de elemente vizibile simultan, dar fiecare element dispune de o casetă de bifare.
ComboBox (Listă derulantă)	Afișează un singur element vizibil, restul elementelor fiind accesibile prin derulare (dropdown).
DateTimePicker (Selector de dată și oră)	Permite utilizatorului selectarea unei date sau a unei ore dintr-un calendar vizual sau dintr-un câmp numeric.
Label (Etichetă)	Afișează un text static pe formular.
LinkLabel (Etichetă cu link)	Permite utilizatorul accesarea prin click pe text pentru a deschide un link sau pentru a declanșa o acțiune în aplicație.
ListBox (Listă)	Afișează o listă de elemente vizibile simultan.
ListView (Listă detaliată)	Afișează o listă de elemente cu mai multe detalii, coloane și/sau iconițe.
MaskedTextBox (Casetă text cu mască)	Permite utilizatorului introducerea caracterelor care respectă tiparul definit de mască (de exemplu, numere, litere sau combinații).
MonthCalendar (Calendar lunar)	Permite utilizatorului selectarea unei date sau un interval de date.
NotifyIcon (Iconiță de notificare)	Permite aplicațiilor să ruleze în background și să ofere mesaje sau acces rapid prin meniuri contextuale.
NumericUpDown (Selector numeric)	Permite utilizatorului introducerea unei valori numerice fie tastând direct, fie folosind săgețile sus/jos pentru a crește sau a scădea valoarea.
PictureBox (Casetă pentru imagine)	Afișează imagini (ex: .jpg, .png, .bmp) în formular.
ProgressBar (Bară de progres)	Afișează progresul unei operațiuni sau al unei sarcini.
RadioButton (Buton radio)	Permite utilizatorului selectarea unei singure opțiuni dintr-un grup de butoane.
RichTextBox (Casetă de editare cu formatare)	Permite introducerea și afișarea textului formatat (bold, italic, culori, fonturi, hyperlinkuri etc.).
TextBox (Casetă text)	Permite utilizatorului introducerea unui text.

Tip de control	Descriere
ToolTip (Mesaj informativ)	Afișează un mesaj scurt atunci când utilizatorul trece cursorul peste un alt control.
TreeView (Vizualizare în arbore)	Afișează date ierarhice sub formă de arbore, cu noduri părinte și noduri copil, permițând astfel utilizatorului să extindă sau restrângă nodurile pentru a naviga ușor prin structuri complexe.
WebView2 (Navigator web)	Permite utilizatorului navigarea pe internet fără a deschide un browser separat.

3.1.2. Containers (Controale container)

Tabel 2. Controale de tip container

Tip de control	Descriere
FlowLayoutPanel (Panou de aranjare în flux)	Aranjează automat controalele plasate în interiorul său în rânduri sau coloane, unul după altul, în funcție de spațiul disponibil.
GroupBox (Grup de controale)	Grupează controale asociate între ele.
Panel (Panou)	Grupează controale în formular. Spre deosebire de GroupBox, Panel nu are titlu vizual implicit, dar poate fi folosit pentru organizarea și aranjarea controalelor.
SplitContainer (Panou divizat)	Cuprinde două panouri (Panel1 și Panel2) separate printr-un separator.
TabControl (Control tip tab)	Organizează conținutul pe mai multe file (tab-uri). Fiecare tab poate conține alte controale independente permițând astfel utilizatorului să navigheze între secțiuni fără a încărca mai multe ferestre sau formulare.
TableLayoutPanel (Panou tabelar)	Organizează controalele într-un tabel cu rânduri și coloane. Fiecare celulă poate conține unul sau mai multe controale, iar rândurile/coloanele pot avea dimensiuni fixe, procentuale sau automate.

3.2. Proprietățile unui control

În contextul Windows Forms, proprietățile unui control reprezintă atributele sau caracteristicile acestuia, care definesc aspectul, comportamentul și starea sa pe form (Figura 18). Fiecare control dispune de propriile proprietăți specifice, însă există și proprietăți comune pe care majoritatea controalelor, acestea fiind cele legate de aspectul vizual: text, tipul și dimensiunea fontului, culoarea textului, culoarea de fundal, poziția controlului pe formular, dimensiunea controlului.

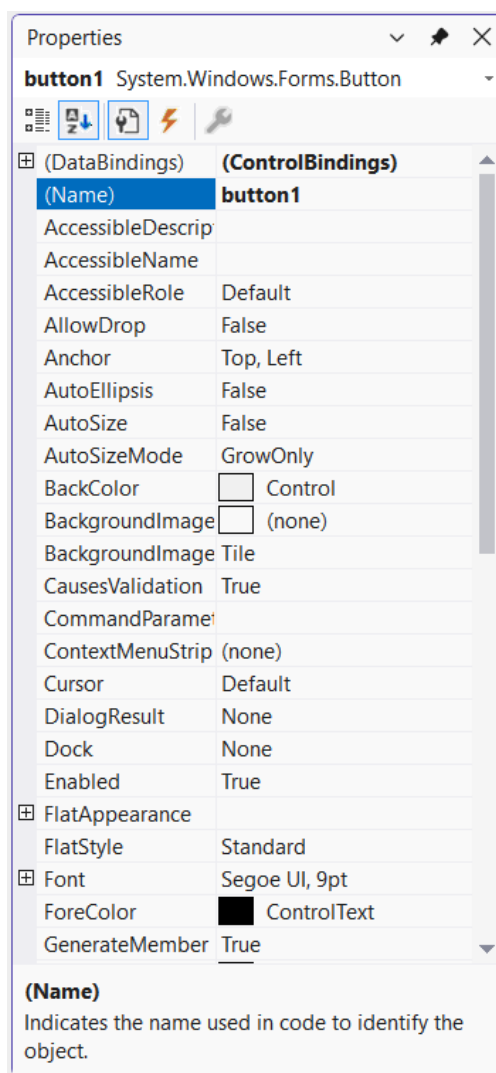


Figura 18 - O parte din proprietățile unui buton

Observații:

1. Poziția fiecărui control plasat pe formular se stabilește în funcție de axa xOy, al cărei punct de origine se află în colțul din stânga sus al formularului. Coordonata x crește spre dreapta, iar y crește în jos. Astfel, fiecare control își are poziția definită prin coordonatele sale pe aceste axe. Dimensiunea controlului se stabilește, de asemenea

din colțul stânga sus al controlului, prin definirea lățimii (`width`) și înălțimi (`height`) acestuia. De exemplu, Figura 19 prezintă un buton adăugat în formular, a cărui poziție generată automat la momentul amplasării este de 220 pe axa X și 132 pe axa Y. Valorile poziției și dimensiunii controlului pot fi modificate de către utilizator în orice moment, fie direct din formular, fie din fereastra de proprietăți a acestuia.

2. Proprietățile unui control pot fi modificate fie în fișierul ".cs" asociat form-ului, prin cod, fie direct în fereastra de proprietăți a acestuia.

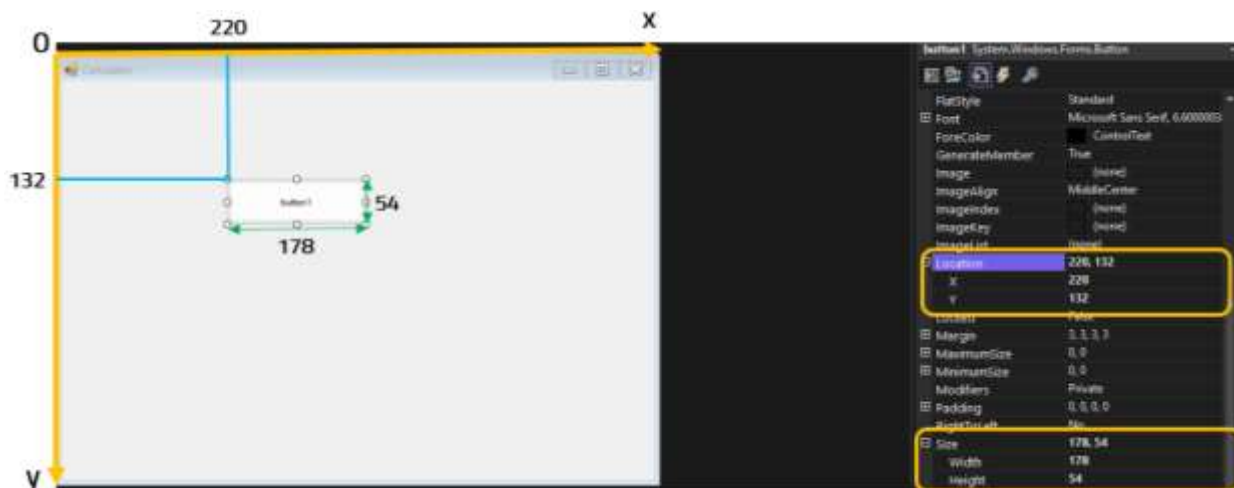


Figura 19 - Poziția și dimensiunea unui buton

3.3. Evenimentele unui control

Un control poate dispune de o varietate de evenimente, acestea permițând programului să reacționeze la acțiunile utilizatorului sau la modificări ale controlului. Câteva exemple de evenimente ar fi:

- Evenimente pentru butoane: `Click` (apăsare buton), `MouseHover` (mouse peste buton);
- Evenimente pentru formular: `Load` (încărcarea formularului).

4. Tipuri de date

În limbajul C#, tipurile de date se împart în două categorii principale:

- Tipuri valoare (*value types*): stochează direct valoarea (ex: tipuri numerice, `bool`, `char`, `struct`, `enum`);
- Tipuri referință (*reference types*): stochează o referință către o instanță a obiectului (ex: `class`, `string`, `array-uri`).

Limbajul C# definește următoarele tipuri de date valoare:

- Tipuri întregi: `byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `long` și `ulong`;
- Tipuri reale: `float`, `double` și `decimal`;
- Tipul caracter: `char`;
- Tipul logic: `bool`.

4.1. CTS

În ecosistemul .NET, CTS (*Common Type System*) este componenta din *Common Language Runtime* care definește toate tipurile de date și regulile lor de utilizare. Scopul său este să asigure interoperabilitatea între limbaje diferite din .NET (de exemplu, C#, VB.NET), astfel încât ele să poată folosi aceleași tipuri și să comunice corect între ele.

CTS stabilește:

- tipurile de bază (ex: `int`, `string`, `object`)
- diferența dintre *value types* (structuri) și *reference types* (obiecte)
- reguli de moștenire, vizibilitate și compatibilitate

Limbajele .Net definesc sinonime pentru acestea.

4.2. Tipuri întregi

Numerele întregi sunt valori fără zecimale, pozitive sau negative. În C#, există mai multe tipuri de numere întregi, cu dimensiuni și intervale diferite, pentru a se potrivi diferitelor situații.

Tabel 3. Tipuri de numere întregi în C#. [2]

Tipul C#	Tipul .NET (CTS)	Octeți alocați	Acoperirea
<code>byte</code>	<code>Byte</code>	1	0 până la 255
<code>sbyte</code>	<code>SByte</code>	1	-128 până la 127
<code>short</code>	<code>Int16</code>	2	-32768 până la 32767
<code>ushort</code>	<code>UInt16</code>	2	0 până la 65535
<code>int</code>	<code>Int32</code>	4	-2147483648 până la 2147483647
<code>uint</code>	<code>UInt32</code>	4	0 până la 4294967295
<code>long</code>	<code>Int64</code>	8	-9223372036854775808 până la 9223372036854775807
<code>ulong</code>	<code>UInt64</code>	8	0 până la 18446744073709551615

4.2.1. Tipurile *byte* și *sbyte*

- *byte* – este folosit pentru valori mici și pozitive;
- *sbyte* – este folosit pentru valori mici care pot fi și negative.

4.2.2. Tipurile *short* și *ushort*

- *short* – este folosit pentru valori întregi;
- *ushort* – este folosit pentru valori pozitive.

4.2.3. Tipurile *int* și *uint*

- *int* – cel mai frecvent tip pentru numere întregi;
- *uint* – este folosit pentru numere întregi mari pozitive.

4.2.4. Tipurile *long* și *ulong*

- *long* – este folosit pentru valori foarte mari care depășesc intervalul *int*;
- *ulong* – este folosit pentru valori foarte mari pozitive.

4.3. Tipuri reale

Numerele reale sunt numere cu zecimale, adică valori care pot avea parte întreagă și parte fracționară. Toate au reprezentare cu semn.

Tabel 4. Tipuri de numere reale în C#. [2]

Tipul C#	Tipul .NET (CTS)	Octeți alocați	Acoperirea
<code>float</code>	<code>Single</code>	4	Aproximativ: 6 - 7 zecimale
<code>double</code>	<code>Double</code>	8	Aproximativ: 15 zecimale
<code>decimal</code>	<code>Decimal</code>	16	Aproximativ: 28 - 29 zecimale

- `float` – este folosit pentru calcul rapid cu zecimale, grafice, jocuri;
- `double` – este folosit pentru calcul general, științific, mai precis decât `float`;
- `decimal` – este folosit pentru calcule unde precizia este foarte importantă.

4.4. Tipul caracter

Tipul caracter (`char`) este un tip de date primitiv care stochează un singur caracter Unicode. Dimensiunea sa este de 2 octeți (16 biți) și poate stoca litere, cifre, simboluri sau caractere speciale care se scriu între apostrofuri (' ').

Următorul tabel prezintă cele mai folosite secvențe escape:

Tabel 5. Secvențe escape.

Secvența escape	Semnificație
<code>\n</code>	Linie nouă
<code>\t</code>	Tab
<code>\\</code>	Backslash
<code>\'</code>	Apostrof
<code>\"</code>	Ghilimele

4.5. Tipul logic

Tipul logic (`bool`) este un tip de date primitiv care stochează o valoare logică. Dimensiunea sa este de 1 octet și poate stoca stările de `true` (adevărat) sau `false` (fals).

4.6. Conversia valorilor între diferite tipuri de date

4.6.1. Parse

Este folosit pentru a converti un șir de caractere (`string`) într-un tip numeric sau logic.

Exemplu:

```
string text = "536";  
int numar = int.Parse(text);    // text -> int
```

4.6.2. Convert

`Convert` este mai sigur și mai flexibil. Poate converti între tipuri diferite, nu doar șiruri în numere. Dacă conversia nu este posibilă, metodele aruncă o excepție (`FormatException` sau `InvalidCastException`).

Exemplu:

```
string text = "536";  
int numar = Convert.ToInt32(text);    // text -> int
```

4.6.3. TryParse

`TryParse` este o metodă utilizată pentru conversia sigură a unui șir (`string`) într-un tip de date numeric sau structural, fără a genera excepții în cazul în care conversia eșuează.

Metoda returnează `true` dacă conversia reușește sau `false` dacă nu reușește, iar rezultatul este furnizat printr-un parametru `out`.

```
string text = "536";  
if (int.TryParse(text, out int numar))  
{  
    Console.WriteLine(numar);  
}
```

`TryParse` evită utilizarea excepțiilor (`FormatException`), ceea ce:

- îmbunătățește performanța
- simplifică validarea inputului utilizatorului

4.6.4. *ToString*

Convertește o valoare într-un șir de caractere. Este metoda standard pentru a afișa valorile sau pentru a le folosi ca text. Toate tipurile referință moștenesc această funcție membră din clasa `object` (care este clasa de bază a tuturor claselor în limbajul C#).

Exemplu:

```
int numar = 23;  
string text = numar.ToString(); // int -> text
```

4.7. Variabile

În timpul execuției unui program, datele sunt stocate temporar în memorie. O variabilă este numele atribuit unei locații de memorie care stochează un anumit tip de date. Astfel, fiecărei variabile îi este asociat un tip de date și o valoare. În C#, variabilele se declară astfel:

```
<tipul de date> <variabila>;
```

De exemplu:

```
char a = 'd';
```

4.8. Constante

Constantele sunt variabile ale căror valori, odată definite, nu pot fi schimbate de acest program. Variabilele constante sunt declarate folosind cuvântul-cheie *const*, cum ar fi:

```
const double PI = 3.14159;
```

4.9. Operatori

4.9.1. Operatori aritmetici

Operatorii aritmetici sunt folosiți pentru calcul numeric.

Tabel 6. Operatori aritmetici în C# [2]

Operator	Operație
+	Adunare
-	Scădere
*	Înmulțire
/	Împărțire (câtul împărțirii întregi, dacă operanzii sunt numere întregi)
%	Restul împărțirii
++	Incrementare
--	Decrementare

4.9.2. Operatori de atribuire

Operatorii de atribuire sunt folosiți pentru stocarea valorilor în variabile.

Tabel 7. Operatori de atribuire în C#

Operator	Descriere
=	Atribuire simplă
+=	Incrementare prin adunare și atribuire
--	Decrementare prin scădere și atribuire
*=	Înmulțire cu actualizarea variabilei
/=	Împărțire cu actualizarea variabilei
%=	Restul împărțirii cu actualizarea variabilei

Se pot folosi și alți operatori în combinație cu operatorul = (de exemplu cei la nivel de bit).

4.9.3. Operatori relaționali

Operatorii relaționali sunt folosiți pentru comparații.

Tabel 8. Operatori relaționali în C#

Operator	Descriere
==	Egalitate
!=	Diferit
>	Mai mare
<	Mai mic
>=	Mai mare sau egal
<=	Mai mic sau egal

4.9.4. Operatori logici

Operatorii logici sunt folosiți pentru conjuncții și disjuncții logice.

Tabel 9. Operatori logici în C#

Operator	Descriere
&&	ȘI logic
	SAU logic
!	Negare

4.9.5. Operatori bitwise

Operatorii *bitwise* lucrează la nivel de bit.

Tabel 10. Operatori bitwise în C#

Operator	Descriere
&	ȘI la nivel de bit
	SAU la nivel de bit
^	Sau exclusiv (XOR)
~	Negare pe biți
<<	Deplasare la stânga
>>	Deplasare la dreapta (bitul de semn este propagat la noul MSB)
>>>	Deplasare la dreapta (noul MSB este 0)

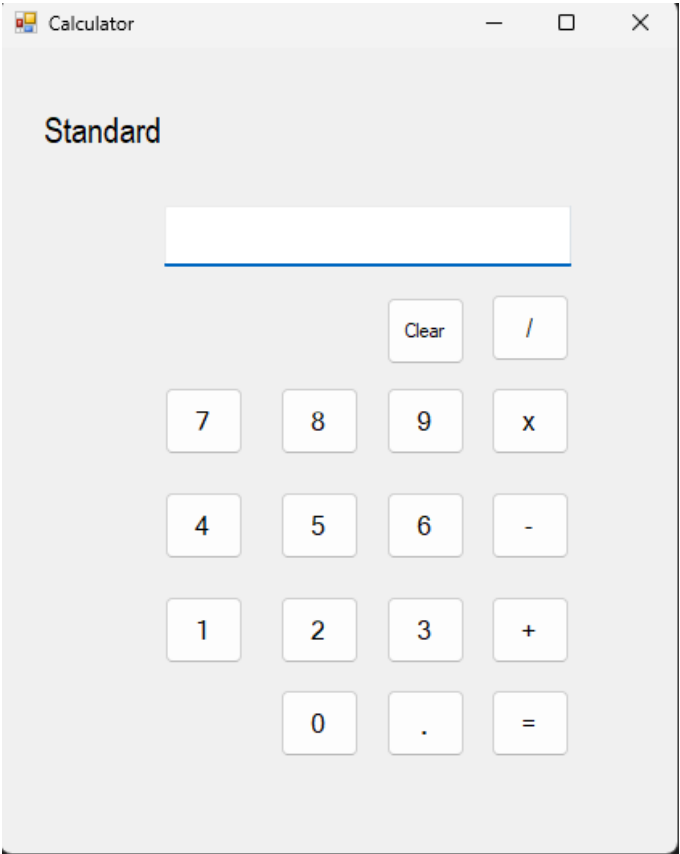
4.9.6. Operatori speciali

Tabel 11. Operatori speciali în C#

Operator	Descriere
?:	Operator ternar – înlocuiește un if-else simplu pentru calculul unei expresii
??	Alege prima valoare care nu este null
is	Verifică tipul unui obiect
as	Convertește un obiect la un anumit tip. Dacă nu reușește returnează "null".

4.10. Aplicație cu operatori aritmetici (Calculator)

Form1.cs [Design]



Proprietăți și evenimente pentru controale:

Tip control	Nume (Name)	Setări relevante	Rol
TextBox	tbAfisaj	-	Afișează cifrele numerelor și rezultatele.

Tip control	Nume (Name)	Setări relevante	Rol
Button	btnNumeric<n>, n=0..9	Text = "<n>", n=0..9 Click → buttonNumeric_Click	Permite utilizatorului să introducă cifrele numerelor.
Button	btnOperator<S>, S = Plus, Minus, Inmultire, Impartire	Text = "S", S=(+, -, x, /) Click → buttonOperator_Click	Permite utilizatorului să aleagă operația dorită.
Button	buttonEqual	Text = "=" Click → buttonEqual_Click	Declanșează efectuarea calculului și afișarea rezultatului.
Button	buttonDot	Text = "." Click → buttonDot_Click	Permite utilizatorului să introducă separatorul zecimal.
Button	buttonClear	Text = "Clear" Click → buttonClear_Click	Declanșează resetarea calculatorului.

Form1.cs

```
using System;
using System.Windows.Forms;

namespace CalculatorApp
{
    public partial class Form1 : Form
    {
        // Primul număr introdus
        double nr = 0;

        // Operatorul ales (+, -, x, /)
        string operationType = "";

        // Indică dacă tocmai s-a ales o operație
        // și următorul număr trebuie introdus de la zero
        bool operationStatus = false;

        public Form1()
        {
            InitializeComponent();

            // La încărcarea form-ului se poate executa cod: vom folosi
            // evenimentul Load al Form1 pentru a inițializa căsuța textBox1
            private void Form1_Load(object sender, EventArgs e)
            {
                // valoarea inițială din textbox
                tbAfisaj.Text = "0";
            }
        }
    }
}
```

```
// =====  
// BUTOANE NUMERICE (0-9)  
// =====  
// Toate butoanele numerice folosesc același handler  
// Se va asocia evenimentului Click al butoanelor numerice  
// același handler de eveniment  
private void buttonNumeric_Click(object sender, EventArgs e)  
{  
    // Conversia lui sender într-un obiect de tip Button  
    // pentru a putea fi accesate proprietatile butonului, cum ar fi Text  
    Button btn = (Button)sender;  
  
    // Dacă textbox-ul este 0 sau tocmai s-a ales o operație  
    // se începe introducerea unui număr nou  
    if (tbAfisaj.Text == "0" || operationStatus)  
    {  
        // Textul din tbAfisaj devine textul butonului apasat (btn.Text).  
        // si operationStatus se resetează la false pentru a permite  
        // introducerea următoarei cifre  
        tbAfisaj.Text = btn.Text;  
        operationStatus = false;  
    }  
    else  
    {  
        // altfel cifra se concatenează la final  
        tbAfisaj.Text += btn.Text;  
    }  
}  
  
// =====  
// OPERAȚII ARITMETICE (+ - x /)  
// =====  
// Se va asocia evenimentului Click al butoanelor cu operații  
// același handler de eveniment  
private void buttonOperator_Click(object sender, EventArgs e)  
{  
    Button btn = (Button)sender;  
  
    // Textul din tbAfisaj (numărul introdus până acum) este convertit în  
    // double și apoi este stocat în variabila nr.  
    nr = Convert.ToDouble(tbAfisaj.Text);  
  
    // Memorăm operatorul selectat  
    operationType = btn.Text;  
  
    // Se semnalează ca o operație a fost aleasă și ca la  
    // următoarea apăsare de cifră, în textBox1  
    // textul trebuie înlocuit, nu concatenat  
    operationStatus = true;  
}  
  
// =====  
// BUTONUL "="  
// =====  
private void buttonEqual_Click(object sender, EventArgs e)  
{  
    double secondNumber = Convert.ToDouble(tbAfisaj.Text);  
    double result = 0;
```

```
// Calculul rezultatului în funcție de operator
switch (operationType)
{
    case "+":
        result = nr + secondNumber;
        break;
    case "-":
        result = nr - secondNumber;
        break;
    case "x":
        result = nr * secondNumber;
        break;
    case "/":
        // Verificare împărțire la zero
        if (secondNumber == 0)
        {
            MessageBox.Show("Impartirea la zero nu este permisa!");
            return;
        }
        result = nr / secondNumber;
        break;
}

// Afișarea rezultatului
tbAfisaj.Text = result.ToString();

// Resetarea stării
operationStatus = true;
}

// =====
// BUTONUL "." (separator zecimal)
// =====
private void buttonDot_Click(object sender, EventArgs e)
{
    // Dacă s-a ales o operație, se începe un număr nou
    if (operationStatus)
    {
        tbAfisaj.Text = "0.";
        operationStatus = false;
        return;
    }

    // Adăugăm punctul zecimal doar dacă nu există deja
    if (!tbAfisaj.Text.Contains("."))
    {
        tbAfisaj.Text += ".";
    }
}

// =====
// BUTONUL CLEAR
// =====
private void buttonClear_Click(object sender, EventArgs e)
{
    // Resetarea valorii din textbox
    tbAfisaj.Text = "0";

    // Resetarea variabilelor interne
    nr = 0;
}
```

```
        operationType = "";  
        operationStatus = false;  
    }  
}  
}
```

De obicei se preferă separarea logicii aplicației de interfață grafică, pentru a fi mai ușor de înțeles și de modificat. Astfel, aplicația poate fi îmbunătățită și scrisă astfel:

CalculatorEngine.cs

```
using System;  
  
namespace CalculatorApp  
{  
    /// <summary>  
    /// Motorul calculatorului: nu știe nimic despre UI (TextBox, Button etc.).  
    /// Primește comenzi (digit, dot, operator, equal, clear)  
    /// și produce textul de afișat.  
    /// </summary>  
    public class CalculatorEngine  
    {  
        // operandul din stânga (memorăm când se apasă + - x /)  
        private double _firstNumber = 0;  
        // operatorul ales  
        private string _op = "";  
        // după ce se alege operatorul, următoarele cifre încep un număr nou  
        private bool _waitingSecond = false;  
        // după "=", următoarea cifră înlocuiește rezultatul  
        private bool _justCalculated = false;  
        public string Display { get; private set; } = "0";  
  
        /// <summary>Reset total.</summary>  
        public void Clear()  
        {  
            Display = "0";  
            _firstNumber = 0;  
            _op = "";  
            _waitingSecond = false;  
            _justCalculated = false;  
        }  
  
        /// <summary>  
        /// Apăsare cifră (0-9). Actualizează Display.  
        /// </summary>  
        public void InputDigit(char digit)  
        {  
            if (digit < '0' || digit > '9') return;  
  
            // Când începe un număr nou (după operator sau după "="),  
            // se înlocuiește display-ul.  
            if (Display == "0" || _waitingSecond || _justCalculated)  
            {  
                Display = digit.ToString();  
                _waitingSecond = false;  
            }  
        }  
    }  
}
```

```
        _justCalculated = false;
        return;
    }

    // Altfel se concatenează cifra la final.
    Display += digit;
}

/// <summary>
/// Apăsare separator zecimal. În UI se va folosi '.' ca simbol.
/// </summary>
public void InputDot()
{
    if (_waitingSecond || _justCalculated)
    {
        Display = "0.";
        _waitingSecond = false;
        _justCalculated = false;
        return;
    }

    // Se evită două sau mai multe puncte zecimale.
    if (!Display.Contains("."))
        Display += ".";
}

/// <summary>
/// Apăsare operator: +, -, x, /.
/// Memorează operandul curent și operatorul,
/// pregătind introducerea celui de-al doilea număr.
/// </summary>
public void SetOperator(string op)
{
    if (op is not ("+" or "-" or "x" or "/" or "*")) return;

    // Dacă utilizatorul apasă operator de mai multe ori la rând,
    // doar actualizăm operatorul (nu reconvertim/recalculăm).
    if (_waitingSecond)
    {
        _op = NormalizeOp(op);
        return;
    }

    _firstNumber = ReadDisplayAsDouble();
    _op = NormalizeOp(op);

    _waitingSecond = true;
    _justCalculated = false;
}

/// <summary>
/// Apăsare "=". Calculează folosind _firstNumber,
/// operatorul și operandul curent (Display).
/// </summary>
public void Equal()
{
    if (string.IsNullOrEmpty(_op))
        return; // nimic de calculat
}
```

```

        double secondNumber = ReadDisplayAsDouble();
        double result;

        switch (_op)
        {
            case "+":
                result = _firstNumber + secondNumber;
                break;
            case "-":
                result = _firstNumber - secondNumber;
                break;
            case "x":
                result = _firstNumber * secondNumber;
                break;
            case "/":
                if (secondNumber == 0)
                    throw new
                        DivideByZeroException("Împărțirea la zero nu este permisă.");
                result = _firstNumber / secondNumber;
                break;
            default:
                return;
        }

        Display = result.ToString();

        // Pregătim următoarea intrare:
        // rezultatul devine primul număr implicit.
        _firstNumber = result;
        _op = "";
        _waitingSecond = false;
        _justCalculated = true;
    }

    // ----- Helpers -----

    private static string NormalizeOp(string op) => op == "*" ? "x" : op;

    /// <summary>
    /// Conversie controlată fără TryParse:
    /// - acceptăm atât '.' cât și ',' în Display
    /// (în caz că UI sau userul a introdus alt simbol)
    /// - convertim în format compatibil cu
    /// Convert.ToDouble pentru cultura curentă
    /// </summary>
    private double ReadDisplayAsDouble()
    {
        string s = (Display ?? "0").Trim();
        return Convert.ToDouble(s);
    }
}

```

Codul din Form1.cs devine:

Form1.cs

```
using System;
using System.Windows.Forms;

namespace CalculatorApp
{
    public partial class Form1 : Form
    {
        private readonly CalculatorEngine _engine = new CalculatorEngine();

        public Form1()
        {
            InitializeComponent();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            // Afișăm valoarea inițială furnizată de engine
            tbAfisaj.Text = _engine.Display;
        }

        // Butoane numerice 0-9 (toate legate la acest handler)
        private void buttonNumeric_Click(object sender, EventArgs e)
        {
            Button btn = (Button)sender;

            // presupunem că Text este "0"... "9"
            char digit = btn.Text[0];

            _engine.InputDigit(digit);
            tbAfisaj.Text = _engine.Display;
        }

        // Butoane operații (+ - x /) (toate legate la acest handler)
        private void buttonOperatie_Click(object sender, EventArgs e)
        {
            Button btn = (Button)sender;

            _engine.SetOperator(btn.Text);
            tbAfisaj.Text = _engine.Display;
        }

        // Buton "."
        private void buttonDot_Click(object sender, EventArgs e)
        {
            _engine.InputDot();
            tbAfisaj.Text = _engine.Display;
        }

        // Buton "="
        private void buttonEqual_Click(object sender, EventArgs e)
        {
            try
            {
                _engine.Equal();
                tbAfisaj.Text = _engine.Display;
            }
        }
    }
}
```

```
        catch (DivideByZeroException ex)
        {
            MessageBox.Show(ex.Message, "Eroare", MessageBoxButtons.OK,
                            MessageBoxIcon.Error);
        }
    }

    // Buton "Clear"
    private void buttonClear_Click(object sender, EventArgs e)
    {
        _engine.Clear();
        tbAfisaj.Text = _engine.Display;
    }
}
```

4.11. Exerciții

4.11.1. Calculul ariei unui dreptunghi

Realizați o aplicație Windows Forms care permite calculul ariei unui dreptunghi.

Aplicația va conține două câmpuri de introducere (TextBox) pentru lungime și lățime, precum și un buton *Calculează*. La apăsarea butonului, aplicația va calcula aria dreptunghiului utilizând formula:

$$A = L \times l$$

Rezultatul va fi afișat într-un control Label sau TextBox (în mod ReadOnly).

Cerințe:

- utilizarea controalelor TextBox, Label și Button;
- conversia valorilor introduse de utilizator în tip numeric;
- afișarea rezultatului calculului în interfața aplicației.

4.11.2. Conversia temperaturii

Realizați o aplicație Windows Forms care permite conversia temperaturii între grade Celsius și Fahrenheit.

Aplicația va conține un câmp de introducere pentru valoarea temperaturii, două butoane pentru conversie (*Celsius* → *Fahrenheit* și *Fahrenheit* → *Celsius*) și un control pentru afișarea rezultatului.

Formulele utilizate sunt:

$$F = \frac{9}{5} \times C + 32 ; C = \frac{5}{9} \times (F - 32)$$

Cerințe:

- utilizarea controalelor `TextBox`, `Button` și `Label`;
- implementarea conversiei temperaturii în funcție de butonul selectat;
- afișarea rezultatului în interfața aplicației.

LABORATORUL 3

5. Structuri de control

5.1. Instrucțiuni de decizie / selecție

În C# instrucțiunile de decizie/selecție sunt folosite pentru a lua decizii în funcție de anumite condiții. Ele permit programului să execute un bloc de cod doar dacă o condiție este adevărată, sau să aleagă între mai multe opțiuni.

5.1.1. Instrucțiunea *if*

Instrucțiunea *if* este o instrucțiune de decizie care permite executarea unui bloc de cod numai dacă o condiție este evaluată la valoarea de adevăr *true*. Dacă rezultatul evaluării condiției este *true*, se execută instrucțiunile din blocul *if*. Dacă este evaluat la *false*, instrucțiunile respective nu sunt executate.

Format general:

```
if (<conditie_logica>)  
    [<instructiune_adevarat>];
```

Exemplu:

```
private void button1_Click(object sender, EventArgs e)  
{  
    int nr = int.Parse(textBox1.Text);  
  
    if (nr > 0)  
    {  
        label1.Text = "Numarul este pozitiv";  
    }  
}
```

Fig. 19 prezintă schema logică pentru instrucțiunea *if*.

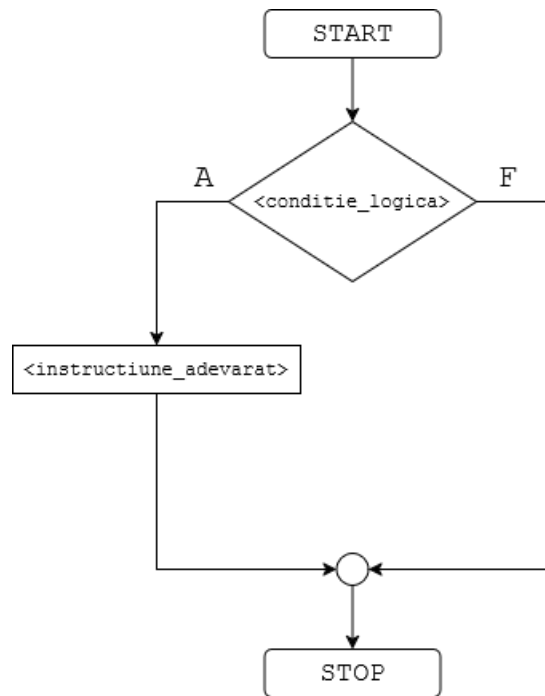


Fig. 1. Schema logică a instrucțiunii "if".

5.1.1.1. Clauza *else*

Clauza *else* face parte din instrucțiunea *if* și se folosește pentru a executa un bloc de instrucțiuni atunci când condiția din *if* este evaluată ca fiind falsă (*false*).

Format general:

```
if (<conditie_logica>)  
    [<instructiune_adevarat>];  
[else  
    [<instructiune_fals>]];
```

Exemplu:

```
private void button1_Click(object sender, EventArgs e)  
{  
    int nr = int.Parse(textBox1.Text);  
  
    if (nr > 0)  
    {  
        label1.Text = "Numarul este pozitiv";  
    }  
    else  
    {  
        label1.Text = "Numarul este negativ sau zero";  
    }  
}
```

Fig. 20 prezintă schema logică pentru instrucțiunea *if else*.

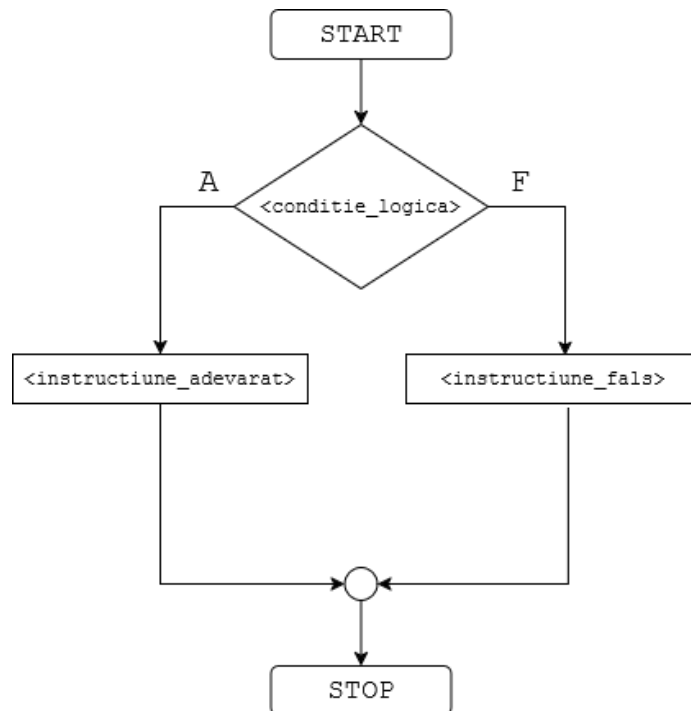


Fig. 2. Schema logică a instrucțiunii "if else".

5.1.1.2. Clauza *else if*

Clauza *else if* se folosește împreună cu instrucțiunea *if* pentru a verifica mai multe condiții succesive.

Format general:

```
if (<conditie_logica_1>)
    <instructiune_adevarat_1>;
else if (<conditie_logica_2>)
    <instructiune_adevarat_2>;
else
    <instructiune_fals>;
```

Exemplu:

```
private void button1_Click(object sender, EventArgs e)
{
    int nota = int.Parse(textBox1.Text);

    if (nota >= 9)
```

```
{
    label1.Text = "Foarte bine";
}
else if (nota >= 7)
{
    label1.Text = "Bine";
}
else if (nota >= 5)
{
    label1.Text = "Suficient";
}
else
{
    label1.Text = "Insuficient";
}
}
```

5.1.1.3. Instrucțiuni *if* imbricate

Instrucțiunile *if* imbricate se folosesc atunci când o decizie depinde de alta. Astfel, un *if* se află în interiorul altui *if*, permițând verificarea mai multor condiții într-un mod structurat.

Format general:

```
if (<conditie_logica>)
    if (<conditie_logica_i>)
        <instructiune_adevarat_i>;
    else
        <instructiune_fals_i>;
else
    <instructiune_fals>;
```

Exemplu:

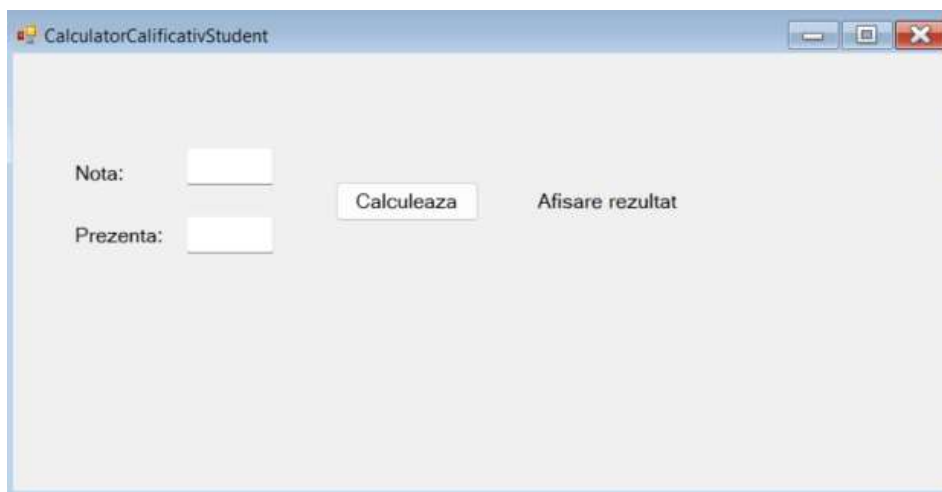
```
private void button1_Click(object sender, EventArgs e)
{
    int nr = int.Parse(textBox1.Text);

    if (nr > 0) // conditie exterioara: numarul este pozitiv
    {
        // conditie interioara: numarul este par
    }
}
```

```
        if (nr % 2 == 0)
        {
            label1.Text = "Numarul este pozitiv si par";
        }
        else
        {
            label1.Text = "Numarul este pozitiv si impar";
        }
    }
    else
    {
        label1.Text = "Numarul nu este pozitiv";
    }
}
```

5.1.1.4. Utilizarea instrucțiunii *if*

Form1.cs [Design]



În Designer, controalele și evenimentele sunt configurate după cum urmează:

- Butonul "Calculeaza" are asociat evenimentul Click += buttonCalculeaza_Click;
- Câmpul "textBoxNota" este utilizat pentru introducerea notei elevului;
- Câmpul "textBoxPrezenta" este utilizat pentru introducerea procentului de prezență;
- Eticheta "labelRezultat" este folosită pentru afișarea calificativului și a mesajului rezultat.

Form1.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CalculatorCalificativStudent
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void buttonCalculeaza_Click(object sender, EventArgs e)
        {
            // Variabila in care se va salva mesajul final afisat utilizatorului
            string mesaj = "";

            // Verificam daca ambele campuri (nota si prezenta) sunt goale
            if (textBoxNota.Text == "" && textBoxPrezenta.Text == "")
            {
                MessageBox.Show("Campurile sunt necompletate!");
                return;
            }

            // Verificam daca campul pentru nota este gol
            if (textBoxNota.Text == "")
            {
                MessageBox.Show("Campul pentru nota nu a fost completat!");
                return;
            }

            // Verificam daca campul pentru prezenta este gol
            if (textBoxPrezenta.Text == "")
            {
                MessageBox.Show("Campul pentru prezenta nu a fost completat!");
                return;
            }

            // Convertirea valorilor string in numere intregi
            int nota = int.Parse(textBoxNota.Text);
            int prezenta = int.Parse(textBoxPrezenta.Text);

            // Verificam daca nota introdusa se afla in intervalul valid (1 - 10)
            if (nota < 1 || nota > 10)
            {
                MessageBox.Show("Nota trebuie sa fie intre 1 si 10!");
                return;
            }

            // Verificam daca procentul de prezenta este in intervalul valid:0-100
            if (prezenta < 0 || prezenta > 100)
            {
                MessageBox.Show("Prezenta trebuie sa fie intre 0 si 100!");
            }
        }
    }
}
```

```
        return;
    }

    // Verificam calificativul in functie de nota obtinuta
    if (nota >= 9)
    {
        // Daca nota este 9 sau 10, calificativul este "Foarte bine"
        mesaj = "Calificativ: Foarte bine";

        if (prezenta >= 90)
        {
            mesaj += " - Felicitari pentru prezenta!";
        }
    }
    else if (nota >= 7)
    {
        // Daca nota este intre 7 si 8
        mesaj = "Calificativ: Bine";

        if (prezenta >= 90)
        {
            mesaj += " - Prezenta foarte buna!";
        }
    }
    else if (nota >= 5)
    {
        // Daca nota este intre 5 si 6
        mesaj = "Calificativ: Suficient";

        if (prezenta >= 90)
        {
            mesaj += " - Prezenta buna!";
        }
    }
    else
    {
        mesaj = "Calificativ: Insuficient";

        if (prezenta >= 90)
        {
            mesaj += " - Prezenta buna, dar nota mica!";
        }
    }

    // Afisarea rezultatului final
    labelRezultat.Text = mesaj;
}
}
```

5.1.2. Instrucțiunea switch

Instrucțiunea `switch` permite alegerea uneia dintre mai multe ramuri de cod în funcție de valoarea unei expresii. Este utilă atunci când există mai multe condiții pe aceeași variabilă, fiind mai simplă decât un *if-else if*.

- Se compară valoarea expresiei cu valorile din fiecare `case`.
- Dacă există o potrivire, se execută codul aferent.

- `break` oprește execuția în interiorul `switch` după ce un `case` a fost executat. **Prezența comenzii `break` este obligatorie.**
- Se poate folosi și `default` pentru cazul în care niciun `case` nu se potrivește.

Format general:

```
switch (<expresie>)
{
    case (<valoare_1>): <instructiuni_1>;
                        break;
    case (<valoare_2>): <instructiuni_2>;
                        break;
    ...
    case (<valoare_n>): <instructiuni_n>;
                        break;
    default: <instructiuni>;
            break;
}
```

Exemplu:

```
private void buttonCalculeaza_Click(object sender, EventArgs e)
{
    int nota = int.Parse(textBoxNota.Text);
    string mesaj = "";

    switch (nota)
    {
        case 5:
            mesaj = "Calificativ: Foarte bine";
            break;
        case 4:
            mesaj = "Calificativ: Bine";
            break;

        case 3:
            mesaj = "Calificativ: Suficient";
            break;
        case 2:
        case 1:
            mesaj = "Calificativ: Insuficient";
            break;
        default:
            mesaj = "Nota invalida! Introduceti valoare 1 - 5.";
    }
}
```

```
        break;
    }

    labelRezultat.Text = mesaj;
}
```

Limbajul C# permite utilizarea și unor relații în case:

Exemplu:

```
DisplayMeasurement(-4);           // Prea joasa.
DisplayMeasurement(5);            // Valoare masurata: 5.
DisplayMeasurement(30);           // Prea mare.
DisplayMeasurement(double.NaN);   // Eroare.

void DisplayMeasurement(double measurement)
{
    switch (measurement)
    {
        case < 0.0:
            Console.WriteLine("Prea joasa.");
            break;
        case > 15.0:
            Console.WriteLine("Prea mare.");
            break;
        case double.NaN:
            Console.WriteLine("Eroare.");
            break;
        default:
            Console.WriteLine($"Valoare          masurata:
{measurement}");
            break;
    }
}
```

5.2. Instrucțiuni iterative

Instrucțiunile iterative, numite și structuri repetitive sau bucle, permit programului să execute același bloc de cod de mai multe ori. Acestea sunt utilizate atunci când o acțiune trebuie repetată până la îndeplinirea unei condiții sau de un număr prestabilit de repetări.

În C# există 4 tipuri de instrucțiuni iterative:

- **for** : Instrucțiune iterativă cu test inițial, cu număr cunoscut de pași;

- `while` : Instrucțiune iterativă cu test inițial, cu număr necunoscut de pași;
- `do..while` : Instrucțiune iterativă cu test final, cu număr necunoscut de pași.
- `foreach` : Instrucțiune iterativă cu test inițial, cu număr cunoscut de pași, utilizată de obicei pentru parcurgerea colecțiilor.

5.2.1. Instrucțiunea *for*

Instrucțiunea `for` este o structură repetitivă utilizată pentru a executa un bloc de cod de un număr determinat de ori. Aceasta este folosită în special atunci când se cunoaște dinainte numărul exact de repetări ale unei operații.

Format general:

```
for
([<initializare>]; [<conditie_continuare>]; [<modificare_stare>])
{
    [<instructiuni>];
}
```

Exemplu:

```
private void btnGenerate_Click(object sender, EventArgs e)
{
    txtOutput.Clear();

    // Buclă for: de la 1 la 10
    for (int i = 1; i <= 10; i++)
    {
        txtOutput.AppendText($"Numărul{i}
{Environment.NewLine}");
    }
}
```

Figura 21 prezintă schema logică pentru instrucțiunea *for*.

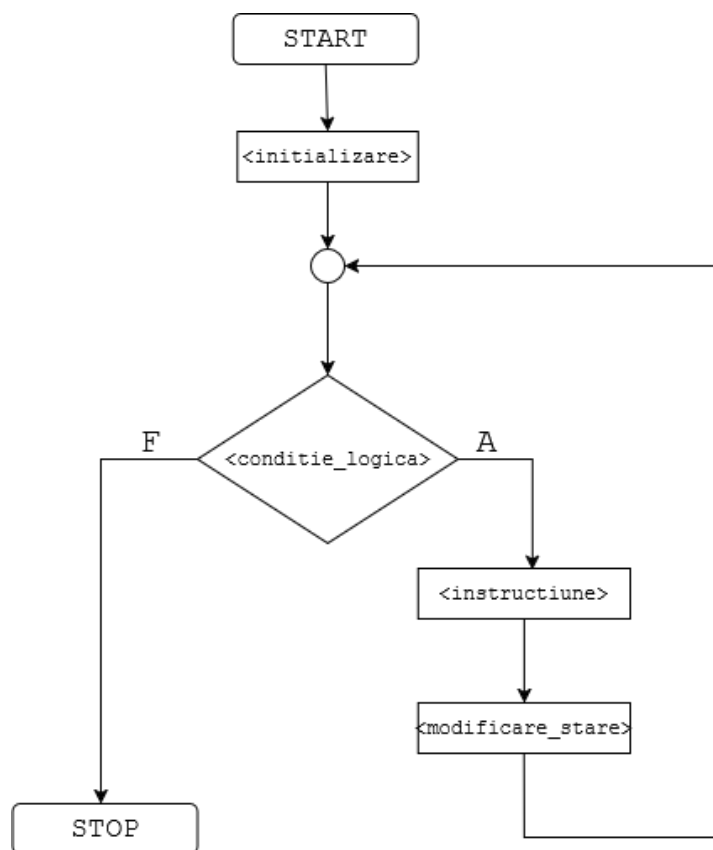


Fig. 3. Schema logică a instrucțiunii "for".

Observații:

1. Instrucțiunea `for` are, în paranteze, trei expresii separate prin caracterul `' ; '`. Toate expresiile sunt opționale, dar caracterele `' ; '` sunt obligatorii. Condiția logică este considerată `true` dacă lipsește. O structură de tipul: `for ( ; ; ) ;` este validă și va genera un ciclu infinit la rulare.
2. După parantezele instrucțiunii `for` nu se pune `' ; '`. Caracterul `' ; '` în această poziție are semnificația instrucțiunii vide, adică instrucțiunea care nu face nimic. Practic această instrucțiune vidă se va executa.

5.2.1.1. Cicluri *for* imbricate

Ciclurile `for` imbricate au loc atunci când o buclă `for` este plasată în interiorul altei bucle `for`, astfel încât bucla interioară se execută complet pentru fiecare iterare a buclei exterioare.

Exemplu:

```
private void btnGenerate_Click(object sender, EventArgs e)
{
    // Curatare TextBox
    txtOutput.Clear();

    int rows = 3; // nr de randuri
    int cols = 3; // nr de coloane

    // Bucla exterioara pentru randuri
    for (int i = 1; i <= rows; i++)
    {
        // Bucla interioara pentru coloane
        for (int j = 1; j <= cols; j++)
        {
            // Afisam coordonatele
            // AppendText - adauga text la sfarsitul textului
            // deja existent în TextBox
            txtOutput.AppendText($"({i},{j})\t");
        }

        // Facem trecerea la un nou rand
        txtOutput.AppendText(Environment.NewLine);
    }
}
```

Rezultatul execuției:

```
(1,1) (1,2) (1,3)
(2,1) (2,2) (2,3)
(3,1) (3,2) (3,3)
```

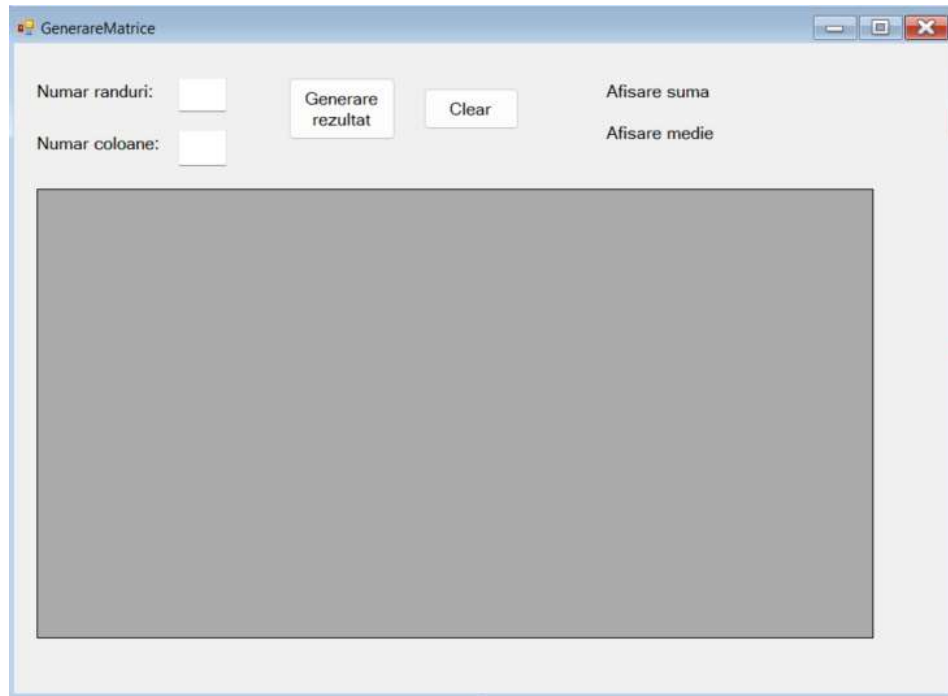
5.2.1.2. Utilizarea instrucțiunii *for*

Form1.cs [Design]

În Designer, controalele și evenimentele sunt configurate după cum urmează:

- Butonul "Generare rezultat" are asociat evenimentul Click += btnGenerate_Click;
- Butonul "Clear" are asociat evenimentul Click += btnClear_Click;
- Câmpul "txtRows" este utilizat pentru introducerea numărului de rânduri;
- Câmpul "txtCols" este utilizat pentru introducerea numărului de coloane;

- Eticheta "Afisare suma" este folosită pentru afișarea sumei elementelor;
 - Eticheta "Afisare medie" este folosită pentru afișarea mediei elementelor;
- Grid-ul "dataGridViewMatrix" este folosit pentru afișarea datelor în format tabelar.



Form1.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ExempluFor
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void btnGenerate_Click(object sender, EventArgs e)
        {
            // -----
            // RESETAREA FUNDALULUI TEXTBOX-URILOR
            // -----
            txtRows.BackColor = Color.White;
            txtCols.BackColor = Color.White;
        }
    }
}
```

```
// -----  
// INITIALIZAREA CODULUI DE EROARE  
// -----  
// Folosim un cod de eroare pentru a gestiona diferite situatii:  
// 0 = totul ok, 1 = ambele campuri goale  
// 2 = input randuri invalid, 3 = input coloane invalid  
int errorCode = 0;  
  
// -----  
// VERIFICARE INPUT  
// -----  
// Daca ambele TextBox-uri sunt goale  
if (string.IsNullOrEmpty(txtRows.Text) &&  
    string.IsNullOrEmpty(txtCols.Text))  
{  
    errorCode = 1;  
}  
// Daca TextBox-ul pentru randuri este gol sau nu contine un nr valid  
else if (string.IsNullOrEmpty(txtRows.Text) ||  
    !int.TryParse(txtRows.Text, out int rows) || rows <= 0)  
{  
    errorCode = 2;  
}  
// Daca TextBox-ul pentru coloane este gol sau nu contine un nr valid  
else if (string.IsNullOrEmpty(txtCols.Text) ||  
    !int.TryParse(txtCols.Text, out int cols) || cols <= 0)  
{  
    errorCode = 3;  
}  
  
switch (errorCode)  
{  
    case 0:  
        // Totul este ok, continuare generare matrice  
        break;  
  
    case 1:  
        // Ambele campuri sunt goale  
        // Fundal rosu deschis pentru randuri  
        txtRows.BackColor = Color.MistyRose;  
        // Fundal rosu deschis pentru coloane  
        txtCols.BackColor = Color.MistyRose;  
        // Afisare mesaj  
        MessageBox.Show("Ambele campuri sunt goale!");  
        // Setarea cursorului pe TextBox-ul pentru randuri  
        txtRows.Focus();  
        return;  
  
    case 2:  
        // Verificam campul pentru randuri  
        txtRows.BackColor = Color.MistyRose;  
        MessageBox.Show("Campul pentru randuri este gol/invalid!");  
        txtRows.Focus();  
        return;  
  
    case 3:  
        // Verificam campul pentru coloane  
        txtCols.BackColor = Color.MistyRose;  
        MessageBox.Show("Campul pentru coloane este gol/invalid!");  
        txtCols.Focus();  
        return;  
}
```

```
// -----  
// EXTRAGERE VALORI DACA INPUT-UL ESTE VALID  
// -----  
// Extragem numarul de randuri  
int rowsValue = int.Parse(txtRows.Text);  
// Extragem numarul de coloane  
int colsValue = int.Parse(txtCols.Text);  
  
// -----  
// INITIALIZARE RANDOM SI SUMA  
// -----  
// Generator de numere aleatoare  
Random rnd = new Random();  
// Variabila pentru suma tuturor valorilor  
int sum = 0;  
  
// -----  
// CREARE COLOANE IN DATAGRIDVIEW  
// -----  
for (int c = 0; c < colsValue; c++)  
{  
    // Adaugam fiecare coloana cu un nume unic si un text vizibil  
    dataGridViewMatrix.Columns.Add($"col{c}", $"Col {c + 1}");  
}  
  
// -----  
// POPULARE RANDURI CU VALORI ALEATORII SI CALCUL SUMA  
// -----  
for (int i = 0; i < rowsValue; i++)  
{  
    // Array pentru valorile unui rand  
    object[] rowValues = new object[colsValue];  
    for (int j = 0; j < colsValue; j++)  
    {  
        // Generam numar aleator intre 1 si 100  
        int value = rnd.Next(1, 101);  
        // Adaugam valoarea in array-ul randului  
        rowValues[j] = value;  
        // Adaugam valoarea la suma totala  
        sum += value;  
    }  
    // Adaugam randul in DataGridView  
    dataGridViewMatrix.Rows.Add(rowValues);  
}  
  
// -----  
// EVIDENTIERE CELULE CU VALORI MAI MARI DE 50  
// -----  
for (int i = 0; i < dataGridViewMatrix.Rows.Count; i++)  
{  
    for (int j = 0; j < dataGridViewMatrix.Columns.Count; j++)  
    {  
        // Obtinem celula curenta  
        var cell = dataGridViewMatrix.Rows[i].Cells[j];  
        if (cell.Value != null &&  
            int.TryParse(cell.Value.ToString(), out int value))  
        {  
            if (value > 50)  
            {  
                // Fundal verde pentru valori > 50  
                cell.Style.BackColor = Color.LightGreen;  
                // Text negru pentru vizibilitate  
                cell.Style.ForeColor = Color.Black;  
            }  
        }  
    }  
}
```

```
    }  
    }  
    }  
    // -----  
    // CALCUL SI AFISARE SUMA SI MEDIA  
    // -----  
    // Calcularea mediei elementelor din matrice  
    double average = (double)sum / (rowsValue * colsValue);  
    // Afisarea sumei elementelor din matrice  
    lblSum.Text = $"Suma elementelor: {sum}";  
    // Afisarea mediei cu 2 zecimale a elementelor din matrice  
    lblAverage.Text = $"Media elementelor: {average:F2}";  
}  
  
private void btnClear_Click(object sender, EventArgs e)  
{  
    // Stergem toate randurile din DataGridView  
    dataGridViewMatrix.Rows.Clear();  
    // Stergem toate coloanele din DataGridView  
    dataGridViewMatrix.Columns.Clear();  
    // Resetam text label-ului care afiseaza suma  
    lblSum.Text = "";  
    // Resetam text label-ului care afiseaza media  
    lblAverage.Text = "";  
  
    // Stergem valoarea introdusa pentru numarul de randuri  
    txtRows.Text = "";  
    // Stergem valoarea introdusa pentru numarul de pentru coloane  
    txtCols.Text = "";  
  
    // Resetam culoarea de fundal a TextBox-ului pentru randuri  
    txtRows.BackColor = Color.White;  
    // Resetam culoarea de fundal a TextBox-ului pentru coloane  
    txtCols.BackColor = Color.White;  
  
    // Setam cursorul pe TextBox-ul pentru randuri  
    txtRows.Focus();  
}  
}  
}
```

5.2.1.3. Instrucțiunea *foreach*

Instrucțiunea *foreach* este utilizată pentru a parcurge toate elementele unei colecții. Aceasta execută un bloc de cod pentru fiecare element din colecție, fără utilizarea unui index.

Format general:

```
foreach (tip_element variabila in colectie)  
{  
    [<instructiuni>];  
}
```

Exemplu:

```
foreach (DataGridViewRow row in dataGridViewMatrix.Rows)
{
    foreach (DataGridViewCell cell in row.Cells)
    {
        if (cell.Value != null)
        {
            int value = int.Parse(cell.Value.ToString());
            if (value > 50)
            {
                cell.Style.BackColor = Color.LightGreen;
            }
        }
    }
}
```

5.3. Exerciții

5.3.1. Aplicație Windows Forms pentru calculul mediei ponderate

Să se realizeze o aplicație Windows Forms care permite introducerea disciplinelor, a numărului de credite și a notelor obținute, iar la apăsarea unui buton să calculeze și să afișeze **media ponderată**.

Media ponderată se calculează cu formula:

$$\text{Media ponderată} = \frac{\sum(\text{nota} \times \text{număr credite})}{\sum(\text{număr credite})}$$

Aplicația va oferi următoarele funcționalități:

1. Utilizatorul introduce datele într-un tabel de tip **DataGridView**. Tabelul va conține următoarele coloane:
 - **Disciplina**
 - **Număr credite**
 - **Nota**
2. La apăsarea butonului „**Calculează media**”, aplicația va:
 - verifica validitatea datelor introduse;
 - calcula suma ponderată a notelor;
 - calcula totalul creditelor;
 - afișa media ponderată într-un **Label** sau **TextBox** (în mod **ReadOnly**).

Aplicația va conține cel puțin următoarele controale:

- un control **DataGridView** (editabil) cu 3 coloane:
 - Disciplina
 - Credite
 - Nota
- un buton: **Calculează media**
- un control **Label** pentru afișarea rezultatului
- un **Label** pentru mesaje de validare
- un **Label** pentru totalul creditelor

Regulile de validare sunt:

- **Disciplina** nu trebuie să fie goală
- **Numărul de credite** trebuie să fie un număr întreg pozitiv
- **Nota** trebuie să fie o valoare întreagă între **1** și **10**

Dacă există date invalide, aplicația va afișa un mesaj corespunzător și nu va continua calculul până la corectarea acestora.

LABORATORUL 4

5.4. Instrucțiunea while

Instrucțiunea `while` este o structură repetitivă care execută un bloc de cod atât timp cât o condiție este adevărată. Instrucțiunea `while` verifică mai întâi condiția, dacă condiția este adevărată atunci se execută codul, iar în caz contrar bucla se oprește. Dacă condiția nu devine niciodată falsă, apare o buclă infinită.

Format general:

```
while ([<conditie_continuare>])  
    [<instructiune>];
```

Fig. 22 prezintă schema logică pentru instrucțiunea `while`.

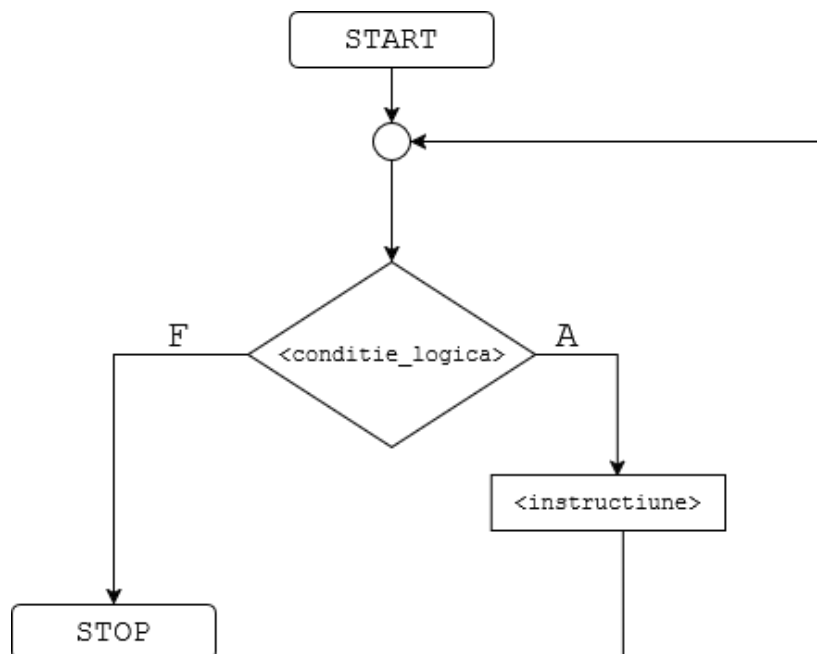


Fig. 4. Schema logică a instrucțiunii "while".

Exemplu:

```
private void button1_Click(object sender, EventArgs e)
{
    int i = 1;
    int suma = 0;

    while (i <= 10)
    {
        suma = suma + i;
        i++;
    }

    MessageBox.Show("Suma este: " + suma);
}
```

5.4.1. Utilizarea instrucțiunii while

Să se realizeze o aplicație Windows Forms care permite simularea evoluției unui depozit bancar. Utilizatorul va introduce suma inițială și depunerea lunară. Aplicația va considera o dobândă lunară fixă de 0,6% și va calcula, cu ajutorul instrucțiunii `while`, evoluția soldului până la atingerea sumei de 10.000 lei. Pentru fiecare lună se vor afișa în `ListView`: luna curentă, soldul inițial, depunerea lunară, dobânda obținută și soldul final.

Form1.cs [Design]

Luna	Sold inițial (lei)	Depunere (lei)	Dobândă (lei)	Sold final (lei)

În Designer, controalele și evenimentele sunt configurate după cum urmează:

- Butonul "Simuleaza" are asociat evenimentul Click += btnSimuleaza_Click;
- Butonul "Reset" are asociat evenimentul Click += btnReset_Click;
- Câmpul "tbSumaInitiala" este utilizat pentru introducerea sumei depuse inițial;
- Câmpul "tbDepunereLunara" este utilizat pentru introducerea sumei lunare depuse;
- Formul Form1 are asociat evenimentul Load += Form1_Load;
- Controlul de tip ListView "lstSimulare" este folosit pentru afișarea tabelului cu valorilor simulate. Configurarea controlului (din fereastra de proprietăți) se face cu următoarele valori:

```
lstSimulare.FullRowSelect = true;  
lstSimulare.GridLines = true;  
lstSimulare.MultiSelect = false;  
lstSimulare.Name = "lstSimulare";  
lstSimulare.View = View.Details;
```

Form1.cs

```
namespace SimulatorDepozitBancar  
{  
    public partial class Form1 : Form  
    {  
        public Form1()  
        {  
            InitializeComponent();  
        }  
  
        private void Form1_Load(object sender, EventArgs e)  
        {  
            lstSimulare.Columns.Clear();  
            lstSimulare.Items.Clear();  
  
            lstSimulare.Columns.Add("Luna", 70, HorizontalAlignment.Center);  
            lstSimulare.Columns.Add("Sold inițial (lei)", 120,  
                                    HorizontalAlignment.Center);  
            lstSimulare.Columns.Add("Depunere (lei)", 110,  
                                    HorizontalAlignment.Center);  
            lstSimulare.Columns.Add("Dobândă (lei)", 110,  
                                    HorizontalAlignment.Center);  
            lstSimulare.Columns.Add("Sold final (lei)", 120,  
                                    HorizontalAlignment.Center);  
  
            tbSumaInitiala.Focus();  
        }  
  
        private void btnSimulare_Click(object sender, EventArgs e)  
        {  
            // Resetare culori  
            tbSumaInitiala.BackColor = Color.White;  
            tbDepunereLunara.BackColor = Color.White;
```

```
// Validare sumă inițială
if (!double.TryParse(tbSumaInitiala.Text, out double sumaInitiala) ||
    sumaInitiala < 0)
{
    tbSumaInitiala.BackColor = Color.MistyRose;
    MessageBox.Show(
        "Introduceți o sumă inițială validă (număr pozitiv sau zero).");

    tbSumaInitiala.Focus();
    return;
}

// Validare depunere lunară
if (!double.TryParse(tbDepunereLunara.Text,
                    out double depunereLunara) ||
    depunereLunara <= 0)
{
    tbDepunereLunara.BackColor = Color.MistyRose;
    MessageBox.Show(
        "Introduceți o depunere lunară validă (număr pozitiv).");

    tbDepunereLunara.Focus();
    return;
}

// Resetare rezultate anterioare
lstSimulare.Items.Clear();

// Parametri simulare
double sold = sumaInitiala;
double sumaTinta = 10000.0;
double rataDobanzii = 0.006; // 0.6% pe lună
int luna = 1;

// Simulare lunară
while (sold < sumaTinta)
{
    double soldInitial = sold;
    double dobanda = (soldInitial + depunereLunara) * rataDobanzii;
    double soldFinal = soldInitial + depunereLunara + dobanda;

    ListViewItem item = new ListViewItem(luna.ToString());
    item.SubItems.Add(soldInitial.ToString("F2"));
    item.SubItems.Add(depunereLunara.ToString("F2"));
    item.SubItems.Add(dobanda.ToString("F2"));
    item.SubItems.Add(soldFinal.ToString("F2"));

    lstSimulare.Items.Add(item);

    sold = soldFinal;
    luna++;
}

MessageBox.Show(
    $"Depozitul a atins suma de {sold:F2} lei în {luna - 1} luni.",
    "Simulare finalizată",
    MessageBoxButtons.OK,
    MessageBoxIcon.Information
);

private void btnReset_Click(object sender, EventArgs e)
{
    tbSumaInitiala.Text = "";
}
```

```
        tbDepunereLunara.Text = "";  
  
        tbSumaInitiala.BackColor = Color.White;  
        tbDepunereLunara.BackColor = Color.White;  
  
        lstSimulare.Items.Clear();  
  
        tbSumaInitiala.Focus();  
    }  
}
```

5.5. Instrucțiunea do..while

Instrucțiunea `do..while` este o structură de control care execută un bloc de cod cel puțin o dată, iar apoi repetă execuția cât timp condiția este adevărată.

Diferența principală față de `while` este că la `while` se verifică condiția înainte de executarea unui ciclu, pe când la `do..while` se verifică condiția după executarea lui. Astfel, codul din `do..while` se va executa cel puțin o dată, chiar dacă condiția de continuare este falsă.

Format general:

```
do  
{  
    [<instructiuni>];  
} while ([<conditie_continuare>]);
```

Fig. 23 prezintă schema logică pentru instrucțiunea `do..while`.

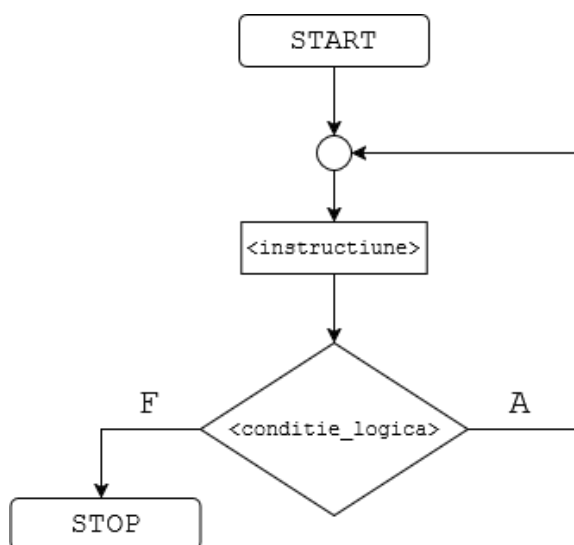


Fig. 5. Schema logică a instrucțiunii "do..while".

Exemplu:

```
private void button1_Click(object sender, EventArgs e)
{
    int i = 1;
    int suma = 0;

    do
    {
        suma = suma + i;
        i++;

    } while (i <= 5);

    MessageBox.Show("Suma este: " + suma);
}
```

5.6. Instrucțiuni de salt

Instrucțiunile de salt sunt instrucțiuni care modifică fluxul normal de execuție al programului, adică programul „sare” la o altă instrucțiune sau iese dintr-o structură de control.

5.6.1. Instrucțiunea break

Instrucțiunea `break` este o instrucțiune de salt care oprește imediat execuția unei bucle sau a unei instrucțiuni `switch`. Când programul întâlnește instrucțiunea `break`, execuția buclei se întrerupe, iar programul continuă cu prima instrucțiune după buclă.

Exemplu:

```
private void button1_Click(object sender, EventArgs e)
{
    for (int i = 1; i <= 10; i++)
    {
        if (i == 5)
            break;

        MessageBox.Show("Numarul este: " + i);
    }
}
```

Rezultatul rulării:

```
Numarul este: 1
Numarul este: 2
Numarul este: 3
Numarul este: 4
```

Bucula `for` începe cu valoarea `i=1` și afișează fiecare număr într-un `MessageBox`. În momentul în care `i` devine 5, condiția `i==5` devine adevărată, se execută instrucțiunea `break`, iar bucla se oprește.

5.6.2. Instrucțiunea *continue*

Instrucțiunea `continue` este o instrucțiune de salt care sare peste restul instrucțiunilor din iterația curentă a unei bucle și trece direct la următoarea iterație. Aceasta se utilizează în bucle precum `for`, `while` sau `do...while`, atunci când se dorește ignorarea unor anumite valori sau situații fără a opri complet instrucțiunea de ciclare. În cazul instrucțiunii `for`, prima instrucțiune după o instrucțiune `continue` este partea de modificare a stării.

Exemplu:

```
private void button1_Click(object sender, EventArgs e)
{
    for (int i = 1; i <= 10; i++)
    {
        if (i % 2 == 0)
            continue;

        MessageBox.Show("Numar impar: " + i);
    }
}
```

Rezultatul rulării:

```
Numar impar: 1
Numar impar: 3
Numar impar: 5
Numar impar: 7
Numar impar: 9
```

Bucula `for` parcurge numerele de la 1 la 10. Pentru fiecare număr se verifică dacă acesta este par folosind condiția `i%2 == 0`. Dacă numărul este par, se execută instrucțiunea `continue`, iar programul sare peste instrucțiunea `MessageBox` și trece la următoarea iterație. Astfel, sunt afișate doar numerele impare.

5.6.3. Instrucțiunea *return*

Instrucțiunea `return` este o instrucțiune de salt care oprește execuția unei funcții și returnează controlul către locul din program de unde a fost apelată funcția. În aplicațiile

Windows Forms, `return` este folosită frecvent în metodele asociate evenimentelor (de exemplu `button_Click`) pentru a întrerupe executarea codului dacă o anumită condiție este îndeplinită.

Exemplu:

```
private void button1_Click(object sender, EventArgs e)
{
    if (textBox1.Text == "")
    {
        MessageBox.Show("Introduceti un numar!");
        return;
    }
    MessageBox.Show("Ati introdus: " + textBox1.Text);
}
```

Când utilizatorul apasă butonul, programul verifică dacă `TextBox`-ul este gol. Dacă nu a fost introdus niciun text, se afișează un mesaj de avertizare și instrucțiunea `return` oprește executarea metodei. Dacă există text introdus, programul continuă și afișează valoarea introdusă.

Observație:

Diferența dintre instrucțiunile `break` și `return` este că `break` întrerupe doar bucla sau instrucțiunea `switch`, în timp ce `return` întrerupe complet execuția funcției.

5.6.4. Instrucțiunea `goto`

Instrucțiunea `goto` permite programului să treacă direct la instrucțiune identificată printr-o etichetă (label) definită în cod. Se folosește extrem de rar, deoarece poate face codul mai greu de urmărit, dar este utilă în anumite situații, de exemplu pentru a repeta rapid un bloc de cod fără a folosi o buclă formală.

Eticheta (label) se definește folosind un nume urmat de două puncte (:), iar `goto` transferă execuția programului direct la etichetă.

Exemplu:

```
private void button1_Click(object sender, EventArgs e)
{
    int i = 1;

    // label
```

```
start:
    MessageBox.Show("Numarul este: " + i);
    i++;

    if (i <= 3)
        // continuă execuția începând de la eticheta "start"
        goto start;
}
```

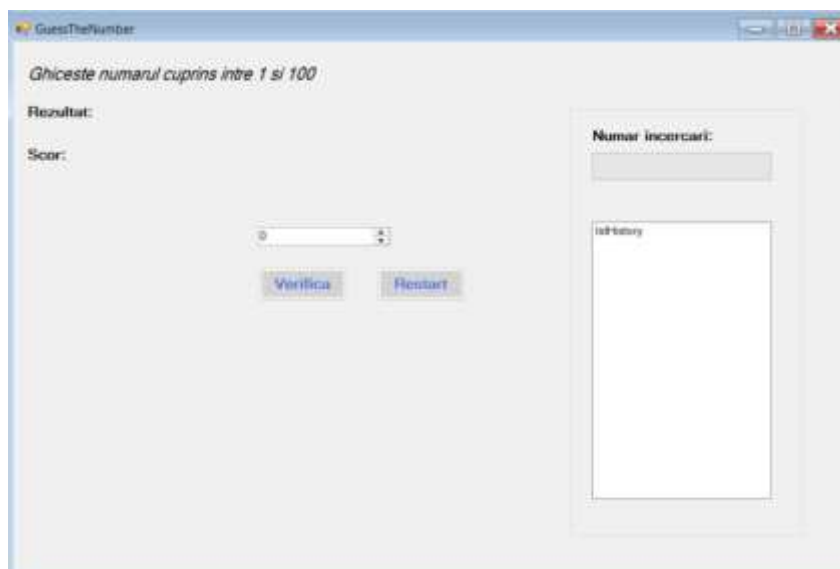
Rezultatul rulării:

```
Numarul este: 1
Numarul este: 2
Numarul este: 3
```

Variabila *i* începe de la 1, se afișează într-un `MessageBox`, apoi se incrementează, iar dacă *i* este mai mic sau egal decât 3, atunci programul trece înapoi la eticheta `start` folosind `goto`. Când *i* este mai mare decât 3, execuția continuă după blocul de cod și bucla implicită se încheie.

5.7. Utilizarea instrucțiunilor `do..while`, `break`, `continue` și `goto`

Form1.cs [Design]



În Designer, controalele și evenimentele sunt configurate după cum urmează:

- Eticheta "Rezultat" (`lblResult`) este folosită pentru afișarea rezultatului jocului;
- Eticheta "Scor" (`lblScore`) este folosită pentru afișarea scorului;

- Butonul "Verifica" are asociat evenimentul Click += btnCheck_Click;
- Butonul "Restart" are asociat evenimentul Click += btnRestart_Click;
- Selectorul numeric (NumericUpDown - numGuess) permite utilizatorului să introducă un număr fie tastând direct, fie folosind săgețile sus/jos pentru a crește sau scădea valoarea;
- Bara de progres (Progress Bar - progressAttempts) este folosită pentru progresul vizual, sub formă de bară;
- Lista de elemente (ListBox - lstHistory) este folosită pentru afișarea fiecărei încercări a jucătorului.

Form1.cs

```
using System;
using System.Windows.Forms;

namespace GuessTheNumber
{
    public partial class Form1 : Form
    {
        // Obiect Random pentru a genera numar aleator
        Random rand = new Random();

        // Variabila care va retine numarul ce trebuie ghicit
        int number;

        // Contor pentru numarul de incercari
        int attempts = 0;

        // Scorul jucatorului (scade la fiecare incercare)
        int score = 100;

        public Form1()
        {
            InitializeComponent();
            StartGame();
        }

        // Metoda care initializeaza sau reseteaza jocul
        void StartGame()
        {
            // Generam un numar aleator intre 1 si 100
            number = rand.Next(1, 101);

            // Resetam numarul de incercari
            attempts = 0;

            // Resetam scorul
            score = 100;

            // Setam valorile pentru progress bar
            progressAttempts.Value = 0;
            progressAttempts.Minimum = 0;
            progressAttempts.Maximum = 10;
        }
    }
}
```

```
// Stergem istoricul incercarilor
lstHistory.Items.Clear();

// Stergem mesajul rezultat
lblResult.Text = "";

// Afisam scorul initial
lblScore.Text = "Score: " + score;

// Resetam NumericUpDown la minim
numGuess.Value = numGuess.Minimum;

// Activam butonul Verifica
btnCheck.Enabled = true;
}

private void btnCheck_Click(object sender, EventArgs e)
{
    int guess;

    // Folosim do..while pentru a executa blocul de verificare
    // cel puțin o dată
    do
    {
        // Luam valoarea introdusă de utilizator
        guess = (int)numGuess.Value;

        // Incrementăm numărul de încercări
        attempts++;

        // Scădem 10 puncte din scor
        score -= 10;

        // Dacă am depășit numărul maxim de încercări
        if (attempts >= progressAttempts.Maximum)
        {
            progressAttempts.Value = progressAttempts.Maximum;
            lstHistory.Items.Add("Încercarea " + attempts + ": " + guess);
            lblScore.Text = "Score: " + score;

            MessageBox.Show("Ai pierdut! Numărul era " + number);
            btnCheck.Enabled = false;
            break;
        }

        // Actualizăm progress bar
        progressAttempts.Value = attempts;

        // Salvăm încercarea în istoric
        lstHistory.Items.Add("Încercarea " + attempts + ": " + guess);

        // Verificăm răspunsul
        if (guess < number)
        {
            lblResult.Text = "Numărul este MAI MARE";
            lblScore.Text = "Score: " + score;
        }
        else if (guess > number)
        {
            lblResult.Text = "Numărul este MAI MIC";
            lblScore.Text = "Score: " + score;
        }
        else
        {

```

```
        lblResult.Text = "Ai castigat! Felicitari!";  
        lblScore.Text = "Score: " + score;  
  
        MessageBox.Show("Ai ghicit numarul!\nScor final: " + score);  
        btnCheck.Enabled = false;  
    }  
    } while (false);  
}  
  
// Buton pentru restartarea jocului  
private void btnRestart_Click(object sender, EventArgs e)  
{  
    StartGame();  
}  
}  
}
```

5.8. Exerciții

5.8.1. Extinderea aplicației de simulare a depozitului bancar

Se cere extinderea aplicației de simulare a depozitului bancar astfel încât utilizatorul să poată configura parametrii principali ai simulării. În varianta inițială a aplicației, suma-țintă și rata dobânzii lunare sunt valori fixe definite în cod. În această temă, aplicația va fi modificată astfel încât aceste valori să poată fi introduse direct de către utilizator prin intermediul interfeței grafice.

Cerințe:

Aplicația va fi modificată pentru a permite introducerea următoarelor valori:

- suma inițială depusă
- depunerea lunară
- suma-țintă pe care utilizatorul dorește să o atingă
- rata dobânzii lunare (în procente)

În interfața aplicației se vor adăuga două controale `TextBox` suplimentare pentru introducerea:

- sumei-țintă
- ratei dobânzii

Reguli de validare

Aplicația trebuie să verifice următoarele condiții:

- suma-țintă trebuie să fie mai mare decât suma inițială;
- rata dobânzii trebuie să fie un număr real pozitiv.

Dacă una dintre valori este invalidă, aplicația trebuie să afișeze un mesaj de eroare și să oprească execuția simulării.

Observații

Rata dobânzii va fi introdusă în procente. De exemplu, dacă utilizatorul introduce **0.6**, aceasta reprezintă **0,6% dobândă lunară**.

În calcule, valoarea trebuie convertită în formă zecimală:

$$rataDobanzii = \frac{dobanda}{100}$$

LABORATORUL 5

6. Tablouri

Un tablou (array) este o structură de date de dimensiune fixă, care grupează mai multe elemente de același tip într-o zonă contiguă¹ de memorie. Accesul la oricare dintre elemente se realizează prin intermediul unui index numeric, primul element având indexul 0.

În aplicațiile Windows Forms, tablourile sunt frecvent utilizate pentru a organiza datele care alimentează controalele vizuale ale interfeței grafice – precum `ListBox`, `ComboBox` sau `DataGridView` – facilitând astfel afișarea, parcurgerea și manipularea acestora de către utilizator.

6.1.1. Declararea unui tablou

Declararea unui tablou se face utilizând următoarea sintaxă:

```
<tip_elemente>[] <identificator_tablou>;
```

unde:

- `<tip_elemente>` reprezintă tipul de date al elementelor tabloului;
- `[]` indică faptul că este declarat un tablou;
- `<identificator_tablou>` reprezintă numele variabilei tablou.

6.1.2. Instanțierea

Instanțierea unui tablou se referă la etapa în care se rezervă spațiu în memorie pentru obiectul tablou, formându-se astfel o structură ce poate să găzduiască un număr prestabilit de elemente de același tip. Procesul de instanțiere implică folosirea operatorului `new` prin care se creează efectiv obiectul tablou, indicând tipul datelor stocate și dimensiunea sa.

Un tablou poate fi declarat și instanțiat într-o singură instrucțiune.

¹ locații de memorie cu adrese consecutive

```
double[] rezultate = new double[10];
```

Observații:

- Dimensiunea unui tablou este stabilită la momentul instanțierii și nu poate fi modificată ulterior;
- Tablourile sunt tipuri referință, ceea ce înseamnă că variabila reține o referință către zona de memorie alocată;
- Indexarea elementelor din tablou pornește de la 0;
- lungimea unui tablou este disponibilă prin proprietatea `Length`;

6.1.3. Inițializarea

Inițializarea unui tablou constă în atribuirea de valori elementelor sale, fie automat (cu valori implicite), fie explicit, prin specificarea acestora.

La inițializarea unui tablou valorile inițiale trebuie incluse între acolade și separate prin virgule.

```
double[] rezultate = { 6.23, 10.52, 25.78 };
```

La inițializare tabloul este declarat, instanțiat și inițializat într-o singură instrucțiune.

7. Colecții

O colecție în C# este o structură de date care permite stocarea și gestionarea unui set de obiecte, de obicei de același tip, cu funcționalități suplimentare față de un tablou (array), precum dimensiune dinamică, adăugare, ștergere și căutare a elementelor.

În contextul unei aplicații Windows Forms, colecțiile sunt utilizate pentru:

- Stocarea elementelor GUI (controale, date) într-o structură organizată.
- Manipularea și afișarea datelor în controale grafice precum `ListBox`, `ComboBox` sau `DataGridView`.
- Gestionarea dinamică a obiectelor la rularea aplicației, fără a fi nevoie cunoașterea dimensiunii exacte dinainte.

7.1. Liste

`List<T>` este o colecție generică oferită de `System.Collections.Generic`. Aceasta implementează o structură de date care funcționează ca un tablou dinamic, permițând

gestionarea unei secvențe ordonate de elemente de tip T . Principala caracteristică a listei este flexibilitatea: dimensiunea ei se poate modifica automat, pe măsură ce se adaugă sau se elimină elemente, spre deosebire de tablourile clasice care au dimensiune fixă.

7.1.1. Declararea listei

Declararea și instanțierea listei se face utilizând următoarea sintaxă:

```
List<T> nume_lista = new List<T>();
```

unde:

- `List<T>` reprezintă o clasă generică ce reprezintă o colecție dinamică de elemente de tip T , iar T este o variabilă de tip;
- `<nume_lista>` reprezintă numele listei.

Exemplu:

```
List<int> numere = new List<int>();
```

7.1.2. Constructorii listei

Tab. 1. Constructorii listei `List<T>` [4].

Nume	Descriere
<code>List<T>()</code>	Inițializează o nouă instanță a clasei <code>List<T></code> care este goală și are capacitatea inițială implicită.
<code>List<T>(IEnumerable<T>)</code>	Inițializează o nouă instanță a clasei <code>List<T></code> care conține elemente copiate din colecția specificată și are o capacitate suficientă pentru a acomoda numărul de elemente copiate.
<code>List<T>(Int32)</code>	Inițializează o nouă instanță a clasei <code>List<T></code> care este goală și are capacitatea inițială specificată.

7.1.3. Parcurgerea listei cu `foreach`

Instrucțiunea `foreach` este o structură de control care permite iterarea automată printr-o colecție, fără a folosi index.

Format general:

```
foreach (tip_element element in colectie)
{
    [<instructiuni>];
}
```

unde:

- **tip_element** reprezintă tipul fiecărui element din colecția parcursă, iar acest tip trebuie să fie similar cu tipul elementelor din colecția respectivă. Astfel, **tip_element** trebuie să corespundă cu tipul generic **T** al listei **List<T>**;
- **element** este o variabilă de tipul **tip_element** care reprezintă elementul curent al colecției la fiecare iterație. Aceasta se actualizează automat de fiecare dată când **foreach** parcurge un nou element din colecție;
- **colectie** este obiectul parcurs cu **foreach**. Aceasta poate fi orice colecție care implementează interfața **IEnumerable<T>**, precum: tablouri (array), **List**, **Dictionary**, **Queue**, **Stack**, sau orice alt tip de colecție iterabilă.

Exemplu:

```
private void Form1_Load(object sender, EventArgs e)
{
    List<string> nume = new List<string> { "George", "Alina", "Mihai" };

    foreach (string n in nume)
    {
        listNames.Items.Add(n);
    }
}
```

7.1.4. Metode ale colecției **List<T>**

Colecția generică dinamică **List<T>** definește un număr mare de metode. Acestea pot fi grupate în funcție de rolul lor principal: adăugare, ștergere, căutare, parcurgere, sortare și inversare, conversie/copiere și gestionarea capacității.

Tab. 2. Metode ale colecției List<T>. [4]

Nume	Descriere
Add (T)	Adaugă un element la finalul listei.
AddRange (IEnumerable<T>)	Adaugă elementele colecției specificate la sfârșitul listei.
Remove (T)	Elimină prima apariție a unui obiect specific din listă.
RemoveAt (Int32)	Elimină elementul de la indexul specificat din listă.
RemoveAll (Predicate<T>)	Elimină toate elementele care îndeplinesc condițiile definite de predicatul specificat.
RemoveRange (Int32, Int32)	Elimină un interval (o secvență) de elemente din listă.
Clear ()	Elimină toate elementele din listă.
Contains (T)	Determină dacă un element se află în listă.
IndexOf (T)	Caută obiectul specificat și returnează indexul (începând de la 0) al primei apariții în întreaga listă.
IndexOf (T, Int32)	Caută obiectul specificat și returnează indexul (începând de la 0) al primei apariții în intervalul de elemente din listă care se întinde de la indexul specificat până la ultimul element.
LastIndexOf (T)	Caută obiectul specificat și returnează indexul (începând de la 0) al ultimei apariții în întreaga listă.
LastIndexOf (T, Int32, Int32)	Caută obiectul specificat și returnează indexul (începând de la 0) al ultimei apariții în intervalul de elemente din listă care conține numărul specificat de elemente și se termină la indexul specificat.
Find (Predicate<T>)	Caută un element care respectă condițiile definite de predicatul specificat și returnează prima apariție din întreaga listă.
FindAll (Predicate<T>)	Returnează toate elementele care respectă condițiile definite de predicatul specificat.

Nume	Descriere
<code>Equals (Object)</code>	Determină dacă obiectul specificat este egal cu obiectul curent.
<code>Exists (Predicate<T>)</code>	Determină dacă lista conține elemente care respectă condițiile definite de predicatul specificat.
<code>TrueForAll (Predicate<T>)</code>	Determină dacă toate elementele din listă respectă condițiile definite de predicatul specificat.
<code>ForEach (Action<T>)</code>	Execută acțiunea specificată pentru fiecare element din listă.
<code>GetEnumerator ()</code>	Returnează un enumerator care iterează prin listă.
<code>Insert (Int32, T)</code>	Inserează un element în listă la indexul specificat.
<code>InsertRange (Int32, IEnumerable<T>)</code>	Inserează elementele unei colecții în listă la indexul specificat.
<code>Sort ()</code>	Sortează elementele din întreaga listă folosind comparatorul implicit.
<code>Sort (Comparison<T>)</code>	Sortează elementele din întreaga listă folosind comparația specificată (<code>Comparison<T></code>).
<code>BinarySearch (T)</code>	Caută un element în întreaga listă sortată, folosind comparatorul implicit, și returnează indexul elementului.
<code>Reverse ()</code>	Inversează ordinea elementelor din întreaga <code>List<T></code> .
<code>CopyTo (T[])</code>	Copiază întreaga listă într-un tablou unidimensional compatibil, începând de la începutul tabloului destinație.
<code>GetRange (Int32, Int32)</code>	Creează o copie superficială (<i>shallow copy</i>) a unui interval de elemente din listă sursă.
<code>EnsureCapacity (Int32)</code>	Asigură capacitatea acestei liste la capacitatea specificată. Dacă valoarea capacității curente este mai mică decât cea specificată, aceasta este mărită până la cel puțin valoarea respectivă.

Nume	Descriere
TrimExcess()	Setează capacitatea la numărul efectiv de elemente din listă.

7.1.4.1. Metoda Add(T)

Metoda `Add(T)` adaugă un obiect de tipul `T` la sfârșitul listei. După adăugarea elementului, lista își ajustează dimensiunea (contorul de elemente), crescând automat cu 1 pentru a reda numărul de elemente din colecție.

Exemplu:

```
private void Form1_Load(object sender, EventArgs e)
{
    List<string> nume = new List<string> { "George", "Alina", "Mihai" };

    foreach (string n in nume)
    {
        listNames.Items.Add(n);
    }

    listNames.Items.Add("");
    listNames.Items.Add("Noua lista este:");

    // Adaugarea noilor elemente in lista
    nume.Add("Manuela");
    nume.Add("Dragos");

    // Afisarea listei actualizate
    foreach (string m in nume)
    {
        listNames.Items.Add(m);
    }
}
```

7.1.4.2. Metoda Remove(T)

Metoda `Remove(T)` caută și elimină prima apariție a elementului `item` din listă. Dacă elementul există în listă, acesta este eliminat și metoda returnează `true`. În caz contrar, nu se efectuează nicio modificare și se returnează `false`.

- Dacă există elementul - se elimină primul element care corespunde cu valoarea `T` din listă;
- Dacă nu există elementul - lista rămâne neschimbată și metoda returnează `false`.

Exemplu:

```
private void Form1_Load(object sender, EventArgs e)
{
    List<string> nume = new List<string> { "George", "Alina", "Mihai" };

    foreach (string n in nume)
    {
        listNames.Items.Add(n);
    }

    // Eliminarea elementului "Alina" din lista
    bool removedElement = nume.Remove("Alina");

    // Verificare rezultat
    if(removedElement)
        MessageBox.Show("Alina a fost stearsa din lista.");
    else
        MessageBox.Show("Alina nu a fost gasita in lista.");

    // Afisare lista actualizata dupa eliminare
    listNames.Items.Add("");
    listNames.Items.Add("Lista actualizata:");

    foreach (string m in nume)
    {
        listNames.Items.Add(m);
    }
}
```

7.1.4.3. Metoda *IndexOf(T)*

Metoda `IndexOf(T)` este folosită pentru a afla poziția unui element într-o listă. Metoda returnează indexul primului element care se potrivește cu valoarea căutată, iar în caz contrar, returnează -1.

Exemplu:

```
List<string> nume;
private void Form1_Load(object sender, EventArgs e)
{
    nume = new List<string> { "George", "Alina", "Mihai" };

    foreach (string n in nume)
    {
        listNames.Items.Add(n);
    }
}
```

```
}  
  
private void btnRezultat_Click(object sender, EventArgs e)  
{  
    int index = nume.IndexOf("Mihai");  
  
    // Verificare rezultat  
    if (index != -1)  
        MessageBox.Show("Numele Mihai se afla in lista pe pozitia " + index);  
    else  
        MessageBox.Show("Numele Mihai nu se afla in lista.");  
}
```

7.1.4.4. Metoda *LastIndexOf(T)*

Metoda `LastIndexOf(T)` este folosită pentru a determina indexul ultimei apariții a unui element într-o listă.

- Dacă elementul există de mai multe ori, metoda returnează poziția ultimei apariții;
- Dacă elementul nu există, metoda returnează -1.

Exemplu:

```
List<string> nume;  
private void Form1_Load(object sender, EventArgs e)  
{  
    nume = new List<string> { "George", "Alina", "Carmen", "Mihai", "Carmen"};  
  
    foreach (string n in nume)  
    {  
        listNames.Items.Add(n);  
    }  
}  
  
private void btnRezultat_Click(object sender, EventArgs e)  
{  
    int lastIndex = nume.LastIndexOf("Carmen");  
  
    // Verificare rezultat  
    if (lastIndex != -1)  
    {  
        MessageBox.Show("Numele Carmen se afla in lista pe pozitia " +  
                        lastIndex);  
    }  
    else  
    {  
        MessageBox.Show("Numele Carmen nu se afla in lista.");  
    }  
}
```

7.1.4.5. Metoda *Sort()*

Metoda `Sort()` este folosită pentru a ordona elementele listei în ordine crescătoare sau conform unui criteriu specificat.

Exemplu:

```
List<string> nume;
private void Form1_Load(object sender, EventArgs e)
{
    nume = new List<string> { "George", "Alina", "Carmen", "Mihaela" };

    foreach (string n in nume)
    {
        listNames.Items.Add(n);
    }
}

private void btnRezultat_Click(object sender, EventArgs e)
{
    nume.Sort();

    listNames.Items.Add("");
    listNames.Items.Add("Lista sortata in ordine crescatoare:");
    foreach (string n in nume)
    {
        listNames.Items.Add(n);
    }
}
```

7.1.4.6. Metoda *BinarySearch(T)*

Metoda `BinarySearch(T)` este folosită pentru a căuta eficient un element într-o listă sortată.

- Folosește algoritmul de căutare binară;
- Returnează indexul elementului găsit dacă există;
- Dacă elementul nu există, returnează un număr negativ care poate fi folosit pentru a determina unde ar trebui inserat elementul pentru a păstra ordinea.

Observație:

Înainte de a folosi `BinarySearch` lista trebuie să fie sortată. Dacă lista nu este sortată, rezultatul va fi greșit.

Exemplu:

```
List<string> nume;  
private void Form1_Load(object sender, EventArgs e)  
{  
    nume = new List<string> { "George", "Alina", "Carmen", "Mihaela"};  
  
    foreach (string n in nume)  
    {  
        listNames.Items.Add(n);  
    }  
  
    nume.Sort();  
  
    listNames.Items.Add("");  
    listNames.Items.Add("Lista sortata in ordine crescatoare:");  
  
    foreach (string n in nume)  
    {  
        listNames.Items.Add(n);  
    }  
}  
  
private void btnRezultat_Click(object sender, EventArgs e)  
{  
    int cautareNume = nume.BinarySearch("Carmen");  
  
    if (cautareNume >= 0)  
    {  
        MessageBox.Show("Numele Carmen se afla in lista pe pozitia "  
            + cautareNume);  
    }  
    else  
    {  
        MessageBox.Show("Numele Carmen NU se afla in lista");  
    }  
}
```

7.1.4.7. Metoda *CopyTo(T[])*

Metoda `CopyTo(T[])` realizează o copiere secvențială a tuturor elementelor din listă în tabloul dat, începând de la indexul 0 al tabloului.

Exemplu:

```
List<string> nume1;  
private void Form1_Load(object sender, EventArgs e)  
{  
    nume1 = new List<string> { "George", "Alina", "Carmen", "Mihaela"};
```

```
listNames.Items.Add("Lista originala:");  
foreach (string n in nume1)  
{  
    listNames.Items.Add(n);  
}  
}  
  
private void btnRezultat_Click(object sender, EventArgs e)  
{  
    string[] numeArray = new string[nume1.Count];  
    nume1.CopyTo(numeArray);  
  
    listNames.Items.Add("");  
    listNames.Items.Add("Copia listei:");  
    foreach (string n in numeArray)  
    {  
        listNames.Items.Add(n);  
    }  
}
```

7.1.5. Clasa Enumerable

Metodele din această clasă oferă o implementare a operatorilor standard de interogare pentru accesarea surselor de date care implementează `IEnumerable<T>`. Operatorii standard de interogare sunt metode de uz general care urmează modelul *LINQ (Language Integrated Query)* și permit exprimarea operațiilor de parcurgere, filtrare și proiecție asupra datelor în orice limbaj de programare bazat pe .NET. [5]

Tab. 3. Metodele clasei Enumerable. [5]

Nume	Descriere
Sum()	Calculează suma elementelor.
Average()	Calculează media aritmetică a elementelor.
Min()	Returnează valoarea minimă dintr-o secvență de valori.
Max()	Returnează valoarea maximă dintr-o secvență de valori.
Count()	Returnează numărul de elemente dintr-o secvență de valori.

Nume	Descriere
Aggregate()	Aplică o funcție de acumulare peste o secvență.

7.1.5.1. Metoda Sum()

Metoda `Sum()` este o funcție de agregare din LINQ (`System.Linq`) care calculează suma elementelor unei colecții. Aceasta parcurge toate elementele și returnează o valoare scalară numerică reprezentând suma acestora.

Exemplu:

```
List<int> valori;  
private void Form1_Load(object sender, EventArgs e)  
{  
    valori = new List<int> {23, 46, 3, 73};  
    textBox1.Text = "";  
  
    foreach (int x in valori)  
    {  
        textBox1.Text += x + " ";  
    }  
}  
  
private void btnCalculeaza_Click(object sender, EventArgs e)  
{  
    int suma = valori.Sum();  
    lblRezultat.Text = "Suma elementelor din lista este:" + suma.ToString();  
}
```

7.1.5.2. Metoda Average()

Metoda `Average()` este o metodă folosită pentru a calcula media aritmetică a elementelor dintr-o colecție. Aceasta adună toate elementele din listă și împarte suma la numărul de elemente.

Exemplu:

```
List<int> valori;  
private void Form1_Load(object sender, EventArgs e)  
{  
    valori = new List<int> {23, 46, 3, 73};  
    textBox1.Text = "";  
  
    foreach (int x in valori)  
    {
```

```
        textBox1.Text += x + " ";
    }
}

private void btnCalculeaza_Click(object sender, EventArgs e)
{
    double media = valori.Average();
    lblRezultat.Text = "Media elementelor din lista este:" + media.ToString();
}
```

7.1.5.3. Metoda *Min()*

Metoda `Min()` este o metodă folosită pentru a găsi valoarea minimă dintr-o listă de elemente. Este similară cu funcția matematică de „minim”, dar adaptată pentru colecții de date.

Exemplu:

```
List<int> valori;
private void Form1_Load(object sender, EventArgs e)
{
    valori = new List<int> {23, 46, 3, 73};
    textBox1.Text = "";

    foreach (int x in valori)
    {
        textBox1.Text += x + " ";
    }
}

private void btnCalculeaza_Click(object sender, EventArgs e)
{
    int minim = valori.Min();
    lblRezultat.Text = "Elementul minim din lista este:" + minim.ToString();
}
```

7.1.5.4. Metoda *Max()*

Metoda `Max()` este o metodă ce returnează valoarea maximă dintr-o listă sau secvență de elemente.

Exemplu:

```
List<int> valori;
private void Form1_Load(object sender, EventArgs e)
{
    valori = new List<int> {23, 46, 3, 73};
    textBox1.Text = "";
}
```

```
        foreach (int x in valori)
        {
            textBox1.Text += x + " ";
        }
    }

    private void btnCalculeaza_Click(object sender, EventArgs e)
    {
        int maxim = valori.Max();
        lblRezultat.Text = "Elementul maxim din lista este:" + maxim.ToString();
    }
}
```

7.1.5.5. Metoda *Count()*

Metoda `Count()` este o metodă folosită pentru a afla numărul de elemente dintr-o listă sau colecție.

Exemplu:

```
List<int> valori;
private void Form1_Load(object sender, EventArgs e)
{
    valori = new List<int> {23, 46, 3, 73};
    textBox1.Text = "";

    foreach (int x in valori)
    {
        textBox1.Text += x + " ";
    }
}

private void btnCalculeaza_Click(object sender, EventArgs e)
{
    int nrElemente = valori.Count();
    lblRezultat.Text = "Lista are " + nrElemente.ToString() + " elemente";
}
}
```

7.1.5.6. Metoda *Aggregate()*

Metoda `Aggregate()` este o metodă folosită pentru a combina toate elementele unei liste într-o singură valoare, aplicând o funcție acumulatoare. Permite orice tip de combinație: sumă, înmulțire, concatenare, minim/maxim.

Observații:

- Metoda poate avea sau nu valoare inițială;

- Metoda este utilă atunci când este necesară aplicarea unei logici personalizate de agregare asupra elementelor unei colecții, precum concatenarea de șiruri de caractere, calculul produsului elementelor sau alte operații complexe definite de utilizator.

Format general:

```
<tip_variabila> <nume_variabila> =  
                                nume_lista.Aggregate((acc, element) => acc +  
                                element);
```

unde:

- `acc` reprezintă valoarea acumulată până la elementul curent;
- `element` reprezintă elementul curent din listă.

Funcția returnează valoarea finală după parcurgerea întregii liste.

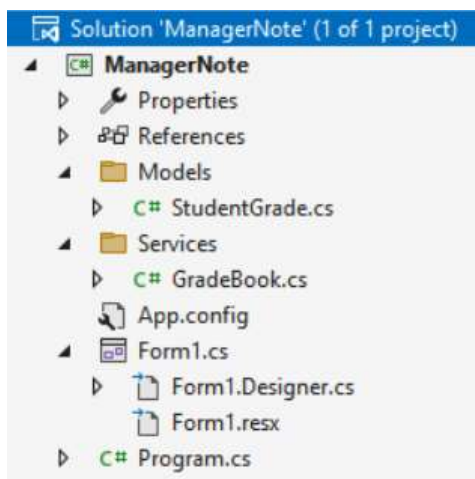
Exemplu:

```
List<int> valori,elementeInmultire;  
List<string> cuvinte;  
private void Form1_Load(object sender, EventArgs e)  
{  
    valori = new List<int> {25, 65, 98};  
  
    cuvinte = new List<string>{ "Palatul", "Buckingham", "are", "602",  
                                "de", "incaperi","."};  
  
    elementeInmultire = new List<int> {4,6};  
  
    lstResults.Items.Add("Elementele listei pentru suma:");  
  
    foreach (int v in valori)  
    {  
        lstResults.Items.Add(v.ToString());  
    }  
  
    lstResults.Items.Add("");  
    lstResults.Items.Add("Cuvintele listei:");  
  
    foreach (string c in cuvinte)  
    {  
        lstResults.Items.Add(c);  
    }  
  
    lstResults.Items.Add("");  
    lstResults.Items.Add("Elementele listei pentru inmultire:");  
    foreach (int p in elementeInmultire)
```

```
        {  
            lstResults.Items.Add(p.ToString());  
        }  
    }  
  
    private void btnAfiseaza_Click(object sender, EventArgs e)  
    {  
        int suma = valori.Aggregate((acc, n) => acc + n);  
        lblSuma.Text = "Suma numerelor este: " + suma.ToString();  
  
        string fraza = cuvinte.Aggregate((acc, cuv) => acc + " " + cuv);  
        lblConcatenare.Text = "Fraza este: " + fraza;  
  
        int inmultire = elementeInmultire.Aggregate(2, (acc, m) => acc * m);  
        lblMedia.Text = "Produsul elementelor este: " + inmultire.ToString();  
    }  
}
```

7.2. Aplicație cu liste - Manager de note pentru studenți

7.2.1. Structura proiectului:



7.2.2. Design grafic – Form1

The screenshot shows a Windows application window titled "ManagerNote". The interface includes the following elements:

- Input fields:**
 - "Nume student:" with a text box.
 - "Notele studentului:" with a text box.
- Buttons:**
 - "Adaugare" (Add) in green.
 - "Stergere" (Delete) in green.
 - "Calculare medie" (Calculate average).
 - "Afisare note min si max" (Display min and max notes).
 - "Afisare studenti sub medie" (Display students below average).
- Output/Status area:**
 - "Număr total studenți: 0"
 - "Media notelor: -"
 - "Nota maximă: -"
 - "Nota minimă: -"
 - Two large empty rectangular boxes for displaying student lists or results.

7.2.3. Proprietăți și evenimente pentru controale

Tip control	Nume (Name)	Setări relevante	Rol
TextBox	tbNume	-	Permite introducerea numelui studentului
TextBox	tbNota	-	Permite introducerea notei studentului și validarea acesteia
Button	btnAdauga	Text = "Adaugare" Click → btnAdauga_Click	Adaugă studentul și nota în colecție
Button	btnSterge	Text = "Stergere" Click → btnSterge_Click	Șterge elementul selectat din listă
Button	btnCalculeazaMedia	Text = "Calculare medie" Click → btnCalculeazaMedia_Click	Calculează media notelor
Button	btnAfisareMinMax	Text = "Afisare note min si max" Click → btnMinMax_Click	Afișează nota minimă și maximă
Button	btnStudentiSubMedie	Text = "Afisare studenti sub medie" Click → btnStudentiSubMedie_Click	Afișează studenții cu note sub medie

Tip control	Nume (Name)	Setări relevante	Rol
ListBox	lbxNote	-	Afișează lista studenților și a notelor
ListView	lvStudentiSubMedie	View = List	Afișează studenții sub medie
Label	label1	Text = "Nume student:"	Etichetă pentru câmpul nume
Label	label2	Text = "Nota studentului:"	Etichetă pentru câmpul notă
Label	lbMedia	Text = "Media"	Afișează media calculată
Label	lbNrStudenti	Text = "Nr"	Afișează numărul de studenți
Label	lbNotaMax	Text = "Max"	Afișează nota maximă
Label	lbNotaMin	Text = "Min"	Afișează nota minimă
Form	Form1	Text = "ManagerNote" Load → Form1_Load	Formular principal al aplicației

7.2.4. Cod sursă

Form1.cs
<pre> using System; using System.Windows.Forms; using ManagerNote.Services; namespace ManagerNote { public partial class Form1 : Form { private GradeBook _gradeBook = new GradeBook(); public Form1() { InitializeComponent(); } private void Form1_Load(object sender, EventArgs e) { ResetStatistics(); tbNume.Focus(); } private void btnAdauga_Click(object sender, EventArgs e) { try { _gradeBook.Add(tbNume.Text, tbNota.Text); } catch { } } } } </pre>

```
        RefreshStudentList();
        ClearInputFields();
        UpdateStatistics();
    }
    catch (Exception ex) when (ex is ArgumentException
                                || ex is ArgumentOutOfRangeException)
    {
        ShowValidationError(ex.Message);
    }
}

private void btnCalculeazaMedia_Click(object sender, EventArgs e)
{
    try
    {
        UpdateStatistics();
    }
    catch (InvalidOperationException ex)
    {
        ShowValidationError(ex.Message);
    }
}

private void btnMinMax_Click(object sender, EventArgs e)
{
    try
    {
        var maxEntry = _gradeBook.GetMaxGradeEntry();
        var minEntry = _gradeBook.GetMinGradeEntry();

        lbNotaMax.Text =
            $"Nota maximă: {maxEntry.Grade:F2} - {maxEntry.StudentName}";
        lbNotaMin.Text =
            $"Nota minimă: {minEntry.Grade:F2} - {minEntry.StudentName}";
    }
    catch (InvalidOperationException ex)
    {
        ShowValidationError(ex.Message);
    }
}

private void btnSterge_Click(object sender, EventArgs e)
{
    if (lbxNote.SelectedIndex < 0)
    {
        ShowValidationError(
            "Selectați un student din listă pentru ștergere.");
        return;
    }

    try
    {
        _gradeBook.RemoveAt(lbxNote.SelectedIndex);
        RefreshStudentList();
        RefreshStudentsBelowAverage();

        if (_gradeBook.Count == 0)
        {
            ResetStatistics();
        }
    }
}
```

```
        else
        {
            UpdateStatistics();
        }
    }
    catch (ArgumentOutOfRangeException ex)
    {
        ShowValidationError(ex.Message);
    }
}

private void btnStudentiSubMedie_Click(object sender, EventArgs e)
{
    try
    {
        RefreshStudentsBelowAverage();
    }
    catch (InvalidOperationException ex)
    {
        ShowValidationError(ex.Message);
    }
}

private void RefreshStudentList()
{
    lbxNote.BeginUpdate();
    lbxNote.Items.Clear();

    foreach (var entry in _gradeBook.Entries)
    {
        lbxNote.Items.Add(entry);
    }

    lbxNote.EndUpdate();
}

private void RefreshStudentsBelowAverage()
{
    lvStudentiSubMedie.BeginUpdate();
    lvStudentiSubMedie.Items.Clear();

    foreach (var entry in _gradeBook.GetStudentsBelowAverage())
    {
        lvStudentiSubMedie.Items.Add(entry.ToString());
    }

    lvStudentiSubMedie.EndUpdate();
}

private void UpdateStatistics()
{
    var average = _gradeBook.GetAverage();
    lbNrStudenti.Text = $"Număr total studenți: {_gradeBook.Count}";
    lbMedia.Text = $"Media notelor: {average:F2}";
}

private void ResetStatistics()
{
    lbNrStudenti.Text = "Număr total studenți: 0";
    lbMedia.Text = "Media notelor: -";
}
```

```
        lbNotaMax.Text = "Nota maximă: -";  
        lbNotaMin.Text = "Nota minimă: -";  
        lvStudentiSubMedie.Items.Clear();  
    }  
  
    private void ClearInputFields()  
    {  
        tbNume.Clear();  
        tbNota.Clear();  
        tbNume.Focus();  
    }  
  
    private static void ShowValidationError(string message)  
    {  
        MessageBox.Show(message, "Date invalide",  
                          MessageBoxButtons.OK, MessageBoxIcon.Warning);  
    }  
}  
}
```

Models/StudentGrade.cs

```
using System;  
  
namespace ManagerNote.Models  
{  
    public class StudentGrade  
    {  
        public StudentGrade(string studentName, double grade)  
        {  
            StudentName = studentName ??  
                throw new ArgumentNullException(nameof(studentName));  
            Grade = grade;  
        }  
  
        public string StudentName { get; }  
        public double Grade { get; }  
  
        public override string ToString() => $"{StudentName} - {Grade:F2}";  
    }  
}
```

Services/GradeBook.cs

```
using System;  
using System.Collections.Generic;  
using System.Globalization;  
using System.Linq;  
using ManagerNote.Models;  
  
namespace ManagerNote.Services  
{  
    public sealed class GradeBook  
    {  
        private readonly List<StudentGrade> _entries = new List<StudentGrade>();  
    }  
}
```

```
public IReadOnlyList<StudentGrade> Entries => _entries;

public int Count => _entries.Count;

public void Add(string studentName, string gradeText)
{
    var normalizedName = NormalizeStudentName(studentName);
    var grade = ParseGrade(gradeText);

    _entries.Add(new StudentGrade(normalizedName, grade));
}

public void RemoveAt(int index)
{
    if (index < 0 || index >= _entries.Count)
    {
        throw new ArgumentOutOfRangeException(
            nameof(index),
            "Indicele selectat nu este valid.");
    }

    _entries.RemoveAt(index);
}

public double GetAverage()
{
    EnsureHasEntries();
    return _entries.Average(entry => entry.Grade);
}

public StudentGrade GetMaxGradeEntry()
{
    EnsureHasEntries();
    return _entries.OrderByDescending(entry => entry.Grade).First();
}

public StudentGrade GetMinGradeEntry()
{
    EnsureHasEntries();
    return _entries.OrderBy(entry => entry.Grade).First();
}

public IReadOnlyList<StudentGrade> GetStudentsBelowAverage()
{
    var average = GetAverage();
    return _entries
        .Where(entry => entry.Grade < average)
        .OrderBy(entry => entry.StudentName)
        .ToList();
}

private static string NormalizeStudentName(string studentName)
{
    if (string.IsNullOrEmpty(studentName))
    {
        throw new ArgumentException(
            "Numele studentului este obligatoriu.");
    }
}
```

```
        var normalized = studentName.Trim();

        if (normalized.Any(char.IsDigit))
        {
            throw new ArgumentException(
                "Numele studentului nu poate conține cifre.");
        }

        return normalized;
    }

    private static double ParseGrade(string gradeText)
    {
        if (string.IsNullOrEmpty(gradeText))
        {
            throw new ArgumentException("Nota studentului este obligatorie.");
        }

        var normalized = gradeText.Trim().Replace(',', '.');

        if (!double.TryParse(normalized, NumberStyles.AllowDecimalPoint,
            CultureInfo.InvariantCulture, out var grade))
        {
            throw new ArgumentException(
                "Nota trebuie să fie un număr valid.");
        }

        if (grade < 1 || grade > 10)
        {
            throw new ArgumentOutOfRangeException(nameof(gradeText),
                "Nota trebuie să fie în intervalul [1, 10].");
        }

        return grade;
    }

    private void EnsureHasEntries()
    {
        if (_entries.Count == 0)
        {
            throw new InvalidOperationException(
                "Nu există note înregistrate.");
        }
    }
}
```

7.3. Exerciții

7.3.1. Exercițiul 1 – Extinderea aplicației Manager de note

Se cere extinderea aplicației „Manager de note” cu următoarele funcționalități:

- Adăugarea unui buton „Sortare” care sortează lista de studenți alfabetic și reafixează datele în ListBox;
- Adăugarea unui buton „Căutare” care, pe baza unui nume introdus într-un TextBox, afișează nota studentului respectiv sau un mesaj dacă studentul nu este găsit;
- Afișarea într-un ListView a studenților cu note peste medie (complementar funcționalității existente pentru studenții sub medie).

LABORATORUL 6

8. Fluxuri de intrare / ieșire (fișiere)

8.1. Fișiere în C#

Gestionarea fișierelor în C# permite programatorilor să creeze, să citească, să scrie și să administreze fișiere stocate pe disc. Platforma .NET oferă spațiul **System.IO**, care conține clase și metode pentru realizarea acestor operații într-un mod simplu și eficient. [6]

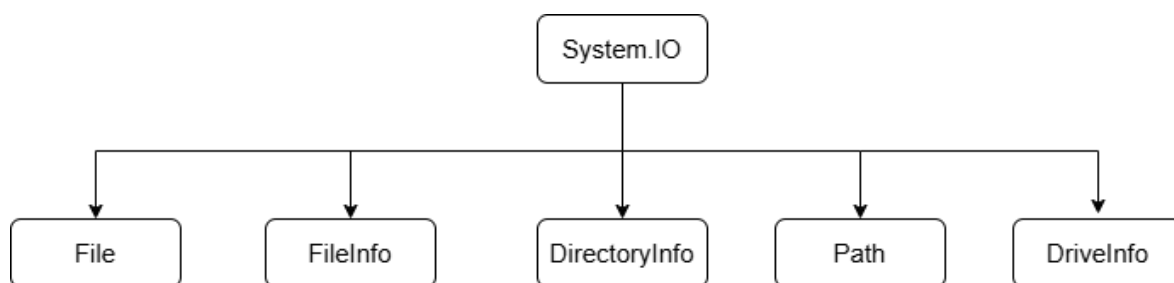


Fig. 6. Clasele pentru gestionarea fișierelor în C#.

Cele mai utilizate clase din spațiul *System.IO* includ: [6]

- **File** - Oferă metode statice pentru crearea, copierea, ștergerea, mutarea și deschiderea fișierelor;
- **FileInfo** - Oferă metode de instanță pentru operații asupra fișierelor, cu funcționalități suplimentare orientate pe obiect;
- **DirectoryInfo** - Oferă metode de instanță pentru operații asupra directoarelor;
- **Path** - Oferă metode statice pentru manipularea și gestionarea căilor fișierelor și directoarelor;
- **DriveInfo** - Oferă metode și proprietăți pentru obținerea informațiilor despre unitățile de stocare (hard disk-uri).

8.2. Clasa File în C#

Clasa **File** oferă metode încorporate care permit efectuarea operațiilor de intrare/ieșire asupra fișierelor.

8.2.1. Metode pentru informații despre fișier

Aceste metode permit obținerea de informații despre un fișier sau director fără a-l deschide sau modifica.

Tab. 4. Metode pentru redarea informațiilor unui fișier. [7]

Nume	Descriere
<code>Exists(string path)</code>	Determină dacă fișierul specificat există. Returnează <code>true</code> dacă fișierul există și <code>false</code> în caz contrar.
<code>GetAttributes(string path)</code>	Returnează attributele fișierului de la calea specificată.
<code>SetAttributes(string path, FileAttributes attributes)</code>	Setează attributele specificate ale fișierului de la calea indicată.
<code>GetCreationTime(string path)</code>	Returnează data și ora creării fișierului sau directorului specificat.
<code>GetLastAccessTime(string path)</code>	Returnează data și ora ultimei accesări a fișierului sau directorului specificat.
<code>GetLastWriteTime(string path)</code>	Returnează data și ora ultimei modificări a fișierului sau directorului specificat.

Exemple de attribute: [7]

- `ReadOnly` - Fișierul este doar pentru citire, nu poate fi modificat;
- `Directory` - Este un director (folder), nu un fișier obișnuit;
- `Encrypted` - Fișierul este criptat;
- `Archive` - Fișierul este marcat pentru backup sau arhivare;
- `Temporary` - Fișier temporar, folosit de aplicații pentru stocare intermediară.

Exemplu:

```
using System.IO;

private void btnVerifica_Click(object sender, EventArgs e)
{
    string path = "F:\\\\nume_fisier.txt";
    txtOutput.Clear();
}
```

```
// Exists
if (File.Exists(path))
{
    txtOutput.AppendText("Fisierul exista.\r\n");

    // GetAttributes
    FileAttributes attr = File.GetAttributes(path);
    txtOutput.AppendText("Attribute: " + attr + "\r\n");

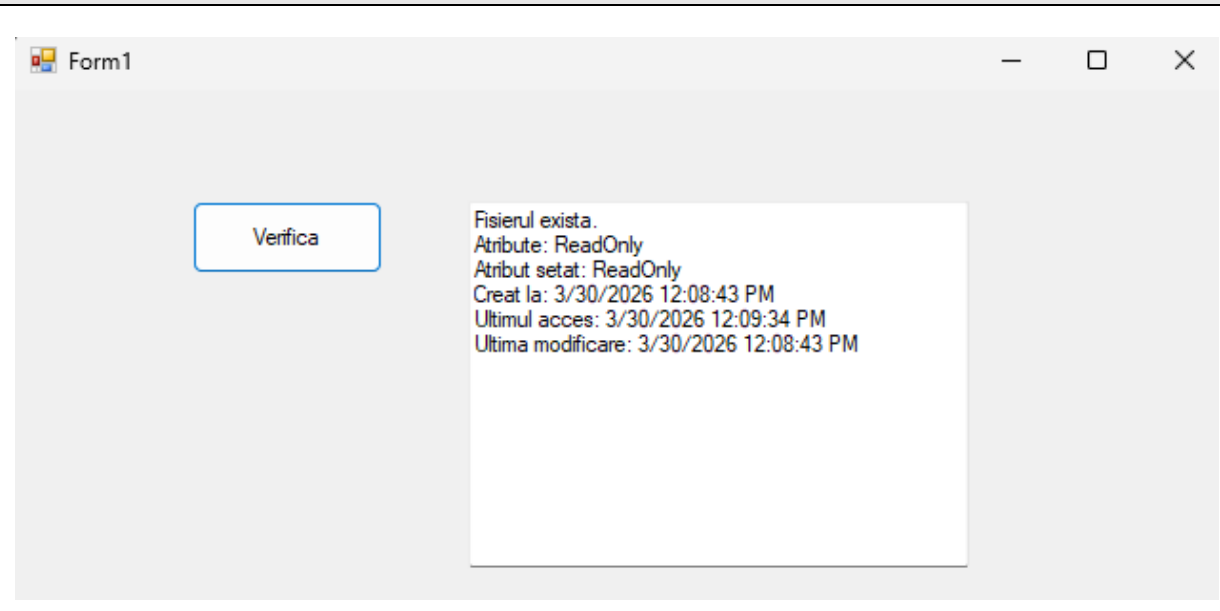
    // SetAttributes (ex: ReadOnly)
    File.SetAttributes(path, FileAttributes.ReadOnly);
    txtOutput.AppendText("Atribut setat: ReadOnly\r\n");

    // GetCreationTime
    DateTime creat = File.GetCreationTime(path);
    txtOutput.AppendText("Creat la: " + creat + "\r\n");

    // GetLastAccessTime
    DateTime acces = File.GetLastAccessTime(path);
    txtOutput.AppendText("Ultimul acces: " + acces + "\r\n");

    // GetLastWriteTime
    DateTime modificat = File.GetLastWriteTime(path);
    txtOutput.AppendText("Ultima modificare: " + modificat + "\r\n");
}
else
{
    txtOutput.AppendText("Fisierul NU există.");
}
}
```

Rezultatul rulării:



8.2.2. Metode de creare și scriere a unui fișier

Aceste metode sunt folosite pentru a genera fișiere noi pe disc și pentru a salva date în ele.

Tab. 5. Metode de creare și scriere a unui fișier. [7]

Nume	Descriere
Create(string path)	Deschide un fișier text, citește tot conținutul fișierului și apoi îl închide.
WriteAllText(string path, string contents)	Creează un fișier nou, scrie șirul specificat în fișier și apoi îl închide. Dacă fișierul țintă există deja, acesta este golit și suprascris.
AppendAllText(string path, string contents)	Deschide un fișier, adaugă șirul specificat la sfârșitul fișierului și apoi îl închide. Dacă fișierul nu există, metoda creează fișierul, scrie șirul în el și apoi îl închide.
WriteAllLines(string path, string[] contents)	Creează un fișier nou, scrie șirul de texte specificat în fișier și apoi îl închide.
AppendAllLines(string path, string[] contents)	Adaugă linii la un fișier folosind o codificare specificată și apoi închide fișierul. Dacă fișierul specificat nu există, metoda creează fișierul, scrie liniile date în el și apoi îl închide.

Exemplu:

```
using System.IO;

private void btnExecuta_Click(object sender, EventArgs e)
{
    string path = "F:\\nume_fisier.txt";

    richTxtOutput.Clear();

    // Exists
    if (!File.Exists(path))
    {
        richTxtOutput.AppendText("Fisierul nu exista. Se creeaza...\r\n");

        // Create
        File.Create(path).Close();
    }
    else
    {
        richTxtOutput.AppendText("Fisierul exista deja.\r\n\r\n");
    }

    // WriteAllText (suprascrie)
    File.WriteAllText(path, "Prima linie scrisa.\r\n");
    richTxtOutput.AppendText("WriteAllText executat.\r\n\r\n");
}
```

```
// AppendAllText (adauga)
File.AppendAllText(path, "Linie adaugata cu AppendAllText.\r\n");
richTxtOutput.AppendText("AppendAllText executat.\r\n\r\n");

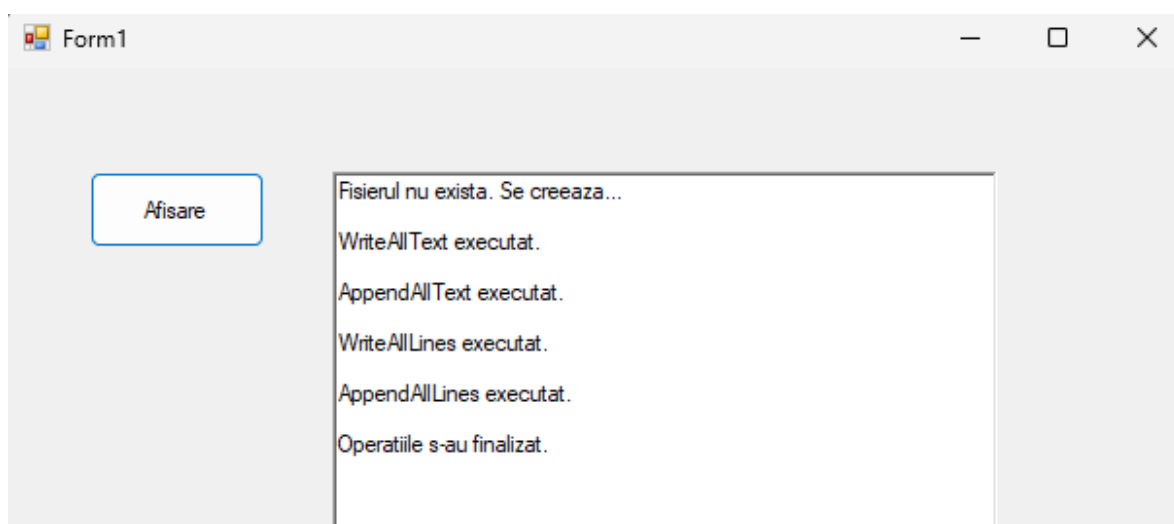
// WriteAllLines (suprascrie)
string[] linii = { "Prima linie", "A doua linie", "A treia linie" };
File.WriteAllLines(path, linii);
richTxtOutput.AppendText("WriteAllLines executat.\r\n\r\n");

// AppendAllLines (adauga)
string[] liniiNoui = { "Linia 4", "Linia 5" };
File.AppendAllLines(path, liniiNoui);
richTxtOutput.AppendText("AppendAllLines executat.\r\n\r\n");

richTxtOutput.AppendText("Operatiile s-au finalizat.");

}
```

Rezultatul rulării:



8.2.3. Metode de citire dintr-un fișier

Aceste metode sunt folosite pentru a prelua datele salvate anterior și a le utiliza în program.

Tab. 6. Metode de citire dintr-un fișier. [7]

Nume	Descriere
ReadAllText(string path)	Deschide un fișier text, citește tot conținutul fișierului și apoi îl închide.
ReadAllLines(string path)	Deschide un fișier text, citește toate liniile fișierului și apoi îl închide.
ReadLines(string path)	Citește liniile unui fișier folosind o codificare specificată.

Deși toate trei metodele permit citirea fișierelor, ele se deosebesc prin caracteristici specifice:

- `ReadAllText(string path):`
 - Returnează un singur șir de caractere (`string`) care conține întreg fișierul;
 - Este potrivit pentru prelucrări de text precum: căutări, înlocuiri, parsare;
 - Încarcă tot fișierul în memorie fiind astfel mai puțin indicat pentru fișiere foarte mari.
- `ReadAllLines(string path):`
 - Încarcă în memorie toate liniile din fișier (o linie este o secvență de caractere care se termină cu LF sau CR+LF) și le returnează ca tablouri de șiruri de caractere (`string[]`);
 - Fiecare element din tablou reprezintă o linie din fișier; liniile sunt separate de LF (`\n`) sau CR+LF (`\r\n`); caracterele de sfârșit de linie nu se includ în șirurile de caractere ale liniilor;
 - Potrivit pentru fișiere de dimensiune mică.
- `ReadLines(string path):`
 - Citește fișierul linie cu linie, folosind "lazy evaluation", adică fișierul nu este încărcat complet în memorie; fiecare linie este încărcată pe rând, pe măsură ce este cerută;
 - Potrivit pentru fișiere de dimensiune mare.

Exemplu:

```
using System.IO;
private void btnExecuta_Click(object sender, EventArgs e)
private void btnCitire_Click(object sender, EventArgs e)
{
    string path = "F:\\nume_fisier.txt";

    richTextBoxOutput.Clear();

    // Exists
    if (File.Exists(path))
    {
        richTextBoxOutput.AppendText("Fișierul exista.\r\n\r\n");

        // ReadAllText
        string continut = File.ReadAllText(path);
        richTextBoxOutput.AppendText("=== ReadAllText ===\r\n");
        richTextBoxOutput.AppendText(continut + "\r\n\r\n");

        // ReadAllLines
        string[] linii = File.ReadAllLines(path);
        richTextBoxOutput.AppendText("=== ReadAllLines ===\r\n");
```

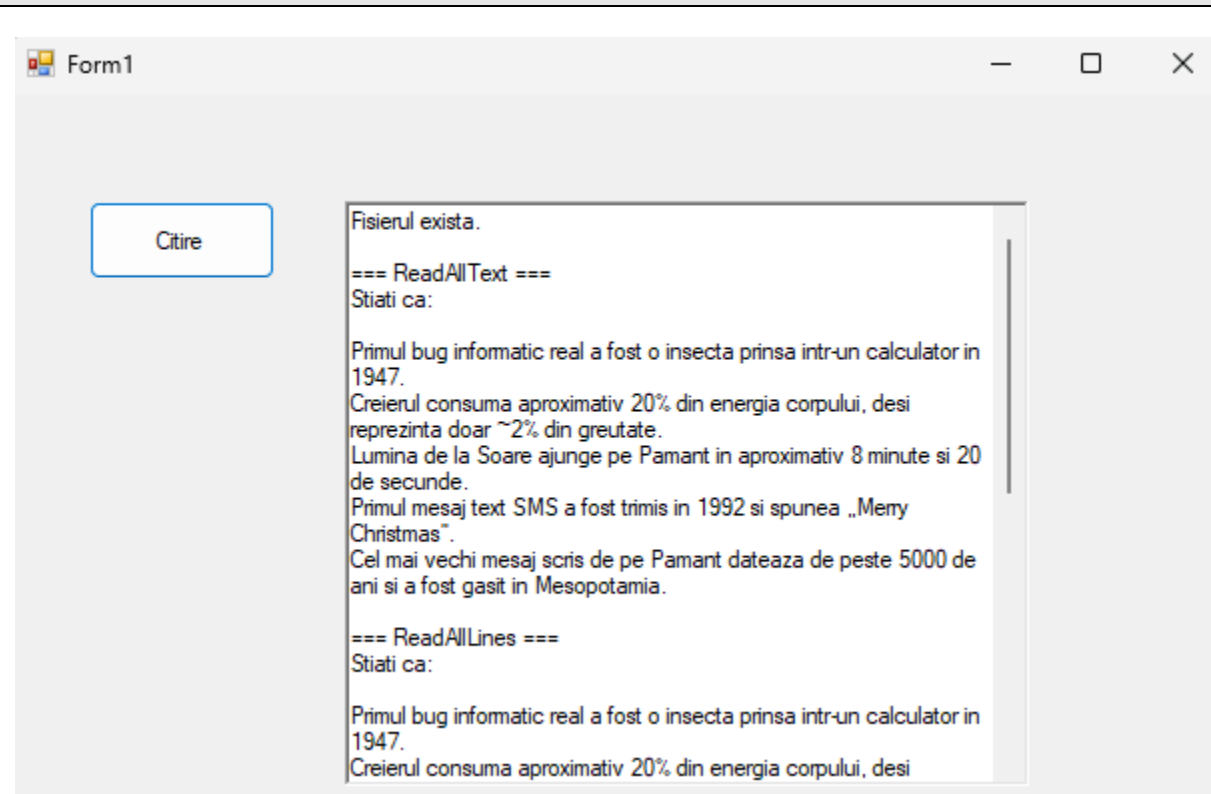
```
foreach (string linie in linii)
{
    richTxtOutput.AppendText(linie + "\r\n");
}

richTxtOutput.AppendText("\r\n");

// ReadLines (citire pe rand)
richTxtOutput.AppendText("=== ReadLines ===\r\n");

foreach (string linie in File.ReadLines(path))
{
    richTxtOutput.AppendText(linie + "\r\n");
}
}
else
{
    richTxtOutput.AppendText("Fisierul NU exista.");
}
}
```

Rezultatul rulării:



8.2.4. Metode de copiere, mutare și ștergere a unui fișier

Aceste metode permit gestionarea fișierelor pe disc (organizare, curățare, copiere etc.).

Tab. 7. Metode de copiere, mutare și ștergere a unui fișier. [7]

Nume	Descriere
Copy(string sourceFileName, string destFileName)	Copiază un fișier existent într-un fișier nou. Suprascrierea unui fișier cu același nume nu este permisă.
Copy(string sourceFileName, string destFileName, bool overwrite)	Copiază un fișier existent într-un fișier nou. Suprascrierea unui fișier cu același nume este permisă.
Move(string sourceFileName, string destFileName)	Mută un fișier specificat într-o locație nouă, oferind opțiunea de a-i da un nou nume.
Delete(string path)	Șterge fișierul specificat.

Exemplu:

```
using System.IO;

private void btnOperatiiFisier_Click(object sender, EventArgs e)
{
    string fisierOriginal = "F:\\\\nume_fisier_original.txt";
    string fisierCopiat = "F:\\\\nume_fisier_copiat.txt";
    string fisierMutat = "F:\\\\nume_fisier_mutat.txt";

    richTxtOutput.Clear();
    // Verificam daca fisierul original exista
    if (!File.Exists(fisierOriginal))
    {
        File.WriteAllText(fisierOriginal, "Acesta este un fisier original.\r\n");
        richTxtOutput.AppendText("Fisier original creat.\r\n\r\n");
    }
    // Copiere fara overwrite
    if (!File.Exists(fisierCopiat))
    {
        File.Copy(fisierOriginal, fisierCopiat);
        richTxtOutput.AppendText(
            $"Fisier copiat ca '{fisierCopiat}' fara overwrite.\r\n\r\n");
    }
    // Copiere cu overwrite = true
    File.Copy(fisierOriginal, fisierCopiat, true);
    richTxtOutput.AppendText(
        $"Fisier copiat ca '{fisierCopiat}' cu overwrite.\r\n\r\n");

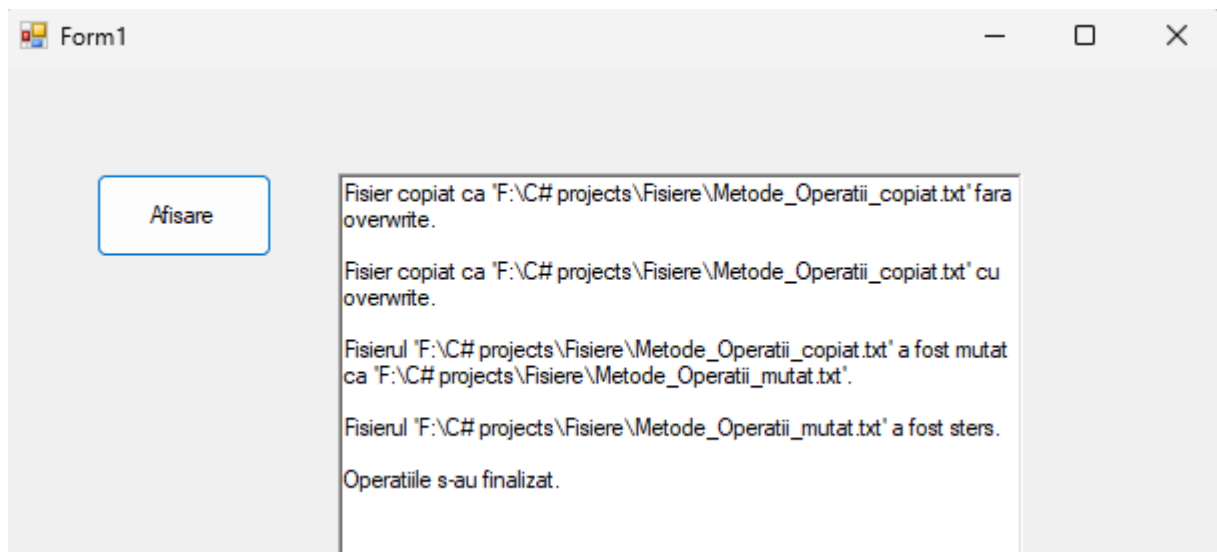
    // Mutare fisier
    if (File.Exists(fisierMutat))
    {
        File.Delete(fisierMutat);
    }

    File.Move(fisierCopiat, fisierMutat);
    richTxtOutput.AppendText(
        $"Fisierul '{fisierCopiat}' a fost mutat ca '{fisierMutat}'.\r\n\r\n");
}
```

```
// Stergere fisier
if (File.Exists(fisierMutat))
{
    File.Delete(fisierMutat);
    richTxtOutput.AppendText(
        $"Fisierul '{fisierMutat}' a fost sters.\r\n\r\n");
}

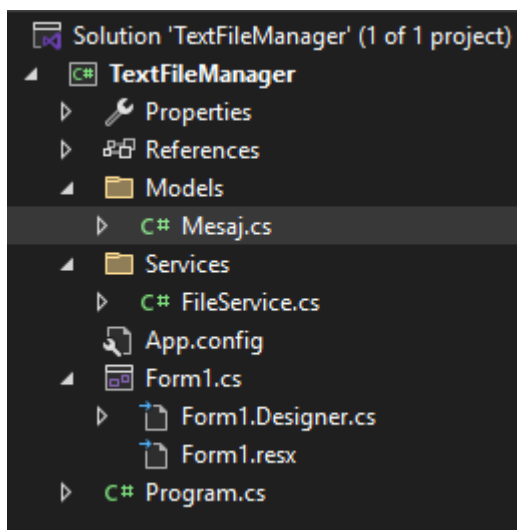
richTxtOutput.AppendText("Operatiile s-au finalizat.");
}
```

Rezultatul rulării:

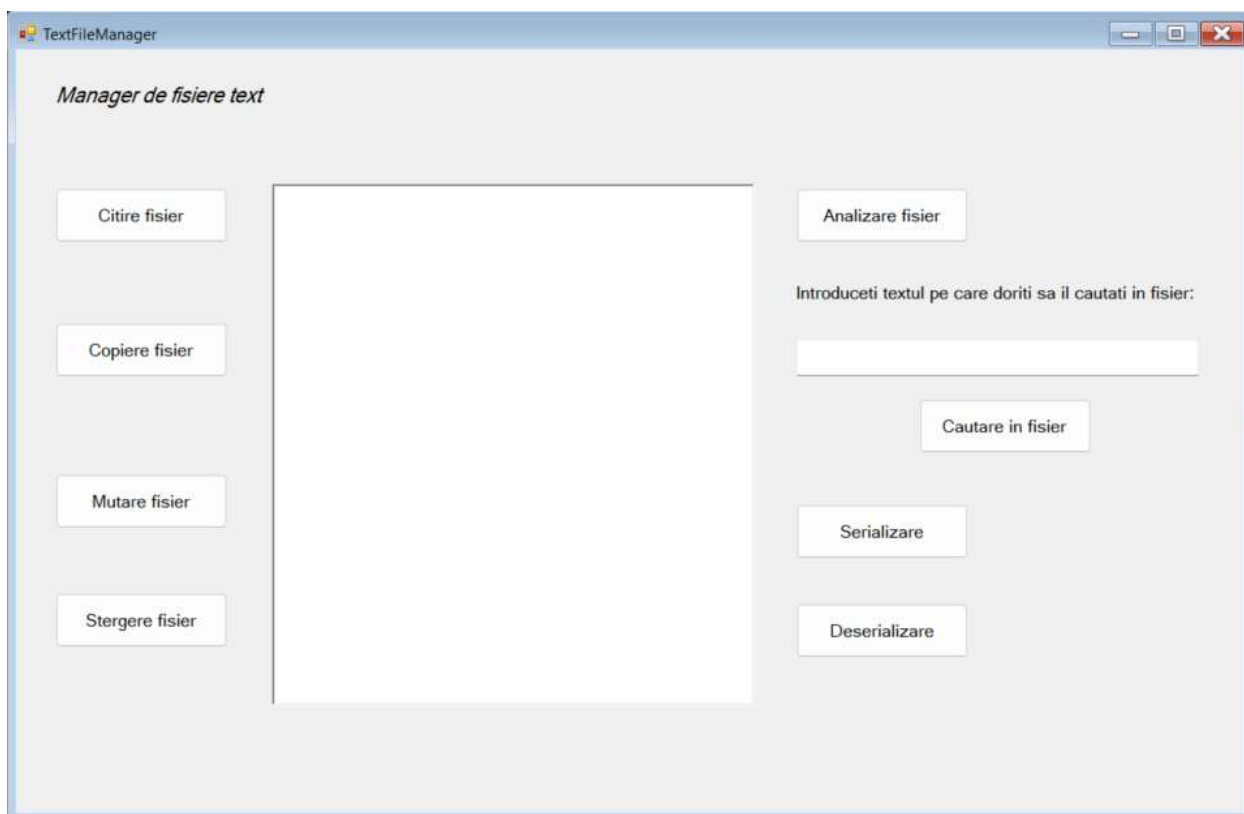


8.3. Aplicație cu fișiere – TextFile Manager

8.3.1. Structura proiectului:



8.3.2. Design grafic – Form1



8.3.3. Proprietăți și evenimente pentru controale

Tip control	Nume (Name)	Setări relevante	Rol
Form	Form1	Text = "TextFileManager" Load → Form1_Load	Formular principal al aplicației
TextBox	tbCautare	-	Permite utilizatorului să introducă un text pentru căutare în fișier.
RichTextBox	richTxtOutput	-	Afișează conținutul fișierului, rezultatele analizelor, căutărilor sau mesajelor serializate/deserializate.
Button	btnReadAllText	Text = "Copiere fisier " Click → btnReadAllText_Click	Afișează tot conținutul fișierului principal în richTxtOutput.
Button	btnCopy	Text = "Stergere" Click → btnCopy_Click	Copiază fișierul principal în fișierul de backup.
Button	btnMove	Text = "Mutare fisier" Click → btnMove_Click	Mută fișierul de backup într-o destinație aleasă.
Button	btnDelete	Text = "Stergere fisier" Click → btnDelete_Click	Șterge fișierul specificat (principal sau de backup).

Tip control	Nume (Name)	Setări relevante	Rol
Button	btnAnaliza	Text = "Analizare fisier" Click → btnAnaliza_Click	Numără caracterele, cuvintele și propozițiile din fișier și afișează rezultatul.
Button	btnCauta	Text = "Cautare in fisier" Click → btnCauta_Click	Caută textul introdus în tbCautare în fișier și afișează liniile găsite în richTxtOutput.
Button	btnSerializare	Text = "Serializare" Click → btnSerializare_Click	Salvează lista de mesaje într-un fișier binar (serializare).
Button	btnDeserializare	Text = "Deserializare" Click → btnDeserializare_Click	Încarcă lista de mesaje din fișierul binar (deserializare) și afișează conținutul în richTxtOutput.

8.3.4. Cod sursă

Services/FileService.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Runtime.Serialization.Formatters.Binary;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using TextFileManager.Models;

namespace TextFileManager.Services
{
    public class FileService
    {
        // Proprietati pentru calea fisierului principal si a fisierului de backup
        public string FilePath { get; } // calea fisierului principal
        public string BackupPath { get; } // calea fisierului de backup

        // Constructor - primeste ca parametri calea fisierului principal
        // si a backup-ului
        public FileService(string filePath, string backupPath)
        {
            FilePath = filePath;
            BackupPath = backupPath;

            // Creeam fisierul principal daca nu exista
            if (!File.Exists(FilePath))
            {
                File.WriteAllText(FilePath,
                    "Mesaj de test.\r\nAlt mesaj.\r\nInca un mesaj.");
            }
        }

        // CITIRE FISIER
    }
}
```

```
// Citim tot continutul fisierului ca un singur string
public string ReadAllText() => File.ReadAllText(FilePath);

// Citim fisierul linie cu linie si returneaza un array de stringuri
public string[] ReadAllLines() => File.ReadAllLines(FilePath);

// OPERATII FISIER
// Copiere a fisierului principal in fisierul de backup
// Parametrul overwrite = true suprascrie backup-ul daca exista
public void Copy(bool overwrite = true) =>
    File.Copy(FilePath, BackupPath, overwrite);

// Mutam fisierul de la backup la o destinatie noua
public void Move(string destinationPath)
{
    // Daca exista deja fisierul la destinatie atunci il stergem
    if (File.Exists(destinationPath)) File.Delete(destinationPath);
    File.Move(BackupPath, destinationPath);
}

// Stergem un fisier daca acesta exista
public void Delete(string path)
{
    if (File.Exists(path)) File.Delete(path);
}

// ANALIZAREA TEXTULUI
// Analizam textul din fisier: numara caractere, cuvinte si propozitii
public (int caractere, int cuvinte, int propozitii) AnalyzeText()
{
    // citeste tot continutul
    string continut = ReadAllText();
    // numara caracterele
    int caractere = continut.Length;
    // numara cuvintele
    int cuvinte = Regex.Matches(continut, @"\w+").Count;
    // numara propozitiile
    int propozitii = Regex.Matches(continut, @"[.!?]").Count;
    return (caractere, cuvinte, propozitii);
}

// CAUTARE FISIER
// Cautam un text in fisier si returnam liniile in care apare
public List<string> Search(string text)
{
    // lista pentru rezultate
    List<string> rezultate = new List<string>();
    // citeste fisierul linie cu linie
    string[] linii = ReadAllLines();
    for (int i = 0; i < linii.Length; i++)
    {
        if (linii[i].Contains(text))
            rezultate.Add($"Linia {i + 1}: {linii[i]}");
    }
    return rezultate;
}

// SERIALIZARE / DESERIALIZARE
// Salvam lista de mesaje intr-un fisier binar
public void SerializeMessages(List<Mesaj> messages, string path)
{
    using (FileStream fs = new FileStream(path, FileMode.Create))
    {
        BinaryFormatter bf = new BinaryFormatter();
    }
}
```

```
        bf.Serialize(fs, messages);
    }
}

// Citim lista de mesaje dintr-un fisier binar
public List<Mesaj> DeserializeMessages(string path)
{
    // daca fisierul nu exista, returneaza lista vida
    if (!File.Exists(path))
    {
        return new List<Mesaj>();
    }

    using (FileStream fs = new FileStream(path, FileMode.Open))
    {
        BinaryFormatter bf = new BinaryFormatter();
        return (List<Mesaj>)bf.Deserialize(fs);
    }
}
}
```

Models/Mesaj.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TextFileManager.Models
{
    [Serializable]
    public class Mesaj
    {
        public string Text { get; set; }

        public Mesaj() { }

        public Mesaj(string text)
        {
            Text = text;
        }

        public override string ToString() => Text;
    }
}
```

Form1.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
```

```
using System.Runtime.Remoting.Messaging;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using TextFileManager.Models;
using TextFileManager.Services;

namespace TextFileManager
{
    public partial class Form1 : Form
    {
        private FileService _fileService;
        private List<Mesaj> _messages;
        private string mainFile;
        private string backupFile;
        private string serializePath;

        public Form1()
        {
            InitializeComponent();
            _messages = new List<Mesaj>();
            mainFile = "F:\\C# projects\\mesaje.txt";
            backupFile = "F: \\mesaje_backup.txt";
            serializePath = "F:\\ mesaje.dat";
            _fileService = new FileService(mainFile, backupFile);
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            richTxtOutput.Clear();
            _messages = _fileService.DeserializeMessages("mesaje.dat");
        }

        private void btnReadAllText_Click(object sender, EventArgs e)
        {
            richTxtOutput.Text = _fileService.ReadAllText();
        }

        private void btnCopy_Click(object sender, EventArgs e)
        {
            _fileService.Copy(true);
            MessageBox.Show("Fisier copiat cu overwrite!");
        }

        private void btnMove_Click(object sender, EventArgs e)
        {
            _fileService.Move(mainFile);
            MessageBox.Show("Fisier mutat!");
        }

        private void btnDelete_Click(object sender, EventArgs e)
        {
            _fileService.Delete(backupFile);
            MessageBox.Show("Fisier sters!");
        }

        private void btnAnaliza_Click(object sender, EventArgs e)
        {
            var (caractere, cuvinte, propozitii) = _fileService.AnalyzeText();
            richTxtOutput.Clear();

            richTxtOutput.AppendText(
```

```
$"Caractere: {caractere}\r\nCuvinte: {cuvinte}\r\nPropozitii: {propozitii}");
}

private void btnCauta_Click(object sender, EventArgs e)
{
    string cauta = tbCautare.Text.Trim();
    var rezultate = _fileService.Search(cauta);
    richTxtOutput.Clear();
    if (rezultate.Count == 0) richTxtOutput.AppendText(
        "Nu s-a gasit textul cautat.");
    else foreach (var r in rezultate)
        richTxtOutput.AppendText(r + "\r\n");
}

private void btnSerializare_Click(object sender, EventArgs e)
{
    if (!string.IsNullOrEmpty(tbCautare.Text))
    {
        _messages.Add(new Mesaj(tbCautare.Text));
        _fileService.SerializeMessages(_messages, serializePath);
        MessageBox.Show("Mesaj salvat!");
    }
}

private void btnDeserializare_Click(object sender, EventArgs e)
{
    _messages = _fileService.DeserializeMessages(serializePath);
    richTxtOutput.Clear();
    foreach (var msg in _messages)
        richTxtOutput.AppendText(msg.Text + "\r\n");
}
}
```

8.4. Exerciții

8.4.1. Exercițiul 1

Se dă un fișier text cu numere (câte unul pe linie).

Aplicația trebuie să:

- citească valorile
- calculeze:
 - suma
 - media
 - minim și maxim
- afișeze rezultatele în interfață

Cerințe tehnice:

- conversie `string` → `int` (`int.Parse` sau `TryParse`)

- ignorarea liniilor invalide

Exemplu fișier:

```
10  
25  
abc  
40
```

→ abc trebuie ignorat.

LABORATORUL 7

9. Programare în rețea

Programarea rețelelor de calculatoare constă în dezvoltarea de programe care permit proceselor să comunice între ele prin intermediul unei rețele de calculatoare. În mod foarte general, majoritatea comunicațiilor pot fi împărțite în două categorii:

- orientate pe conexiune (*connection-oriented*) – necesită stabilirea unei conexiuni înainte de transmitere;
- fără conexiune (*connectionless*) – trimit date fără stabilirea unei conexiuni prealabile.

Faptul că o comunicație este orientată pe conexiune sau fără conexiune este stabilit de protocolul de comunicație, și nu de interfața de programare a aplicațiilor (API). Exemple de protocoale orientate pe conexiune sunt *Transmission Control Protocol* (TCP) și *Sequenced Packet Exchange* (SPX), iar exemple de protocoale fără conexiune sunt *User Datagram Protocol* (UDP), „IP brut” (raw IP) și *Internetwork Packet Exchange* (IPX). [8]

9.1. Modelul client-server

Pentru comunicațiile orientate pe conexiune, părțile implicate au, de regulă, roluri diferite. Una dintre părți așteaptă, de obicei, conexiuni de intrare; această parte este denumită în mod obișnuit „server”. Cealaltă parte este cea care inițiază conexiunea; aceasta este numită, de regulă, „client”.

În comunicațiile fără conexiune, o entitate trimite pachete de date fără a stabili în prealabil o conexiune, iar cealaltă entitate le poate recepționa dacă este disponibilă, fără a exista o relație persistentă între cele două. [8]

Programarea rețelelor acoperă mai multe straturi ale modelului de referință OSI/ISO (Figura 20 - Stiva OSI Figura 20).

Nivelul Fizic (Physical) se ocupă de transmiterea efectivă a biților (0 și 1) prin mediul fizic, cum ar fi cabluri, fibră optică sau semnal wireless. Aici sunt definite caracteristici precum tensiunile, semnalele și conectorii.

Nivelul Legătură de Date (Data Link) asigură transmisia corectă între două dispozitive conectate direct. Acest nivel organizează datele în cadre (frames), detectează erorile și utilizează adrese MAC pentru identificarea dispozitivelor.

Nivelul Rețea (Network) este responsabil pentru rutarea pachetelor între rețele diferite. El stabilește traseul pe care îl urmează datele și utilizează adrese IP pentru identificare.

Nivelul Transport (Transport) asigură livrarea corectă și completă a datelor între aplicații. Aici se realizează controlul erorilor și al fluxului. Protocoale precum TCP oferă transmisie sigură, iar UDP oferă transmisie rapidă fără garanții.

Nivelul Sesiune (Session) gestionează deschiderea, menținerea și închiderea sesiunilor de comunicare între aplicații și asigură sincronizarea acestora.

Nivelul Prezentare (Presentation) se ocupă de formatul datelor, realizând conversii între formate diferite, criptare și decriptare, precum și compresia datelor.

Nivelul Aplicație (Application) reprezintă nivelul cel mai apropiat de utilizator, oferind servicii de rețea aplicațiilor, cum ar fi HTTP, FTP sau SMTP.

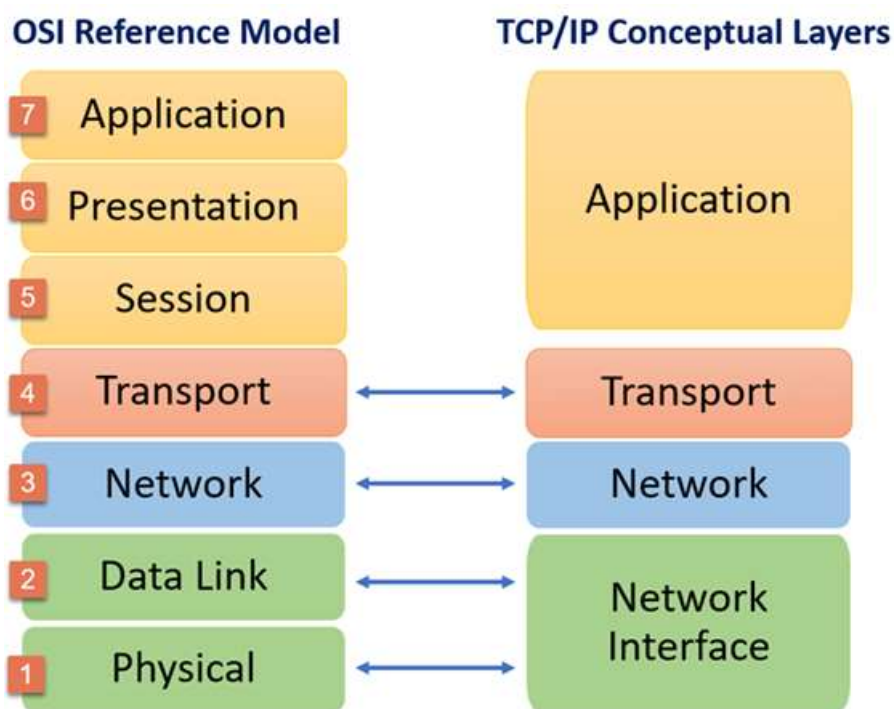


Figura 20 - Stiva OSI [9]

În practică, programatorul de aplicații lucrează cel mai des cu două categorii de protocoale: protocoalele de nivel transport (L4) – care asigură transmiterea fiabilă sau rapidă a datelor

între două procese – și protocoalele de nivel aplicație (L7) – care definesc semantica mesajelor schimbate de aplicații (de exemplu, cum cere un browser o pagină web).

Tabelul de mai jos (Tabel 12) sintetizează câteva protocoale populare și nivelul OSI/ISO căruia îi aparțin.

Tabel 12 - Straturi OSI

Stratul OSI/ISO	Protocol
L3 (network)	IP
L4 (transport)	TCP, UDP, SCTP
L5 (session)	TLS
L7 (application)	HTTP

9.2. Adrese IP

Pentru ca o comunicație să funcționeze, un computer sau un dispozitiv are nevoie de o adresă. Internetul utilizează două sisteme de adresare: [1]

- IPv4 - În prezent, sistemul de adresare dominant; adresele IPv4 au o lungime de 32 de biți. Atunci când sunt scrise sub formă de șir, adresele IPv4 sunt reprezentate ca patru numere zecimale separate prin puncte (de exemplu, 101.102.103.104). O adresă poate fi unică la nivel global sau unică într-o anumită subrețea (de exemplu, într-o rețea corporativă).
- IPv6 - Sistemul de adresare mai nou, pe 128 de biți. Adresele sunt reprezentate sub formă de șiruri în format hexazecimal, separate prin două puncte (de exemplu, [3EA0:FFFF:198A:E4A3:4FF2:54fA:41BC:8D31]). În .NET este necesar să adăugați paranteze drepte în jurul adresei.

Clasa `IPAddress` din spațiul `System.Net` reprezintă o adresă în oricare dintre cele două protocoale. Aceasta are un constructor care acceptă un tablou de octeți (`byte array`) și o metodă statică `Parse`, care acceptă un șir de caractere formatat corect. [1]

9.3. DNS

Domain Name System (DNS) este un serviciu de denumire ierarhic și distribuit, care oferă un sistem de atribuire a numelor pentru calculatoare, servicii și alte resurse de pe Internet sau din alte rețele bazate pe Internet Protocol (IP). Acesta asociază diverse informații cu numele de

domeniu (șiruri de identificare) atribuite fiecărei entități. Cel mai important rol al său este de a traduce numele de domeniu, ușor de reținut, în adrese IP numerice, necesare pentru localizarea și identificarea serviciilor și dispozitivelor din rețea, utilizând protocoalele de bază ale acesteia. DNS delegă responsabilitatea atribuirii numelor de domeniu și a mapării acestora către resursele Internet prin desemnarea unor servere de nume pentru fiecare domeniu. Administratorii de rețea pot delega autoritatea asupra subdomeniilor din spațiul lor de nume către alte servere de nume. [10]

O analogie frecvent utilizată pentru a explica DNS-ul este aceea că acesta funcționează ca o carte de telefoane pentru Internet, traducând numele de gazdă ușor de reținut de către oameni în adrese IP. De exemplu, numele de gazdă **www.example.com** din cadrul domeniului *example.com* se traduce în adresele *93.184.216.34* (IPv4) și *2606:2800:220:1:248:1893:25c8:1946* (IPv6). DNS-ul poate fi actualizat rapid și transparent, permițând schimbarea locației unui serviciu în rețea fără a afecta utilizatorii finali, care continuă să folosească același nume de gazdă. [10]

9.4. Porturi și socket-uri

Adresa IP identifică în mod unic o gazdă (un computer sau un dispozitiv) într-o rețea, însă pe o singură gazdă pot rula simultan zeci sau sute de procese care folosesc rețeaua: un browser, un client de e-mail, un server web, un client de mesagerie etc. Pentru a putea distinge între ele, sistemul de operare asociază fiecărui proces care comunică prin rețea unul sau mai multe numere întregi numite porturi. Un port este un număr pe 16 biți (între 0 și 65535) care, împreună cu adresa IP a gazdei, identifică în mod unic capătul unei conexiuni de rețea.

Perechea (adresă IP, port) se numește socket. O conexiune TCP între două procese este, din perspectiva sistemului de operare, o pereche de socket-uri: socket-ul local (adresa IP și portul procesului client) și socket-ul de la distanță (adresa IP și portul procesului server). În C#, clasele `TcpClient`, `TcpListener` și `UdpClient` din spațiul `System.Net.Sockets` oferă o interfață de nivel înalt peste API-ul de socket-uri al sistemului de operare.

Porturile sunt împărțite, prin convenție stabilită de IANA (*Internet Assigned Numbers Authority*), în trei intervale:

1. Porturi *well-known* (0–1023) – sunt rezervate pentru serviciile standard ale Internetului și, pe sistemele de tip Unix, pot fi folosite doar de procesele care rulează cu

privilegii de administrator. Exemple: 20 și 21 (FTP), 22 (SSH), 25 (SMTP), 53 (DNS), 80 (HTTP), 110 (POP3), 143 (IMAP), 443 (HTTPS).

2. Porturi înregistrate (1024–49151) — sunt alocate de IANA, la cerere, pentru aplicații specifice. Exemple: 3306 (MySQL), 5432 (PostgreSQL), 1433 (Microsoft SQL Server), 8080 (servere web alternative).
3. Porturi dinamice sau efemere (49152–65535) — sunt folosite de sistemul de operare pentru a aloca automat un port local proceselor client care inițiază conexiuni. Aceste porturi nu pot fi rezervate pentru un serviciu anume.

În exemplele din secțiunile următoare, serverele TCP și UDP vor asculta pe portul 5000, ales arbitrar din intervalul porturilor înregistrate, pentru a evita conflictele cu serviciile standard ale sistemului.

9.5. Exemplu cu IP și DNS

În exemplul de mai jos, se preia un nume de domeniu introdus de utilizator (de exemplu, `google.com`) și se realizează conversia acestuia în adrese IP prin intermediul serviciului DNS. Ulterior, lista anterioară este ștearsă, iar toate adresele IP obținute sunt afișate într-un `ListBox`. În situația în care nu este găsită nicio adresă IP asociată domeniului, este afișat un mesaj corespunzător.

Exemplu:

```
private void btnAfisare_Click(object sender, EventArgs e)
{
    try
    {
        string domain = txtDomain.Text;

        listBoxIP.Items.Clear();
        IPAddress[] addresses = Dns.GetHostAddresses(domain);

        foreach (IPAddress ip in addresses)
        {
            listBoxIP.Items.Add(ip.ToString());
        }

        if (addresses.Length == 0)
        {
            listBoxIP.Items.Add("Nu s-au gasit IP-uri.");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Eroare: " + ex.Message);
    }
}
```

9.6. Protocoale de nivel transport (L4)

Nivelul transport (L4) din modelul OSI/ISO are sarcina de a livra date între două procese care rulează pe gazde diferite. Cele două protocoale principale la acest nivel sunt: TCP, care asigură o livrare fiabilă și ordonată într-un flux de octeți, și UDP, care oferă o livrare rapidă, fără garanții, sub formă de datagrame independente. Alegerea între cele două depinde de cerințele aplicației, conform comparației din tabelul de mai jos.

Tabel 13. Comparație între TCP și UDP

Caracteristică	TCP	UDP
Orientare pe conexiune	Da (three-way handshake)	Nu
Fiabilitate	Garantată (retransmisie automată)	Fără garanții
Ordonarea pachetelor	Garantată	Nu este garantată
Control de congestie	Da	Nu
Control de flux	Da	Nu
Overhead per pachet	Mai mare (antet 20 octeți + opțiuni)	Mai mic (antet 8 octeți)
Latență	Mai mare (handshake, confirmări)	Mai mică
Mod de transmisie	Flux de octeți (stream)	Datagrame independente
Cazuri tipice de utilizare	HTTP/HTTPS, FTP, SSH, SMTP, transfer de fișiere	DNS, DHCP, VoIP, streaming video, jocuri online

9.6.1. TCP

Transmission Control Protocol (TCP) este unul dintre principalele protocoale ale suitei de protocoale Internet, oferind livrarea fiabilă, ordonată și verificată a erorilor a unui flux de octeți (bytes) între aplicații care rulează pe gazde ce comunică printr-o rețea IP. Acesta a apărut în implementarea inițială a rețelei, în care completa Internet Protocol (IP). Prin urmare, întreaga suită este denumită în mod obișnuit TCP/IP. [11]

TCP este orientat pe conexiune, ceea ce înseamnă că expeditorul și receptorul trebuie mai întâi să stabilească o conexiune pe baza unor parametri conveniți; acest lucru se realizează printr-o procedură de tip „three-way handshake” (strângere de mână în trei pași). Procesul implică trei pași: clientul trimite segmentul/flag-ul SYN, serverul răspunde cu SYN-ACK, iar clientul confirmă cu ACK, stabilind astfel conexiunea.

Serverul trebuie să fie în starea de ascultare (deschidere pasivă) pentru cererile de conexiune venite de la clienți înainte ca o conexiune să fie stabilită.

Procedura „three-way handshake”, retransmisia și detectarea erorilor contribuie la fiabilitate, dar cresc latența. TCP utilizează mecanisme de evitare a congestiei rețelei.

Totuși, există vulnerabilități în TCP, inclusiv la atacuri de tip refuz al serviciului (*Denial of service* - DoS), deturnarea conexiunii (connection hijacking), „TCP veto” și atacuri de tip reset. [11]

Un exemplu de comunicare prin TCP (serverul și clientul este în același proiect):

Form1.cs (Client)

```
private void button1_Click(object sender, EventArgs e)
{
    try
    {
        // Conectare la server (IP + PORT)
        TcpClient client = new TcpClient("127.0.0.1", 5000);

        // 2. Obținere stream de rețea
        NetworkStream stream = client.GetStream();

        // 3. Preluare mesaj din TextBox
        string message = textBox1.Text;

        // 4. Convertire text în bytes
        byte[] data = Encoding.UTF8.GetBytes(message);

        // 5. Trimiterea datelor
        stream.Write(data, 0, data.Length);

        // 6. Închidere conexiune
        stream.Close();
        client.Close();

        MessageBox.Show("Mesaj trimis cu succes!");
        MessageBox.Show("Ai trimis: " + message);
    }
    catch (Exception ex)
    {
        MessageBox.Show("Eroare: " + ex.Message);
    }
}
```

Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading.Tasks;
using System.Threading;
using System.Windows.Forms;

namespace NetworkProgramming
{
```

```
internal static class Program
{
    [STAThread]
    static void Main()
    {
        // Pornire server pe un thread separat
        Thread serverThread = new Thread(StartServer);
        serverThread.IsBackground = true;
        serverThread.Start();

        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new Form1());
    }

    static void StartServer()
    {
        TcpListener server = new TcpListener(IPAddress.Any, 5000);
        server.Start();

        Console.WriteLine("Server pornit...");

        while (true)
        {
            TcpClient client = server.AcceptTcpClient();
            NetworkStream stream = client.GetStream();

            byte[] buffer = new byte[1024];
            int bytes = stream.Read(buffer, 0, buffer.Length);

            string message = Encoding.UTF8.GetString(buffer, 0, bytes);

            Console.WriteLine("Primit: " + message);

            stream.Close();
            client.Close();
        }
    }
}
```

Notă privind operațiile asincrone

Exemplul de mai sus folosește, pentru claritate, metodele sincrone `Write` și `Read`, iar serverul rulează pe un thread (fir de execuție) separat, creat explicit. În aplicațiile reale, operațiile de rețea blochează thread-ul apelant până la finalizare, ceea ce, dacă s-ar întâmpla pe thread-ul UI, ar îngheța întreaga fereastră. De aceea, în practică se preferă variantele asincrone: `TcpListener.AcceptTcpClientAsync`, `NetworkStream.ReadAsync` și `NetworkStream.WriteAsync`, apelate cu `await` dintr-o metodă `async`. Această abordare permite ca thread-ul UI să rămână receptiv la interacțiunea utilizatorului și ca un singur thread să gestioneze concurrent mai multe conexiuni, ceea ce este esențial pentru scalabilitatea serverelor.

9.6.2. UDP

În rețelele de calculatoare, *User Datagram Protocol* (UDP) este unul dintre protocoalele de comunicație de bază ale suitei de protocoale Internet, utilizat pentru a trimite mesaje (transportate sub formă de datagrame în pachete) către alte gazde dintr-o rețea bazată pe Internet Protocol (IP). Într-o rețea IP, UDP nu necesită o comunicare prealabilă pentru a stabili canale de comunicare sau căi de transmisie a datelor. [12]

UDP este potrivit pentru scopuri în care verificarea și corectarea erorilor nu sunt necesare sau sunt realizate la nivelul aplicației; UDP evită costurile suplimentare generate de astfel de procesări în stiva de protocoale. Aplicațiile sensibile la timp folosesc adesea UDP, deoarece pierderea pachetelor este preferabilă așteptării pachetelor întârziate din cauza retransmiterii, ceea ce poate să nu fie o opțiune într-un sistem în timp real. [12]

Exemplu:

```
UdpClient udpClient;  
UdpClient senderClient = new UdpClient();  
Thread listenThread;  
  
public Form1()  
{  
    InitializeComponent();  
}  
  
private void btnTest_Click(object sender, EventArgs e)  
{  
    // Receiver  
    UdpClient receiver = new UdpClient(5000);  
  
    // Sender  
    UdpClient senderClient = new UdpClient();  
  
    string mesaj = "Hello UDP!";  
    byte[] data = Encoding.UTF8.GetBytes(mesaj);  
  
    // Trimitere catre localhost  
    senderClient.Send(data, data.Length, "127.0.0.1", 5000);  
  
    // Primire mesaj  
    IPEndPoint ep = new IPEndPoint(IPAddress.Any, 0);  
    byte[] receivedData = receiver.Receive(ref ep);  
  
    string primit = Encoding.UTF8.GetString(receivedData);  
  
    txtMessages.AppendText("Primit: " + primit + "\r\n");  
  
    receiver.Close();  
    senderClient.Close();  
}
```

9.7. Protocoale de nivel aplicație (L7)

9.7.1. HTTP / HTTPS

9.7.1.1. HTTP

HTTP (*Hypertext Transfer Protocol*) este un protocol de nivel aplicație din suita de protocoale Internet, utilizat pentru sisteme de informații distribuite. HTTP reprezintă fundamentul comunicării de date pentru World Wide Web, unde documentele hipertext conțin hyperlink-uri către alte resurse pe care utilizatorul le poate accesa cu ușurință. [13]

HTTP este un protocol de tip cerere–răspuns în modelul client–server. O tranzacție începe atunci când un client trimite o cerere către server, iar serverul încearcă să o satisfacă și returnează un răspuns către client, care descrie rezultatul cererii și, opțional, conține o resursă solicitată, cum ar fi un document HTML sau alt tip de conținut. [13]

Într-un scenariu obișnuit, un browser web acționează ca un client, iar un server web, care găzduiește unul sau mai multe site-uri, joacă rolul de server. Un browser web este un exemplu de agent utilizator (*User Agent* – UA). Alte tipuri de agenți de utilizator includ software-ul de indexare utilizat de motoarele de căutare (web crawlers), browser-ele vocale, aplicațiile mobile și alte programe care accesează, procesează sau afișează conținut web. [13]

9.7.1.2. HTTPS

Hypertext Transfer Protocol Secure (HTTPS) este o extensie a *Hypertext Transfer Protocol* (HTTP). Acesta utilizează criptarea pentru a asigura o comunicare securizată într-o rețea de calculatoare și este utilizat pe scară largă pe Internet. În HTTPS, protocolul de comunicare este criptat folosind *Transport Layer Security* (TLS) sau, în trecut, *Secure Sockets Layer* (SSL). Prin urmare, protocolul mai este denumit și HTTP peste TLS sau HTTP peste SSL. [14]

Motivele principale pentru HTTPS sunt autentificarea site-ului accesat și protejarea confidențialității și integrității datelor schimbate în timpul transmiterii. Acesta protejează împotriva atacurilor de tip „man-in-the-middle”, iar criptarea simetrică a comunicațiilor dintre client și server protejează comunicarea împotriva interceptării și modificării neautorizate. [14]

HTTPS creează un canal securizat peste o rețea nesigură. Acesta asigură o protecție rezonabilă împotriva interceptării comunicațiilor și a atacurilor de tip „man-in-the-middle”, cu condiția utilizării metodelor de criptare adecvate și ca certificatul serverului să fie verificat și de încredere. [14]

Înainte de a trece la programarea efectivă a unui client HTTP în C#, este utilă o scurtă recapitulare a două elemente fundamentale ale protocolului HTTP: codurile de stare (returnate de server pentru a indica rezultatul cererii) și metodele HTTP (alese de client pentru a indica acțiunea dorită).

Codurile de stare HTTP sunt grupate în cinci clase:

Tabel 14. Clasele de coduri de stare HTTP

Clasă	Semnificație	Exemple uzuale
1xx	Informațional — cererea a fost primită, procesarea continuă.	100 Continue, 101 Switching Protocols
2xx	Succes — cererea a fost primită, înțeleasă și acceptată.	200 OK, 201 Created, 204 No Content
3xx	Redirecționare — clientul trebuie să întreprindă o acțiune suplimentară.	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Eroare client — cererea conține o greșeală sau nu poate fi îndeplinită.	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Eroare server — serverul a eșuat în îndeplinirea unei cereri valide.	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Metodele HTTP definesc tipul acțiunii pe care clientul o solicită serverului. Cele mai des utilizate sunt:

Tabel 15. Metodele HTTP cele mai utilizate

Metodă	Semnificație	Idempotentă
GET	Solicită o reprezentare a resursei. Nu trebuie să modifice starea serverului.	Da
POST	Trimite date către server pentru a crea o resursă nouă sau a declanșa o procesare.	Nu
PUT	Înlocuiește complet resursa de la URL-ul indicat cu datele trimise.	Da
PATCH	Aplică o modificare parțială asupra resursei existente.	Nu
DELETE	Șterge resursa de la URL-ul indicat.	Da
HEAD	Identică cu GET, dar serverul returnează doar antetele, fără corpul răspunsului.	Da

O metodă este *idempotentă* dacă efectul a N apelări identice este același cu efectul unei singure apelări. Această proprietate este importantă pentru implementarea retransmisiilor

automate în caz de eroare de rețea: o cerere idempotentă poate fi retransmisă în siguranță, în timp ce o cerere POST repetată poate duce la crearea mai multor resurse identice.

În clasa `HttpClient`, codul de stare al unui răspuns este accesibil prin proprietatea `response.StatusCode` (de tip `HttpStatusCode`), iar metoda `response.IsSuccessStatusCode` returnează `true` dacă codul este în intervalul 200–299. Apelul `response.EnsureSuccessStatusCode()` aruncă o excepție `HttpRequestException` dacă cererea a eșuat.

9.7.2. Programare HTTP în C# – clasa `HttpClient`

Clasa `HttpClient` oferă un API modern pentru operațiuni de client HTTP, înlocuind vechile tipuri `WebClient` și `WebRequest/WebResponse` (care între timp au fost marcate ca depășite). [1]

`HttpClient` a fost creat ca răspuns la dezvoltarea rapidă a API-urilor web bazate pe HTTP și a serviciilor REST, oferind o experiență mai bună atunci când se lucrează cu protocoale mai complexe decât simpla descărcare a unei pagini web. În special: [1]

- O singură instanță de `HttpClient` poate gestiona cereri concurente și funcționează bine împreună cu funcționalități precum antete personalizate, cookie-uri și scheme de autentificare;
- `HttpClient` permite scrierea și integrarea de „handler-e” (gestionari) de mesaje personalizate. Acest lucru permite simularea (mocking) în testele unitare și crearea de fluxuri personalizate de procesare (pentru jurnalizare, compresie, criptare etc.);
- `HttpClient` are un sistem de tipuri bogat și extensibil pentru antete și conținut.

Cel mai simplu mod de a utiliza `HttpClient` este instanțierea și apoi apelarea uneia dintre metodele sale `Get*`, transmițând un URI: [1]

```
string html =  
    await new HttpClient().GetStringAsync("http://linqpad.net");
```

Există și metodele `GetByteArrayAsync` și `GetStreamAsync`. Toate metodele din `HttpClient` care implică operații de intrare/ieșire sunt asincrone. [1]

Spre deosebire de predecesorii săi `WebRequest/WebResponse`, pentru a obține performanțe optime cu `HttpClient` este necesară reutilizarea aceeași instanțe (altfel,

lucruri precum rezoluția DNS pot fi repetate inutil, iar socket-urile pot rămâne deschise mai mult decât este necesar). [1]

`HttpClient` are o proprietate `Timeout` și o proprietate `BaseAddress`, care adaugă un prefix URI-ului pentru fiecare cerere. `HttpClient` este, într-o oarecare măsură, un „wrapper” zvelt: majoritatea celorlalte proprietăți pe care te-ai aștepta să le găsești aici sunt definite într-o altă clasă, numită `HttpClientHandler`. Pentru a accesa această clasă, trebuie instanțiată și apoi transmisă în constructorul lui `HttpClient`. [1]

9.7.2.1. `GetAsync`

Metodele `GetStringAsync`, `GetByteArrayAsync` și `GetStreamAsync` sunt scurtături convenabile pentru apelarea metodei mai generale `GetAsync`, care returnează un mesaj de răspuns: [1]

```
var client = new HttpClient();
HttpResponseMessage response =
    await client.GetAsync ("http://...");
response.EnsureSuccessStatusCode();
string html = await response.Content.ReadAsStringAsync();
```

`HttpResponseMessage` expune proprietăți pentru accesarea antetelor și a codului de stare HTTP. Un cod de stare nereușit, cum ar fi 404 (not found), nu determină aruncarea unei excepții decât la apelarea `EnsureSuccessStatusCode`. Totuși, erorile de comunicație sau DNS generează excepții. [1]

`HttpContent` are o metodă `CopyToAsync` pentru scrierea către un alt flux (stream), care este utilă pentru a scrie rezultatul într-un fișier: [1]

```
using (var fileStream = File.Create("linqpad.html"))
    await response.Content.CopyToAsync(fileStream);
```

Exemplu:

```
HttpClient client = new HttpClient();

public Form1()
{
    InitializeComponent();
}

private async void btnGet_Click(object sender, EventArgs e)
{
    string url = "https://...";
```

```
HttpResponseMessage response = await client.GetAsync(url);

if (response.IsSuccessStatusCode)
{
    string data = await response.Content.ReadAsStringAsync();
    txtRes.Text = data;
}
else
{
    txtRes.Text = "Eroare: " + response.StatusCode;
}
}
```

9.7.2.2. SendAsync

GetAsync, PostAsync, PutAsync și DeleteAsync sunt toate scurtături pentru apelarea metodei SendAsync, metoda unică de nivel inferior în care sunt direcționate toate celelalte. Pentru a o utiliza, se construiește mai întâi un HttpRequestMessage: [1]

```
var client = new HttpClient();
var request = new HttpRequestMessage (HttpMethod.Get, "http://...");
HttpResponseMessage response = await client.SendAsync (request);
response.EnsureSuccessStatusCode();
```

Instanțierea unui obiect HttpRequestMessage înseamnă că pot fi personalizate proprietățile cererii, cum ar fi antetele și conținutul propriu-zis, permițând astfel încărcarea de date.

Exemplu:

```
HttpClient client = new HttpClient();

public Form1()
{
    InitializeComponent();
}

private async void btnSend_Click(object sender, EventArgs e)
{
    try
    {
        // GET - preluare date
        var getRequest = new HttpRequestMessage(
            HttpMethod.Get,
            "https://jsonplaceholder.typicode.com/posts/1"
        );

        HttpResponseMessage getResponse = await client.SendAsync(getRequest);

        string originalData = await getResponse.Content.ReadAsStringAsync();

        txtResSend.Text = "ORIGINAL:\r\n" + originalData + "\r\n\r\n";
    }
}
```

```
// PUT – trimitere date modificate
var json =
    "{ \"id\": 1, \"title\": \"MODIFICAT\", \"body\": \"Hello\", \"userId\": 1 }";

var putRequest = new HttpRequestMessage(
    HttpMethod.Put,
    "https://jsonplaceholder.typicode.com/posts/1"
);

putRequest.Content = new StringContent(
    json,
    Encoding.UTF8,
    "application/json"
);

HttpResponseMessage putResponse = await client.SendAsync(putRequest);

string updatedData = await putResponse.Content.ReadAsStringAsync();

txtResSend.AppendText("UPDATED:\r\n" + updatedData);
}
catch (Exception ex)
{
    txtResSend.Text = "Eroare: " + ex.Message;
}
}
```

9.7.2.3. Deserializare JSON cu System.Text.Json

Exemplele anterioare cu `HttpClient` afișează răspunsul brut al serverului, sub formă de șir de caractere. În practică, majoritatea serviciilor web REST returnează date în format JSON, iar aplicația are nevoie să le transforme în obiecte C# pentru a le putea folosi. Începând cu .NET Core 3.0, biblioteca standard include spațiul `System.Text.Json`, care oferă o metodă rapidă și fără dependențe externe pentru serializare și deserializare JSON.

Pentru a deserializa un răspuns JSON, se definește mai întâi o clasă C# ale cărei proprietăți corespund câmpurilor obiectului JSON. Numele proprietăților trebuie să fie identice (sau mapate explicit prin atributul `[JsonPropertyName]`). Apoi se apelează metoda statică `JsonSerializer.Deserialize<T>`, transmițând șirul JSON.

Exemplu:

```
using System.Text.Json;

// Clasa care reflectă structura răspunsului JSON
public class Post
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public string Title { get; set; }
    public string Body { get; set; }
}
```

```
private async void btnLoad_Click(object sender, EventArgs e)
{
    try
    {
        string url = "https://jsonplaceholder.typicode.com/posts/1";
        HttpClient client = new HttpClient();
        string json = await client.GetStringAsync(url);

        // Deserializare în obiect tipizat
        var options = new JsonSerializerOptions
        {
            PropertyNameCaseInsensitive = true
        };
        Post post = JsonSerializer.Deserialize<Post>(json, options);

        // Afișare în controale Label
        lblId.Text = "ID: " + post.Id;
        lblTitle.Text = "Titlu: " + post.Title;
        lblBody.Text = "Text: " + post.Body;
    }
    catch (Exception ex)
    {
        MessageBox.Show("Eroare: " + ex.Message);
    }
}
```

Opțiunea `PropertyNameCaseInsensitive = true` permite ca proprietățile C# scrise cu majusculă inițială (`Id`, `Title`) să fie potrivite cu câmpuri JSON scrise cu literă mică (`id`, `title`) – convenția uzuală pentru API-urile REST. Pentru cazuri mai complexe (de exemplu, nume de câmpuri JSON cu sublinieri sau cratime), se folosește atributul `[JsonPropertyName("nume_camp")]` aplicat fiecărei proprietăți.

9.8. Exerciții

Pentru fiecare exercițiu, se va construi o aplicație Windows Forms care ilustrează utilizarea protocolului corespunzător.

9.8.1. Downloader paralel de imagini cu `HttpClient`.

Realizați o aplicație care primește o listă de URL-uri (câte unul pe linie, într-un `TextBox` multiline), descarcă în paralel toate imaginile folosind o singură instanță de `HttpClient` și metodele asincrone, apoi le salvează într-un folder ales de utilizator. Afișați pentru fiecare descărcare starea (în curs, reușită, eșuată) într-un `DataGridView`.

Indicații:

Citiți URL-urile din `TextBox` folosind `Lines`, eliminați liniile goale și construiți o listă. Pentru fiecare URL adăugați un rând în `DataGridView`, setând inițial starea „În curs” și reținând indexul rândului.

Creați o singură instanță de `HttpClient` (de preferat membru static) și refolosiți-o pentru toate descărcările.

Implementați o metodă asincronă `DownloadImageAsync(string url, int rowIndex, string folder)` în care faceți descărcarea efectivă folosind `await httpClient.GetByteArrayAsync(url)` sau `GetAsync` + citire din stream. Salvați fișierul cu `File.WriteAllBytesAsync`.

În handlerul butonului „Descarcă”, NU folosiți un `for` secvențial cu `await` în interior. În schimb, creați o listă de task-uri: pentru fiecare URL apelați `DownloadImageAsync(...)` fără `await` imediat și adăugați task-ul într-o colecție. La final folosiți `await Task.WhenAll(tasks)` pentru a aștepta finalizarea tuturor descărcărilor.

În metoda asincronă folosiți `try/catch` pentru a trata erorile individuale. În `try` setați starea „În curs”, la final „Reușită”, iar în `catch` „Eșuată”. Fiecare descărcare trebuie să-și actualizeze propriul rând din `DataGridView`.

Aveți grijă la actualizarea UI: dacă apare eroare de tip cross-thread, folosiți `Invoke` sau `BeginInvoke` pentru a modifica `DataGridView`.

Generați nume de fișiere unice (de exemplu folosind indexul sau `Guid`). Opțional, determinați extensia din URL sau din `Content-Type`.

Ideea esențială: **paralelismul se obține prin lansarea mai multor task-uri asincrone și sincronizarea lor cu `Task.WhenAll`, nu prin bucle secvențiale.**

9.8.2. Client REST pentru un API public de vreme

Realizați o aplicație care preia datele meteorologice curente pentru un oraș introdus de utilizator, folosind un API public REST (de exemplu, open-meteo.com). Răspunsul JSON va fi deserializat într-o clasă proprie, iar valorile relevante (temperatură, umiditate, viteza vântului) vor fi afișate în controale `Label`.

LABORATORUL 8

10. Programare orientată pe obiecte în C#

Programarea orientată pe obiecte (POO) este o paradigmă de programare care organizează codul în jurul unor entități numite obiecte, fiecare obiect fiind o instanță a unei clase. O clasă definește structura (datele) și comportamentul (operațiile) obiectelor, iar obiectele sunt instanțele concrete care lucrează cu date reale la execuție.

POO se bazează pe patru principii fundamentale:

- **Abstractizare** – concentrarea pe aspectele esențiale ale unei entități, ignorând detaliile irelevante pentru contextul curent;
- **Încapsulare** – gruparea datelor și a operațiilor asupra lor în aceeași unitate logică și protejarea datelor față de acces necontrolat din exterior;
- **Moștenire** – reutilizarea și extinderea funcționalității existente prin derivarea unor clase noi din clase existente;
- **Polimorfism** – capacitatea unei singure interfețe de a se comporta diferit în funcție de tipul efectiv al obiectului apelant.

În C#, POO este suportată nativ, iar toate tipurile de date – inclusiv tipurile primitive precum `int`, `double` sau `bool` – sunt, în esență, clase sau structuri definite în biblioteca .NET.

10.1. Clase și obiecte

O **clasă** este un tip definit de utilizator care descrie structura și comportamentul unui set de obiecte. O clasă poate fi privită ca un șablon după care se creează obiectele. Un **obiect** este o instanță concretă a unei clase, alocată în memorie și caracterizată de valorile specifice ale câmpurilor sale.

10.1.1. Declararea unei clase

Declararea unei clase se realizează utilizând cuvântul-cheie `class`, conform sintaxei următoare:

Format general:

```
[<modificator_acces>] class <nume_clasa>
{
    // câmpuri, proprietăți, metode, constructori
}
```

Exemplu:

```
public class Student
{
    public string Nume;
    public int Varsta;
}
```

10.1.2. Instanțierea obiectelor

Crearea unui obiect – proces numit *instanțiere* – se realizează cu operatorul *new*. La instanțiere, runtime-ul .NET alocă memorie pentru obiect, apelează constructorul și returnează o referință către obiectul nou creat.

Format general:

```
<nume_clasa> <nume_obiect> = new <nume_clasa>([<argumente>]);
```

Exemplu:

```
Student s1 = new Student();
s1.Nume = "Popescu Ioan";
s1.Varsta = 20;
```

Observații:

- Clasele sunt tipuri referință: variabila reține o referință către zona de memorie în care se află obiectul, nu obiectul în sine;
- Mai multe variabile pot referi același obiect, modificările făcute prin una dintre ele fiind vizibile și prin celelalte;
- Dacă o referință nu indică niciun obiect, valoarea ei este `null`;
- Obiectele nefolosite sunt eliberate automat de garbage collector-ul .NET.

10.1.3.Câmpuri

Câmpurile (*fields*) sunt variabile declarate la nivelul clasei și reprezintă datele pe care le rețin obiectele. Câmpurile pot fi de instanță (valoare specifică fiecărui obiect) sau *static* (valoare comună tuturor obiectelor clasei).

Format general:

```
[<modificator_acces>] [static] <tip> <nume_camp> [= <valoare_implicita>];
```

Exemplu:

```
public class Produs
{
    public string Denumire;
    public decimal Pret = 0m;
    public static int NumarTotalProduce = 0;
}
```

10.1.4.Proprietăți

Proprietățile (*properties*) sunt membri speciali ai unei clase care oferă un mecanism controlat de acces la câmpurile interne, prin intermediul unor accesorii *get* (citire) și mutatori *set* (scriere). O proprietate poate fi doar-citire (se omite *set*), doar-scriere (se omite *get*) sau poate conține validări și transformări interne.

Format general:

```
[<modificator_acces>] <tip> <nume_proprietate>
{
    get { ... }
    set { ... }
}
```

Exemplu:

```
public class Produs
{
    private decimal _pret;
    public decimal Pret
    {
        get { return _pret; }
        set
```

```
        {  
            if (value < 0)  
                throw new ArgumentException("Pretul nu poate fi negativ.");  
            _pret = value;  
        }  
    }  
}
```

Începând cu C# 3.0, compilatorul poate genera automat câmpul privat asociat unei proprietăți simple. Acestea se numesc *proprietăți auto-implementate*:

Exemplu:

```
public class Student  
{  
    public string Nume { get; set; }  
    public int Varsta { get; set; }  
    public DateTime DataInmatricularii { get; } = DateTime.Now;  
}
```

10.1.5. Metode

Metodele reprezintă operațiile pe care le pot efectua obiectele unei clase. O metodă este caracterizată prin nume, tip de returnare, listă de parametri și corp.

Format general:

```
[<modificator_acces>] [static] <tip_returnare> <nume_metoda>([<parametri>])  
{  
    // instrucțiuni  
    return <valoare>;  
}
```

Exemplu:

```
public class Cerc  
{  
    public double Raza { get; set; }  
  
    public double CalculeazaArie()  
    {  
        return Math.PI * Raza * Raza;  
    }  
}
```

```
public double CalculeazaPerimetru()  
{  
    return 2 * Math.PI * Raza;  
}  
}
```

10.1.6.Constructori

Un **constructor** este o metodă specială apelată automat la instanțierea unui obiect. Are același nume cu cel al clasei și **nu are tip de returnare**. Constructorii servesc la inițializarea câmpurilor și la aducerea obiectului într-o stare validă imediat după creare.

Format general:

```
[<modificator_acces>] <nume_clasa>([<parametri>])  
{  
    // inițializări  
}
```

Exemplu:

```
public class Student  
{  
    public string Nume { get; set; }  
    public int Varsta { get; set; }  
  
    public Student()  
    {  
        Nume = "Necunoscut";  
        Varsta = 18;  
    }  
  
    public Student(string nume, int varsta)  
    {  
        Nume = nume;  
        Varsta = varsta;  
    }  
}
```

Observații:

- Dacă nu se definește niciun constructor, compilatorul generează automat un constructor implicit (fără parametri) care inițializează câmpurile cu valorile implicite ale tipurilor lor;

- Într-o clasă pot exista mai mulți constructori, diferențiați prin lista de parametri (supraîncărcare);
- Un constructor poate apela un alt constructor al aceleiași clase folosind `this(...)`;
- Un constructor static se execută o singură dată, înaintea primei utilizări a clasei.

10.1.7. Modificatori de acces

Modificatorii de acces controlează vizibilitatea membrilor unei clase. Principalii modificatori în C# sunt prezentați în tabelul de mai jos.

Modificator	Vizibilitate
public	Acces nerestricționat, din orice clasă și orice assembly.
private	Acces doar din interiorul clasei în care este declarat.
protected	Acces din interiorul clasei și din clasele derivate.
internal	Acces doar din interiorul aceleiași assembly (proiect).
protected internal	Acces din același assembly sau din clasele derivate.
private protected	Acces din clasele derivate doar dacă sunt în același assembly.

În absența unui modificator explicit, membrii unei clase sunt implicit *private*, iar clasele sunt implicit *internal*.

10.2. Încapsulare

Încapsularea este principiul care presupune **ascunderea detaliilor interne de implementare** și expunerea doar a unei interfețe publice controlate. În C#, încapsularea se realizează prin declararea câmpurilor ca `private` și expunerea lor prin proprietăți publice sau prin metode care validează și controlează operațiile.

Exemplu:

```
public class ContBancar
{
    private decimal _sold;

    public string NumeTitular { get; }
    public decimal Sold => _sold;

    public ContBancar(string numeTitular, decimal soldInitial)
    {
        NumeTitular = numeTitular;
        _sold = soldInitial;
    }
}
```

```
    }

    public void Depune(decimal suma)
    {
        if (suma <= 0)
            throw new ArgumentException("Suma depusa trebuie sa fie pozitiva.");
        _sold += suma;
    }

    public void Retrage(decimal suma)
    {
        if (suma <= 0)
            throw new ArgumentException("Suma retrasa trebuie sa fie pozitiva.");
        if (suma > _sold)
            throw new InvalidOperationException("Sold insuficient.");
        _sold -= suma;
    }
}
```

Prin încapsulare, clientul clasei `ContBancar` nu poate modifica direct câmpul `_sold`; singurele căi de modificare sunt metodele `Depune` și `Retrage`, care impun respectarea regulilor de business.

10.3. Moștenire

Moștenirea permite definirea unei noi clase (clasa **derivată**) pe baza unei clase existente (clasa **de bază**). Clasa derivată preia toți membrii `public` și `protected` ai clasei de bază și îi poate extinde cu membri noi sau le poate redefini comportamentul.

În C#, o clasă poate moșteni de la o singură clasă de bază (moștenire simplă), însă poate implementa oricâte interfețe.

Format general:

```
class <nume_clasa_derivata> : <nume_clasa_baza>
{
    // membri noi sau redefiniți
}
```

Exemplu:

```
public class Persoana
{
    public string Nume { get; set; }
    public string Prenume { get; set; }
```

```
        public string NumeComplet() => $"{Nume} {Prenume}";  
    }  
  
    public class Angajat : Persoana  
    {  
        public string Departament { get; set; }  
        public decimal Salariu { get; set; }  
    }
```

10.3.1. Cuvântul cheie *base*

Cuvântul cheie **base** este folosit în clasa derivată pentru a accesa membrii clasei de bază – îndeosebi pentru a apela constructorul clasei părinte sau metode suprascrise.

Exemplu:

```
public class Persoana  
{  
    public string Nume { get; set; }  
    public string Prenume { get; set; }  
  
    public Persoana(string nume, string prenume)  
    {  
        Nume = nume;  
        Prenume = prenume;  
    }  
}  
  
public class Angajat : Persoana  
{  
    public string Departament { get; set; }  
    public decimal Salariu { get; set; }  
  
    public Angajat(string nume, string prenume,  
                    string departament, decimal salariu)  
        : base(nume, prenume)  
    {  
        Departament = departament;  
        Salariu = salariu;  
    }  
}
```

Apelul: `base(nume, prenume)` din semnătura constructorului clasei `Angajat` asigură executarea constructorului din `Persoana` înaintea corpului constructorului derivat.

10.4. Polimorfism

Polimorfismul permite ca obiecte de tipuri diferite să răspundă în mod specific la același apel de metodă. În C#, polimorfismul se manifestă sub două forme principale:

- **Supraîncărcarea metodelor (*overloading*)** – metode cu același nume, dar cu semnături diferite în aceeași clasă;
- **Suprascrierea metodelor (*overriding*)** – redefinirea într-o clasă derivată a unei metode din clasa de bază marcată drept *virtual* sau *abstract*.

10.4.1. Metode virtuale și override

O metodă declarată *virtual* în clasa de bază poate fi redefinită în clasele derivate prin utilizarea cuvântului cheie *override*.

Exemplu:

```
public class Animal
{
    public string Nume { get; set; }

    public virtual string ScoateSunet()
    {
        return "Sunet generic";
    }
}

public class Caine : Animal
{
    public override string ScoateSunet()
    {
        return "Ham! Ham!";
    }
}

public class Pisica : Animal
{
    public override string ScoateSunet()
    {
        return "Miau!";
    }
}

// utilizare polimorfică
Animal a1 = new Caine { Nume = "Rex" };
```

```
Animal a2 = new Pisica { Nume = "Whiskers" };  
MessageBox.Show(a1.ScoateSunet()); // "Ham! Ham!"  
MessageBox.Show(a2.ScoateSunet()); // "Miau!"
```

Deși variabilele `a1` și `a2` sunt declarate de tipul `Animal`, la execuție se apelează versiunea metodei corespunzătoare tipului efectiv al obiectului. Acest mecanism se numește **legare dinamică** (*dynamic dispatch*).

10.4.2. Conversii între tipurile din ierarhie

În ierarhiile de clase, o referință de tip derivat poate fi atribuită direct unei variabile de tip bază (*upcasting*). Pentru operația inversă (*downcasting*), se folosește operatorul `as` (returnează `null` dacă conversia nu este posibilă) sau `is` (verifică compatibilitatea de tip).

Exemplu:

```
Animal a = new Caine { Nume = "Rex" };  
  
Caine c = a as Caine;      // downcasting cu as  
if (c != null) { /* ... */ }  
  
if (a is Pisica p)         // verificare + conversie  
{  
    // p este accesibil de tipul Pisica  
}
```

10.5. Clase abstracte

O **clasă abstractă** este o clasă care nu poate fi instanțiată direct și care conține, de regulă, cel puțin o metodă abstractă (fără implementare). Clasele abstracte sunt folosite pentru a defini un cadru comun pentru clasele derivate, lăsând detaliile de implementare pentru acestea.

Format general:

```
abstract class <nume_clasa>  
{  
    abstract <tip_returnare> <nume_metoda>([<parametri>]); // fără implementare  
    // metode concrete opționale  
}
```

Exemplu:

```
public abstract class Figura
{
    public string Nume { get; set; }

    public abstract double CalculeazaArie();
    public abstract double CalculeazaPerimetru();

    public virtual string Descriere()
    {
        return $"{Nume}: arie = {CalculeazaArie():F2}, " +
            $"perimetru = {CalculeazaPerimetru():F2}";
    }
}

public class Cerc : Figura
{
    public double Raza { get; set; }

    public override double CalculeazaArie()
        => Math.PI * Raza * Raza;

    public override double CalculeazaPerimetru()
        => 2 * Math.PI * Raza;
}
```

Observații:

- O clasă abstractă nu poate fi instanțiată: `Figura f = new Figura();` produce eroare de compilare;
- O clasă derivată dintr-o clasă abstractă este obligată să implementeze toate metodele abstracte, altfel trebuie declarată și ea abstractă;
- O metodă abstractă este implicit virtuală și poate fi declarată doar într-o clasă abstractă.

10.6. Interfețe

O **interfață** este un contract care specifică un set de membri (metode, proprietăți, evenimente) pe care clasele implementatoare trebuie să le furnizeze. Spre deosebire de clasele abstracte, o interfață clasică nu conține implementări și nici câmpuri de stare.

În C#, o clasă poate implementa oricâte interfețe, ceea ce oferă o formă de **moștenire multiplă** pentru comportament. Prin convenție, numele interfețelor începe cu litera I.

Format general:

```
interface <nume_interfata>
{
    <tip_returnare> <nume_metoda>([<parametri>]);
    <tip> <nume_proprietate> { get; set; }
}
```

Exemplu:

```
public interface IDesenabil
{
    void Deseneaza(Graphics g, Point pozitie);
}

public interface ISerializabil
{
    string ToText();
}

public class Cerc : Figura, IDesenabil, ISerializabil
{
    public double Raza { get; set; }

    public override double CalculeazaArie()
        => Math.PI * Raza * Raza;

    public override double CalculeazaPerimetru()
        => 2 * Math.PI * Raza;

    public void Deseneaza(Graphics g, Point pozitie)
    {
        using (Pen pen = new Pen(Color.Blue, 2))
        {
            g.DrawEllipse(pen, pozitie.X, pozitie.Y,
                          (float)(2 * Raza), (float)(2 * Raza));
        }
    }

    public string ToText() => $"Cerc;{Nume};{Raza}";
}
```

Observații:

- Membrii declarați într-o interfață sunt implicit public și abstract;
- O clasă care implementează o interfață trebuie să furnizeze toate metodele declarate în aceasta;
- O interfață poate extinde alte interfețe: `interface IB : IA { ... };`
- Spre deosebire de clasele abstracte, o interfață nu poate conține câmpuri (doar proprietăți cu get/set).

10.7. Aplicație – Gestionar de figuri geometrice

Se realizează o aplicație Windows Forms care permite utilizatorului să creeze, să afișeze și să deseneze figuri geometrice de tipuri diferite (cerc, dreptunghi, triunghi). Aplicația ilustrează utilizarea claselor abstracte, a moștenirii, a polimorfismului și a interfețelor.

10.7.1. Structura proiectului

Soluția conține un singur proiect Windows Forms numit `GestionarFiguri`, organizat în următoarele fișiere:

- `Models/IDesenabil.cs` – interfața pentru desenare;
- `Models/Figura.cs` – clasa abstractă de bază;
- `Models/Cerc.cs` – derivată din `Figura`;
- `Models/Dreptunghi.cs` – derivată din `Figura`;
- `Models/Triunghi.cs` – derivată din `Figura`;
- `Form1.cs` – formularul principal al aplicației.

10.7.2. Design grafic – Form1

Formularul conține: un `ComboBox` pentru selecția tipului figurii, un `TextBox` pentru nume și trei `TextBox`-uri pentru dimensiuni, trei butoane de acțiune, un `ListBox` pentru lista figurilor create, un `Panel` pe care se desenează figurile și un `Label` pentru afișarea totalurilor (Figura 21).

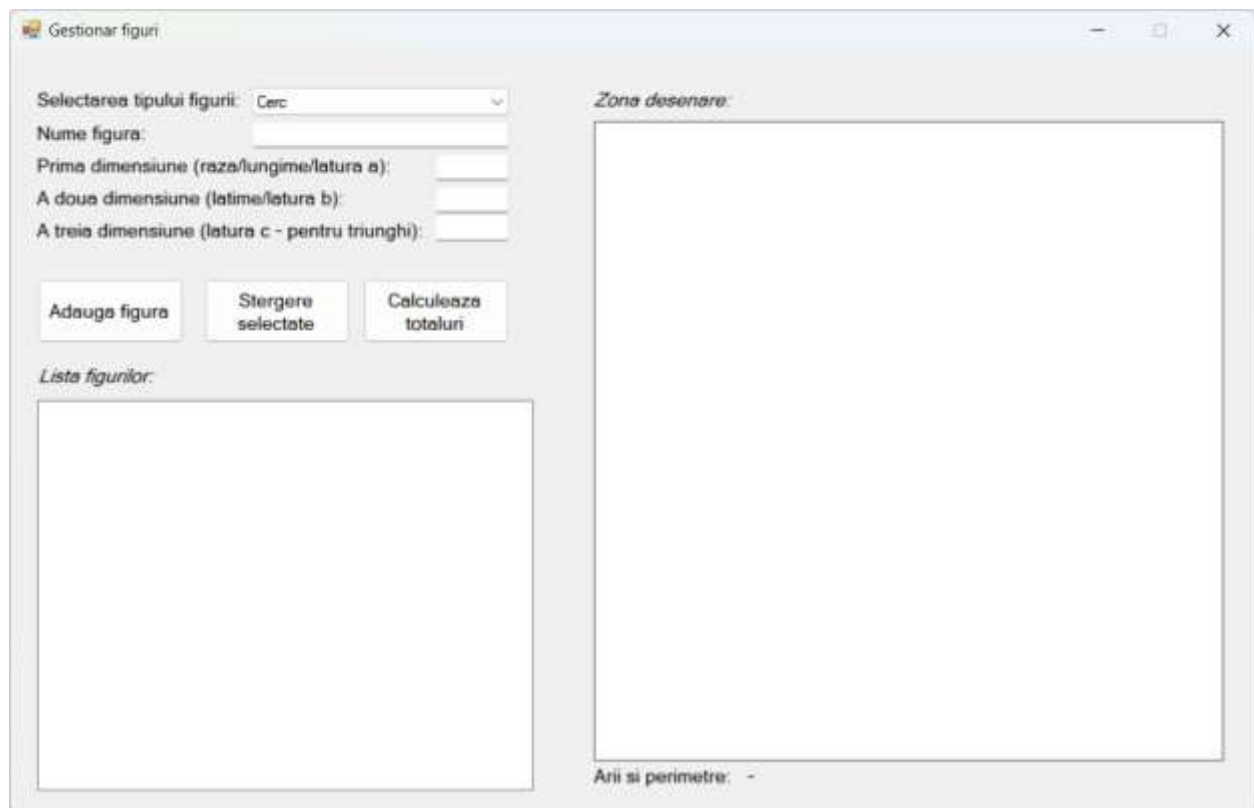


Figura 21 - Interfața grafică pentru aplicația Gestionar figuri

10.7.3. Proprietăți și evenimente pentru controale

Tip control	Nume (Name)	Setări relevante	Rol
ComboBox	cbTipFigura	Items = Cerc, Dreptunghi, Triunghi; DropDownStyle = DropDownList	Selectarea tipului figurii
TextBox	tbNume	-	Numele figurii
TextBox	tbDim1	-	Prima dimensiune (raza / lungime / latura a)
TextBox	tbDim2	-	A doua dimensiune (lățime / latura b)
TextBox	tbDim3	-	A treia dimensiune (latura c – pentru triunghi)
Button	btnAdauga	Text = "Adaugă figură"; Click → btnAdauga_Click	Adaugă figura în colecție
Button	btnSterge	Text = "Șterge selectate"; Click → btnSterge_Click	Șterge figurile selectate
Button	btnTotaluri	Text = "Calculează totaluri"; Click → btnTotaluri_Click	Afișează suma ariilor și perimetrelor
ListBox	lbxFiguri	SelectionMode = MultiExtended	Afișează lista figurilor

Tip control	Nume (Name)	Setări relevante	Rol
Panel	pnlDesen	BackColor = White; BorderStyle = FixedSingle; Paint → pnlDesen_Paint	Zona de desenare
Label	lbTotaluri	Text = "-"	Afișează ariile și perimetrele
Form	Form1	Text = "Gestionar figuri"; Load → Form1_Load	Formularul principal

10.7.4.Cod sursă

Models/IDesenabil.cs

```
using System.Drawing;

namespace GestionarFiguri.Models
{
    public interface IDesenabil
    {
        void Deseneaza(Graphics g, Point pozitie);
    }
}
```

Models/Figura.cs

```
using System.Drawing;

namespace GestionarFiguri.Models
{
    public abstract class Figura : IDesenabil
    {
        public string Nume { get; }

        protected Figura(string nume)
        {
            Nume = string.IsNullOrEmpty(nume) ?
                "Fara nume" : nume.Trim();
        }

        public abstract double CalculeazaArie();
        public abstract double CalculeazaPerimetru();
        public abstract void Deseneaza(Graphics g, Point pozitie);

        public override string ToString()
        => $"{GetType().Name} [{Nume}] " +
            $"A={CalculeazaArie():F2}, " +
            $"P={CalculeazaPerimetru():F2}";
    }
}
```

Models/Cerc.cs

```
using System;
using System.Drawing;

namespace GestionarFiguri.Models
{
    public class Cerc : Figura
    {
        public double Raza { get; }

        public Cerc(string nume, double raza) : base(nume)
        {
            if (raza <= 0)
                throw new ArgumentException("Raza trebuie sa fie pozitiva.");
            Raza = raza;
        }

        public override double CalculeazaArie()
            => Math.PI * Raza * Raza;

        public override double CalculeazaPerimetru()
            => 2 * Math.PI * Raza;

        public override void Deseneaza(Graphics g, Point pozitie)
        {
            using (var pen = new Pen(Color.Blue, 2))
            {
                int d = (int) (2 * Raza);
                g.DrawEllipse(pen, pozitie.X, pozitie.Y, d, d);
            }
        }
    }
}
```

Models/Dreptunghi.cs

```
using System;
using System.Drawing;

namespace GestionarFiguri.Models
{
    public class Dreptunghi : Figura
    {
        public double Lungime { get; }
        public double Latime { get; }

        public Dreptunghi(string nume, double lungime, double latime)
            : base(nume)
        {
            if (lungime <= 0 || latime <= 0)
                throw new ArgumentException(
                    "Dimensiunile trebuie sa fie pozitive.");
            Lungime = lungime;
            Latime = latime;
        }
    }
}
```

```
public override double CalculeazaArie()
    => Lungime * Latime;

public override double CalculeazaPerimetru()
    => 2 * (Lungime + Latime);

public override void Deseneaza(Graphics g, Point pozitie)
{
    using (var pen = new Pen(Color.Red, 2))
    {
        g.DrawRectangle(pen, pozitie.X, pozitie.Y,
            (int)Lungime, (int)Latime);
    }
}
}
```

Models/Triunghi.cs

```
using System;
using System.Drawing;

namespace GestionarFiguri.Models
{
    public class Triunghi : Figura
    {
        public double A { get; }
        public double B { get; }
        public double C { get; }

        public Triunghi(string nume, double a, double b, double c)
            : base(nume)
        {
            if (a <= 0 || b <= 0 || c <= 0)
                throw new ArgumentException(
                    "Laturile trebuie sa fie pozitive.");

            if (a + b <= c || a + c <= b || b + c <= a)
                throw new ArgumentException(
                    "Laturile nu formeaza un triunghi valid.");

            A = a; B = b; C = c;
        }

        public override double CalculeazaArie()
        {
            double s = CalculeazaPerimetru() / 2;
            return Math.Sqrt(s * (s - A) * (s - B) * (s - C));
        }

        public override double CalculeazaPerimetru() => A + B + C;

        public override void Deseneaza(Graphics g, Point pozitie)
        {
            using (var pen = new Pen(Color.Green, 2))
            {
                var p1 = new Point(pozitie.X, pozitie.Y + (int)A);
                var p2 = new Point(pozitie.X + (int)B, pozitie.Y + (int)A);
                var p3 = new Point(pozitie.X + (int)(B / 2), pozitie.Y);
            }
        }
    }
}
```

```
        g.DrawPolygon(pen, new[] { p1, p2, p3 });  
    }  
}  
}
```

Form1.cs

```
using GestionarFiguri.Models;  
using System;  
using System.Collections.Generic;  
using System.Data;  
using System.Drawing;  
using System.Globalization;  
using System.Linq;  
using System.Windows.Forms;  
  
namespace GestionarFiguri  
{  
    public partial class Form1 : Form  
    {  
        private readonly List<Figura> _figuri = new List<Figura>();  
  
        public Form1()  
        {  
            InitializeComponent();  
        }  
  
        private void Form1_Load(object sender, EventArgs e)  
        {  
            cbTipFigura.SelectedIndex = 0;  
            lbTotaluri.Text = "-";  
        }  
  
        private void btnAdauga_Click(object sender, EventArgs e)  
        {  
            try  
            {  
                Figura f = CreeazaFigura();  
                _figuri.Add(f);  
                lbxFiguri.Items.Add(f);  
                pnlDesen.Invalidate();  
                CurataCampuri();  
            }  
            catch (Exception ex)  
            when (ex is ArgumentException || ex is FormatException)  
            {  
                MessageBox.Show(ex.Message, "Date invalide",  
                                MessageBoxButtons.OK, MessageBoxIcon.Warning);  
            }  
        }  
  
        private void btnSterge_Click(object sender, EventArgs e)  
        {  
            var selectate = lbxFiguri.SelectedItems  
                            .Cast<Figura>().ToList();  
            foreach (var fig in selectate)  
            {  
                _figuri.Remove(fig);  
            }  
        }  
    }  
}
```

```
        lbxFiguri.Items.Remove(fig);
    }

    pnlDesen.Invalidate();
}

private void btnTotaluri_Click(object sender, EventArgs e)
{
    if (_figuri.Count == 0)
    {
        lbTotaluri.Text = "Nu exista figuri.";
        return;
    }

    double ariaTotala = _figuri.Sum(f => f.CalculeazaArie());
    double perimetruTotal = _figuri.Sum(f =>
                                                f.CalculeazaPerimetru());

    lbTotaluri.Text =
        $"Arie totala: {ariaTotala:F2}, " +
        $"Perimetru total: {perimetruTotal:F2}";
}

private void pnlDesen_Paint(object sender, PaintEventArgs e)
{
    int x = 10, y = 10, maxH = 0;
    foreach (Figura f in _figuri)
    {
        if (x > pnlDesen.Width - 120)
        {
            x = 10;
            y += maxH + 10;
            maxH = 0;
        }

        // apel polimorfic - fiecare figură își știe desenul
        f.Deseneaza(e.Graphics, new Point(x, y));

        x += 120;
        maxH = Math.Max(maxH, 120);
    }
}

private Figura CreeazaFigura()
{
    string tip = cbTipFigura.SelectedItem?.ToString();
    switch (tip)
    {
        case "Cerc":
            return new Cerc(tbNume.Text,
                ParseDouble(tbDim1.Text, "Raza"));

        case "Dreptunghi":
            return new Dreptunghi(tbNume.Text,
                ParseDouble(tbDim1.Text, "Lungime"),
                ParseDouble(tbDim2.Text, "Latime"));

        case "Triunghi":
            return new Triunghi(tbNume.Text,
                ParseDouble(tbDim1.Text, "Latura a"),
                ParseDouble(tbDim2.Text, "Latura b"),
                ParseDouble(tbDim3.Text, "Latura c"));
    }
}
```

```
        default:
            throw new ArgumentException(
                "Tip de figura necunoscut.");
    }
}

private static double ParseDouble(string text, string nume)
{
    if (string.IsNullOrEmpty(text))
        throw new ArgumentException(
            $"Valoarea {nume} este obligatorie.");

    if (!double.TryParse(text.Replace(',', '.'),
        NumberStyles.Float,
        CultureInfo.InvariantCulture,
        out double val))
        throw new FormatException(
            $"Valoarea {nume} trebuie sa fie numerica.");

    return val;
}

private void CurataCampuri()
{
    tbNume.Clear();
    tbDim1.Clear();
    tbDim2.Clear();
    tbDim3.Clear();
    tbNume.Focus();
}
}
}
```

Notă privind apelurile polimorfe: în metoda `pnlDesen_Paint`, lista `_figuri` este de tip `List<Figura>`, însă la fiecare iterație se apelează `f.Deseneaza(...)`. Runtime-ul alege automat implementarea corespunzătoare tipului efectiv (`Cerc`, `Dreptunghi` sau `Triunghi`), demonstrând mecanismul de legare dinamică. Același principiu se aplică pentru `CalculeazaArie()` și `CalculeazaPerimetru()` în butonul „Calculează totaluri”.

10.8. Exerciții

10.8.1.Exercițiul 1 – Extinderea ierarhiei de figuri

Se cere extinderea aplicației „Gestionar de figuri geometrice” cu următoarele funcționalități:

- adăugarea unei noi clase `Patrat`, derivată fie direct din `Figura`, fie din `Dreptunghi`;
- adăugarea unei noi clase `Elipsa`, cu două semiaxe (`a` și `b`) și formulele de calcul corespunzătoare;

- adăugarea unui buton „Sortare după arie” care reordonează figurile în `ListBox` crescător după aria calculată;
- adăugarea unui buton „Filtrare perimetru” care păstrează în `ListBox` doar figurile al căror perimetru depășește o valoare prag citită dintr-un `TextBox`.

10.8.2.Exercițiul 2 – Interfața *IScalabil*

Se cere:

- definirea unei interfețe `IScalabil` cu o metodă `void Scaleaza(double factor);`
- implementarea interfeței în toate clasele concrete (`Cerc`, `Dreptunghi`, `Triunghi`), astfel încât scalarea să multiplice dimensiunile cu factorul primit (se va ține cont că proprietățile existente sunt doar-citire – se pot folosi câmpuri private sau o redefinire a proprietăților);
- adăugarea în formular a unui `TextBox` pentru factorul de scalare și a unui buton „Scalează selectate” care aplică operația figurilor selectate din `ListBox`;
- după scalare, aria și perimetrul afișate se recalculează și se redesenează figurile pe `Panel`.

LABORATORUL 9

11. Tratarea excepțiilor și depanarea aplicațiilor

O **excepție** este un eveniment anormal care apare în timpul execuției unui program, atunci când o instrucțiune nu poate fi executată corespunzător – de exemplu împărțirea la zero, accesarea unui fișier inexistent, conversia unui șir invalid la număr sau o conexiune de rețea întreruptă. În absența unui mecanism de tratare, excepțiile duc la terminarea bruscă a aplicației și la afișarea unui mesaj de eroare către utilizator.

În contextul unei aplicații Windows Forms, tratarea corectă a excepțiilor este esențială pentru:

- menținerea stabilității aplicației – o eroare apărută într-un handler de buton nu trebuie să închidă formularul;
- afișarea unor mesaje inteligibile utilizatorului, în locul mesajelor tehnice implicite;
- eliberarea resurselor ocupate (fișiere, conexiuni de rețea, conexiuni la baze de date) chiar și atunci când operațiile eșuează;
- jurnalizarea erorilor pentru depanare ulterioară.

Complementar tratării excepțiilor, mediul Visual Studio oferă un *debugger* integrat care permite oprirea execuției la puncte predefinite, rularea pas cu pas și inspectarea valorilor variabilelor – instrumente indispensabile pentru identificarea cauzelor erorilor.

11.1. Tipuri de erori în aplicații

În funcție de momentul în care apar, erorile dintr-o aplicație C# pot fi clasificate în trei categorii:

- **Erori de compilare** (sintactice) – detectate de compilator și semnalate la generarea executabilului (ex. un punct și virgulă lipsă, o variabilă nedeclarată);
- **Erori de execuție** (*runtime errors*) – apar în timpul rulării, atunci când o operație nu poate fi dusă la bun sfârșit (împărțire la zero, referință null, fișier inexistent). Sunt reprezentate în .NET prin excepții;

- **Defecte logice** – aplicația rulează fără erori, dar produce rezultate incorecte. Pentru identificarea lor se folosește preponderent depanatorul.

11.2. Ierarhia claselor de excepții în .NET

Toate excepțiile din .NET derivă din clasa *System.Exception*. Din aceasta derivă două ierarhii principale: *SystemException* (excepții generate de runtime) și *ApplicationException* (excepții definite de aplicații – recomandarea este ca excepțiile personalizate să derive direct din *Exception*).

Cele mai frecvent întâlnite tipuri de excepții sunt prezentate în tabelul de mai jos.

Tipul excepției	Descriere	Exemplu de cauză
<code>FormatException</code>	Conversia unui șir într-un tip primitiv a eșuat.	<code>int.Parse("abc")</code>
<code>OverflowException</code>	Operația aritmetică a depășit domeniul tipului.	<code>checked { int x = int.MaxValue + 1; }</code>
<code>DivideByZeroException</code>	Împărțire la zero pentru tipuri întregi.	<code>int a = 10 / 0;</code>
<code>ArgumentException</code>	Un argument primit nu respectă cerințele.	Argument negativ acolo unde se așteaptă unul pozitiv.
<code>ArgumentNullException</code>	Un argument obligatoriu este null.	<code>new Student(null)</code>
<code>ArgumentOutOfRangeException</code>	Un index este în afara intervalului valid.	<code>lista[10]</code> când lista are 3 elemente.
<code>NullReferenceException</code>	Accesarea unui membru al unei referințe null.	<code>Student s = null; s.Nume;</code>
<code>IndexOutOfRangeException</code>	Index invalid pentru un tablou.	<code>int[] v = new int[3]; v[5];</code>
<code>InvalidOperationException</code>	Operația nu este validă pentru starea curentă.	Retragere dintr-un cont cu sold 0.
<code>FileNotFoundException</code>	Fișierul specificat nu există.	<code>File.ReadAllText("lipsa.txt")</code>
<code>IOException</code>	Eroare la operații de intrare/ieșire.	Fișier blocat de alt proces.
<code>UnauthorizedAccessException</code>	Acces interzis la o resursă.	Scriere într-un fișier read-only.

11.3. Instrucțiunea try-catch

Instrucțiunea *try-catch* permite execuția unui bloc de cod „protejat” (blocul *try*) și capturarea oricărei excepții apărute, într-un bloc *catch* corespunzător tipului excepției.

Format general:

```
try
{
    // instrucțiuni care pot genera excepții
}
catch (<TipExcepție> <variabila>)
{
    // tratarea excepției
}
```

Exemplu:

```
private void btnImparte_Click(object sender, EventArgs e)
{
    try
    {
        int a = int.Parse(tbA.Text);
        int b = int.Parse(tbB.Text);
        int rezultat = a / b;
        lblRezultat.Text = rezultat.ToString();
    }
    catch (FormatException)
    {
        MessageBox.Show("Introduceți valori numerice valide.");
    }
    catch (DivideByZeroException)
    {
        MessageBox.Show("Împărțirea la zero nu este permisă.");
    }
}
```

11.3.1. Mai multe blocuri catch

O instrucțiune *try* poate fi urmată de mai multe blocuri *catch*, fiecare specializat pe un tip de excepție. La apariția unei excepții, blocurile sunt evaluate de sus în jos – se execută primul care corespunde tipului excepției (sau unei clase de bază a acesteia).

Din acest motiv, blocurile *catch* mai specifice trebuie plasate înaintea celor mai generale. Un bloc *catch* (*Exception ex*) plasat la final acționează ca o „plasă de siguranță” pentru orice excepție neprevăzută.

Exemplu:

```
try
{
    string text = File.ReadAllText(caleFisier);
    int numar = int.Parse(text);
}
catch (FileNotFoundException ex)
{
    MessageBox.Show($"Fișierul nu a fost găsit: {ex.FileName}");
}
catch (FormatException)
{
    MessageBox.Show("Fișierul nu conține un număr valid.");
}
catch (IOException ex)
{
    MessageBox.Show($"Eroare de I/O: {ex.Message}");
}
catch (Exception ex)
{
    MessageBox.Show($"Eroare neașteptată: {ex.Message}");
}
```

11.3.2. Accesarea detaliilor excepției

Obiectul excepție capturat în `catch` conține informații utile despre eroare. Cele mai importante proprietăți sunt prezentate mai jos.

Proprietate	Descriere
Message	Mesajul textual al excepției, destinat utilizatorului sau dezvoltatorului.
StackTrace	Ierarhia de apeluri care a dus la excepție.
InnerException	O excepție anterioară, înveluită în cea curentă (<code>null</code> dacă nu există).
Source	Numele assembly-ului sau al obiectului care a generat excepția.
TargetSite	Metoda în care s-a produs excepția.
Data	Dicționar <code>IDictionary</code> în care se pot adăuga informații de context.

11.4. Clauza finally

Clauza `finally` se adaugă după blocurile `catch` și conține cod care se execută **întotdeauna**, indiferent dacă blocul `try` s-a terminat normal sau a fost întrerupt de o excepție. Este folosită în principal pentru eliberarea resurselor (închidere fișiere, conexiuni, eliberare memorie nemanageriată).

Format general:

```
try
{
    // instrucțiuni care pot genera excepții
}
catch (<TipExcepție> <variabila>)
{
    // tratarea excepției
}
finally
{
    // cod executat obligatoriu
}
```

Exemplu:

```
StreamReader sr = null;
try
{
    sr = new StreamReader(caleFisier);
    string continut = sr.ReadToEnd();
    tbContinut.Text = continut;
}
catch (FileNotFoundException)
{
    MessageBox.Show("Fișierul nu există.");
}
finally
{
    // închidem fișierul indiferent dacă am avut sau nu excepție
    sr?.Close();
}
```

Operatorul ? din `sr?.Close()` ; înseamnă că se va apela `Close()` doar dacă `sr` nu este `null`.

Observații:

- Blocul `finally` se execută chiar și atunci când blocul `try` conține un `return` sau o instrucțiune de salt;
- Un bloc `try` poate avea numai `catch`, numai `finally` sau pe amândouă, dar nu poate lipsi și `catch` și `finally`;
- Dacă în `finally` apare o excepție nouă, aceasta o ascunde pe cea originală din `try`.

11.5. Instrucțiunea using și interfața IDisposable

Multe clase din .NET care gestionează resurse (fișiere, conexiuni, grafică) implementează interfața `IDisposable`, cu metoda `Dispose()` ce trebuie apelată pentru eliberarea

resurselor. Instrucțiunea `using` oferă o sintaxă concisă care garantează apelul `Dispose()` chiar și la apariția unei excepții.

Format general:

```
using (<Tip> <variabila> = new <Tip>(...))  
{  
    // utilizarea resursei  
}
```

Exemplu:

```
using (StreamReader sr = new StreamReader(caleFisier))  
{  
    string continut = sr.ReadToEnd();  
    tbContinut.Text = continut;  
}  
// aici sr.Dispose() a fost apelat automat
```

Varianta cu `using` este echivalentă semantic cu un bloc `try-finally` care apelează `Dispose()` în `finally`. Este preferată pentru claritatea codului.

11.6. Aruncarea excepțiilor – instrucțiunea `throw`

Excepțiile nu sunt generate doar de runtime; o metodă poate semnala o situație excepțională utilizând instrucțiunea `throw`. Acest mecanism permite propagarea erorii către apelant, care decide cum să o trateze.

Format general:

```
throw new <TipExcepție>([<mesaj>[, <innerException>]]);
```

Exemplu:

```
public void Retrage(decimal suma)  
{  
    if (suma <= 0)  
        throw new ArgumentException(  
            "Suma retrasă trebuie să fie pozitivă.", nameof(suma));  
    if (suma > _sold)  
        throw new InvalidOperationException(  
            "Sold insuficient.");  
    _sold -= suma;  
}
```

11.6.1. Re-aruncarea unei excepții: *throw*; vs *throw ex*;

În interiorul unui bloc `catch`, o excepție deja capturată poate fi re-aruncată pentru a permite unui strat superior să o trateze. Există două variante cu semantică diferită:

- **throw;** (recomandat) – păstrează informațiile originale ale excepției, inclusiv *StackTrace*. Se folosește când blocul curent doar *observă* excepția (ex. o jurnalizează) și o lasă să se propage mai departe;
- **throw ex;** – re-aruncă excepția, dar resetează *StackTrace* la locul instrucțiunii `throw`. Se pierde informațiile despre originea reală – în general de evitat.

Exemplu:

```
try
{
    Retrage(150);
}
catch (InvalidOperationException ex)
{
    ScrieInJurnal(ex);
    throw;           // păstrează stack trace-ul original
}

// comparativ:
try { ... }

catch (Exception ex)
{
    // Logare
    throw ex;        // resetează stack trace - de evitat!
}
```

11.7. Filtre de excepții – clauza *when*

Începând cu C# 6.0, blocurile `catch` pot fi însoțite de o clauză *when* care conține o expresie logică suplimentară. Blocul `catch` este executat numai dacă excepția corespunde tipului și expresia din *when* este adevărată.

Format general:

```
try { ... }
catch (<TipExcepție> <variabila>) when (<condiție>)
{
    // tratarea excepției
}
```

Exemplu:

```
try
{
    var raspuns = await client.GetAsync(url);
    raspuns.EnsureSuccessStatusCode();
}
catch (HttpRequestException ex) when
    (ex.StatusCode == HttpStatusCode.NotFound)
{
    MessageBox.Show("Resursa nu a fost găsită.");
}
catch (HttpRequestException ex) when
    (ex.StatusCode == HttpStatusCode.Unauthorized)
{
    MessageBox.Show("Autentificare necesară.");
}
catch (HttpRequestException ex)
{
    MessageBox.Show($"Eroare HTTP: {ex.Message}");
}
```

Filtrele `when` sunt mai eficiente decât verificările cu `if` în interiorul blocului `catch`: dacă condiția este falsă, excepția nu este considerată tratată și continuă să se propage, fără resetarea stivei de apeluri.

11.8. Excepții personalizate

Pentru a modela erori specifice domeniului unei aplicații, se pot defini clase proprii de excepții prin derivare din `Exception` sau dintr-o clasă mai specifică (de exemplu `ArgumentException` sau `InvalidOperationException`). Prin convenție, numele claselor de excepții personalizate se termină cu sufixul `Exception`.

O clasă de excepție personalizată trebuie să furnizeze cel puțin trei constructori standard: fără parametri, cu mesaj și cu mesaj + excepție internă.

Exemplu:

```
using System;

public class FondInsuficientException : Exception
{
    public decimal SoldCurent { get; }
    public decimal SumaCeruta { get; }

    public FondInsuficientException() { }

    public FondInsuficientException(string message)
        : base(message) { }
}
```

```
public FondInsuficientException(string message, Exception inner)
    : base(message, inner) { }

public FondInsuficientException(decimal soldCurent,
                                decimal sumaCeruta)
    : base($"Sold insuficient: sold = {soldCurent:F2}, " +
          $"cerut = {sumaCeruta:F2}.")
{
    SoldCurent = soldCurent;
    SumaCeruta = sumaCeruta;
}
}
```

Observații:

- Proprietățile suplimentare (în exemplu `SoldCurent` și `SumaCeruta`) oferă context suplimentar care poate fi folosit de blocurile catch sau de jurnalizare;
- Pentru erori provenite din argumente invalide se derivă din `ArgumentException`, nu din `Exception`, păstrând semantica generală;
- Numele clasei trebuie să exprime clar cauza erorii – un nume generic precum „`AppException`” nu aduce valoare.

12. Depanarea aplicațiilor în Visual Studio

Mediul Visual Studio oferă un *debugger* integrat care permite oprirea execuției într-un punct ales, inspectarea valorilor variabilelor și avansarea controlată prin cod. Pentru a rula aplicația sub debugger se apasă tasta *F5* (Debug → Start Debugging). Pentru rulare fără debugger se folosește *Ctrl+F5*.

12.1. Breakpoints

Un **breakpoint** este un marcaj plasat pe o linie de cod, care oprește execuția aplicației imediat înainte de a executa acea linie. Se inserează apăsând tasta *F9* pe linia dorită sau făcând clic în marginea stângă a editorului. Un breakpoint activ apare ca un cerc roșu.

Breakpoint-urile pot fi condiționate – click-dreapta pe breakpoint → *Conditions* – astfel încât execuția să se oprească doar atunci când o anumită condiție este adevărată (exemplu: `i == 100`).

12.2. Rularea pas cu pas

Odată execuția oprită într-un breakpoint, controlul se face prin următoarele comenzi:

Comandă	Scurtătură	Efect
Step Over	F10	Execută linia curentă; dacă aceasta conține un apel de metodă, metoda se execută integral.
Step Into	F11	Execută linia curentă; dacă aceasta conține un apel de metodă, execuția intră în interiorul metodei.
Step Out	Shift+F11	Termină execuția metodei curente și oprește la întoarcerea în metoda apelantă.
Continue	F5	Reia execuția normală până la următorul breakpoint.
Stop Debugging	Shift+F5	Oprește sesiunea de depanare.

12.3. Ferestrele Locals, Watch, Immediate

- **Locals** (Debug → Windows → Locals) – afișează toate variabilele vizibile în contextul curent, împreună cu valorile și tipurile lor;
- **Watch** (Debug → Windows → Watch) – permite adăugarea manuală a unor expresii (nume de variabile, apeluri de proprietăți, operații) care vor fi reevaluate la fiecare pas;
- **Immediate** (Debug → Windows → Immediate) – consolă interactivă în care se pot executa expresii C# în contextul execuției întrerupte (ex. modificarea unei variabile pentru a testa o ramură alternativă).

12.4. Fereastra Call Stack

Fereastra *Call Stack* (Debug → Windows → Call Stack) afișează stiva de apeluri – succesiunea de metode care a dus la linia curentă. Dublu-click pe o intrare din stivă duce la linia de cod corespunzătoare, permițând inspectarea stării la orice nivel al apelurilor.

12.4.1. Setări pentru oprirea la excepții

Din meniul *Debug → Windows → Exception Settings*, se pot bifa categorii de excepții pentru care debugger-ul va opri execuția **în momentul aruncării excepției**, nu doar atunci când aceasta nu este tratată. Această facilitate este utilă pentru a identifica excepții „înghițite” de blocuri *catch* prea permissive.

13. Aplicație – Cont bancar cu jurnal de tranzacții

Se extinde clasa `ContBancar` prezentată în Laboratorul 8 cu un mecanism complet de tratare a excepțiilor și cu un jurnal de tranzacții persistent într-un fișier text. Aplicația permite efectuarea de depuneri și retrageri, afișează istoricul tranzacțiilor și erorile într-un `ListBox`, iar erorile sunt salvate automat într-un fișier de jurnal.

13.1. Structura proiectului

Proiectul Windows Forms `GestionarCont` conține:

- `Models/TipTranzactie.cs` – enumerare pentru tipul tranzacției;
- `Models/Tranzactie.cs` – clasă care descrie o tranzacție;
- `Exceptions/FondInsuficientException.cs` – excepție personalizată;
- `Exceptions/SumaInvalidaException.cs` – excepție personalizată;
- `Services/ContBancar.cs` – logica contului;
- `Services/JurnalErori.cs` – serviciu de jurnalizare în fișier;
- `Form1.cs` – formularul principal.

13.1.1. Design grafic – Form1

Figura 22 - Interfața grafică pentru aplicația *Gestionar Cont*

Formularul (Figura 22) conține un `Label` care afișează soldul curent, două `TextBox`-uri pentru suma de depus/retras și pentru un comentariu opțional, două butoane pentru depunere și retragere, un buton pentru resetarea contului, un `ListBox` care afișează istoricul tranzacțiilor și un `ListBox` secundar pentru afișarea erorilor.

13.1.2. Proprietăți și evenimente pentru controale

Tip control	Nume (Name)	Setări relevante	Rol
Label	lbSold	Text = "Sold: 0,00 RON"; Font bold	Afișează soldul curent
TextBox	tbSuma	-	Introducerea sumei pentru tranzacție
TextBox	tbComentariu	-	Comentariu opțional al tranzacției
Button	btnDepune	Text = "Depune"; Click → btnDepune_Click	Efectuează o depunere
Button	btnRetrage	Text = "Retrage"; Click → btnRetrage_Click	Efectuează o retragere
Button	btnReset	Text = "Resetează contul"; Click → btnReset_Click	Resetează soldul și istoricul
ListBox	lbxTranzactii	-	Afișează tranzacțiile efectuate
ListBox	lbxErori	ForeColor = Red	Afișează mesajele de eroare
Label	label1	Text = "Sumă:"	Etichetă
Label	label2	Text = "Comentariu:"	Etichetă
Label	label3	Text = "Istoric tranzacții"	Etichetă
Label	label4	Text = "Erori"	Etichetă
Form	Form1	Text = "Gestionar cont"; Load → Form1_Load	Form-ul principal

13.1.3.Cod sursă

Models/TipTranzactie.cs

```
namespace GestionarCont.Models
{
    public enum TipTranzactie
    {
        Depunere,
        Retragere
    }
}
```

Models/Tranzactie.cs

```
using System;

namespace GestionarCont.Models
{
    public class Tranzactie
    {
        public DateTime DataOra { get; }
        public TipTranzactie Tip { get; }
        public decimal Suma { get; }
        public decimal SoldDupaTranzactie { get; }
        public string Comentariu { get; }

        public Tranzactie(TipTranzactie tip,
                           decimal suma,
                           decimal soldDupaTranzactie,
                           string comentariu)
        {
            DataOra = DateTime.Now;
            Tip = tip;
            Suma = suma;
            SoldDupaTranzactie = soldDupaTranzactie;
            Comentariu = comentariu ?? string.Empty;
        }

        public override string ToString()
        => $"{DataOra:HH:mm:ss} | {Tip,-10} | " +
           $"{Suma,10:F2} | Sold: {SoldDupaTranzactie,10:F2} | " +
           $"{Comentariu}";
    }
}
```

Exceptions/SumaInvalidaException.cs

```
using System;

namespace GestionarCont.Exceptions
{
    public class SumaInvalidaException : ArgumentException
    {
        public decimal SumaPrimita { get; }
    }
}
```

```
public SumaInvalidaException() { }

public SumaInvalidaException(string message)
    : base(message) { }

public SumaInvalidaException(string message, Exception inner)
    : base(message, inner) { }

public SumaInvalidaException(decimal suma)
    : base($"Suma {suma:F2} este invalidă. " +
          "Trebuie să fie un număr pozitiv.")
{
    SumaPrimita = suma;
}
}
```

Exceptions/FondInsuficientException.cs

```
using System;

namespace GestionarCont.Exceptions
{
    public class FondInsuficientException : Exception
    {
        public decimal SoldCurent { get; }
        public decimal SumaCeruta { get; }

        public FondInsuficientException() { }

        public FondInsuficientException(string message)
            : base(message) { }

        public FondInsuficientException(string message,
                                         Exception inner)
            : base(message, inner) { }

        public FondInsuficientException(decimal soldCurent,
                                         decimal sumaCeruta)
            : base($"Fond insuficient: sold = {soldCurent:F2}, " +
                  "$cerut = {sumaCeruta:F2}.")
        {
            SoldCurent = soldCurent;
            SumaCeruta = sumaCeruta;
        }
    }
}
```

Services/ContBancar.cs

```
using GestionarCont.Exceptions;
using GestionarCont.Models;
using System;
using System.Collections.Generic;

namespace GestionarCont.Services
{
```

```
public class ContBancar
{
    private decimal _sold;
    private readonly List<Tranzactie> _tranzactii =
        new List<Tranzactie>();

    public string NumeTitular { get; }
    public decimal Sold => _sold;
    public IReadOnlyList<Tranzactie> Tranzactii => _tranzactii;

    public ContBancar(string numeTitular,
        decimal soldInitial = 0)
    {
        if (string.IsNullOrEmpty(numeTitular))
            throw new ArgumentException(
                "Numele titularului este obligatoriu.",
                nameof(numeTitular));
        if (soldInitial < 0)
            throw new SumaInvalidaException(soldInitial);

        NumeTitular = numeTitular.Trim();
        _sold = soldInitial;
    }

    public Tranzactie Depune(decimal suma, string comentariu = "")
    {
        if (suma <= 0)
            throw new SumaInvalidaException(suma);

        _sold += suma;
        var t = new Tranzactie(TipTranzactie.Depunere,
            suma, _sold, comentariu);
        _tranzactii.Add(t);
        return t;
    }

    public Tranzactie Retrage(decimal suma, string comentariu = "")
    {
        if (suma <= 0)
            throw new SumaInvalidaException(suma);
        if (suma > _sold)
            throw new FondInsuficientException(_sold, suma);

        _sold -= suma;
        var t = new Tranzactie(TipTranzactie.Retragere, suma,
            _sold, comentariu);
        _tranzactii.Add(t);
        return t;
    }

    public void Reseteaza()
    {
        _sold = 0;
        _tranzactii.Clear();
    }
}
```

Services/JurnalErori.cs

```
using System;
using System.IO;

namespace GestionarCont.Services
{
    public static class JurnalErori
    {
        private static readonly string CaleJurnal =
            Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
                "erori.log");

        public static void Scrie(Exception ex)
        {
            try
            {
                using (var sw = new StreamWriter(CaleJurnal, append: true))
                {
                    sw.WriteLine($"[{DateTime.Now:yyyy-MM-dd HH:mm:ss}] " +
                        $"{ex.GetType().Name}: {ex.Message}");
                    if (ex.StackTrace != null)
                    {
                        sw.WriteLine(ex.StackTrace);
                    }
                    sw.WriteLine(new string('-', 60));
                }
            }
            catch
            {
                // dacă jurnalizarea eșuează, nu se propagă eroarea
            }
        }
    }
}
```

Form1.cs

```
using System;
using System.Globalization;
using System.Windows.Forms;
using GestionarCont.Exceptions;
using GestionarCont.Services;

namespace GestionarCont
{
    public partial class Form1 : Form
    {
        private ContBancar _cont;
        public Form1()
        {
            InitializeComponent();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            _cont = new ContBancar("Titular demo");
            ActualizeazaInterfata();
        }
    }
}
```

```
        tbSuma.Focus();
    }

    private void btnDepune_Click(object sender, EventArgs e)
    {
        try
        {
            decimal suma = CitesteSuma();
            var t = _cont.Depune(suma, tbComentariu.Text);
            lbxTranzactii.Items.Add(t);
            ActualizeazaInterfata();
            CurataCampuri();
        }
        catch (SumaInvalidaException ex) when (ex.SumaPrimita == 0)
        {
            // filtru: tratăm separat cazul zero
            AfiseazaEroare("Suma nu poate fi zero.", ex);
        }
        catch (SumaInvalidaException ex)
        {
            AfiseazaEroare(ex.Message, ex);
        }
        catch (FormatException ex)
        {
            AfiseazaEroare("Suma introdusă nu este un număr valid.",
                            ex);
        }
        catch (OverflowException ex)
        {
            AfiseazaEroare("Suma introdusă este prea mare.", ex);
        }
    }

    private void btnRetrage_Click(object sender, EventArgs e)
    {
        try
        {
            decimal suma = CitesteSuma();
            var t = _cont.Retrage(suma, tbComentariu.Text);
            lbxTranzactii.Items.Add(t);
            ActualizeazaInterfata();
            CurataCampuri();
        }
        catch (FondInsuficientException ex)
        {
            AfiseazaEroare(
                $"Fond insuficient (sold: {ex.SoldCurent:F2}).", ex);
        }
        catch (SumaInvalidaException ex)
        {
            AfiseazaEroare(ex.Message, ex);
        }
        catch (Exception ex)
        {
            // plasă de siguranță: orice excepție neprevăzută
            AfiseazaEroare("Eroare neașteptată: " + ex.Message, ex);
        }
    }
}
```

```
private void btnReset_Click(object sender, EventArgs e)
{
    _cont.Reseteaza();
    lbxTranzactii.Items.Clear();
    lbxErori.Items.Clear();
    ActualizeazaInterfata();
}

private decimal CitesteSuma()
{
    if (string.IsNullOrEmpty(tbSuma.Text))
        throw new SumaInvalidaException(
            "Completați câmpul Sumă.");

    string text = tbSuma.Text.Trim().Replace(',', '.');
    return decimal.Parse(text,
        NumberStyles.Number,
        CultureInfo.InvariantCulture);
}

private void AfiseazaEroare(string mesajUtilizator,
    Exception ex)
{
    lbxErori.Items.Add(
        $"{DateTime.Now:HH:mm:ss} - {mesajUtilizator}");
    JurnalErori.Scrie(ex);
}

private void ActualizeazaInterfata()
{
    lbSold.Text = $"Sold: {_cont.Sold:F2} RON";
}

private void CurataCampuri()
{
    tbSuma.Clear();
    tbComentariu.Clear();
    tbSuma.Focus();
}
}
```

Observații asupra codului:

- Ordinea blocurilor `catch` merge de la cazul cel mai specific (`SumaInvalidaException` cu filtru `when`) la cel mai general (`Exception`).
- Prin `catch (Exception ex)` plasat ultimul în handler-ul de retragere se captează orice excepție neprevăzută și aplicația nu se închide brusc.
- Fiecare eroare este jurnalizată într-un fișier *erori.log* aflat în directorul executabilului, pentru analiză ulterioară.
- Metoda privată *CitesteSuma* aruncă excepții proprii, lăsând handler-ul de buton să decidă cum sunt prezentate utilizatorului – separarea responsabilităților.

13.2. Exerciții

13.2.1.Exercițiul 1 – Extinderea jurnalului

Se cere extinderea aplicației „Gestionar cont bancar” cu următoarele funcționalități:

- adăugarea unui buton „Salvează tranzacțiile” care scrie întregul istoric de tranzacții într-un fișier text (*tranzactii.csv*), câte o linie pe tranzacție, folosind clasa *File*. Se va gestiona corect blocul *try-catch-finally* pentru a trata *UnauthorizedAccessException* și *IOException*;
- adăugarea unui buton „Încarcă tranzacțiile” care citește fișierul precedent și repopulează *ListBox*-ul cu istoricul.

13.2.2.Exercițiul 2 – Excepție personalizată nouă

Se cere:

- definirea unei excepții *LimitaZilnicaDepasitaException* (derivată din *InvalidOperationException*) care conține proprietățile *LimitaZilnica* și *SumaTotalaRetrasaAstazi*;
- modificarea clasei *ContBancar* pentru a limita totalul retragerilor dintr-o zi la 1000 RON și a arunca noua excepție atunci când limita este depășită;
- adăugarea unui bloc *catch* corespunzător în *Form1.cs* care afișează un mesaj dedicat, inclusiv valorile din proprietățile excepției;
- extinderea testelor manuale cu scenarii care depășesc limita și verifică mesajul afișat.

13.2.3.Exercițiul 3 – Depanare în Visual Studio

Fără a modifica codul aplicației, se cere efectuarea următoarelor activități de depanare și documentarea rezultatelor într-un document text scurt:

- plasarea unui *breakpoint* condiționat pe linia de adunare a sumei în metoda *Depune*, care să oprească execuția doar atunci când `_sold > 10000`;
- utilizarea ferestrelor *Locals* și *Watch* pentru a inspecta valorile câmpului `_sold` și a numărului de tranzacții pe parcursul mai multor depuneri consecutive;
- urmărirea apelului *Retrage* cu *Step Into* (F11) și identificarea punctului exact în care se aruncă *FondInsuficientException*;

- activarea opririi la excepția `FondInsuficientException` din *Exception Settings* și observarea momentului în care debugger-ul oprește execuția.

BIBLIOGRAFIE

- [1] J. Albahari, C# 10 in a nutshell - The definitive reference, O'Reilly Media, 2022.
- [2] F. Rasheed, C# School - 14 lessons to get you started with C# and .NET, Synchron Data S.L., 2006.
- [3] geeksforgeeks.org, „How to Install and Setup Visual Studio for C#?,” [Interactiv]. Available: <https://www.geeksforgeeks.org/c-sharp/how-to-install-and-setup-visual-studio-for-c-sharp/>. [Accesat 02 2026].
- [4] Microsoft, „List<T> Class,” [Interactiv]. Available: <https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.list-1?view=net-10.0>. [Accesat 03 2026].
- [5] Microsoft, „Enumerable Class,” [Interactiv]. Available: <https://learn.microsoft.com/en-us/dotnet/api/system.linq.enumerable?view=net-10.0>. [Accesat 03 2026].
- [6] geeksforgeek, „Basics of File Handling in C#,” [Interactiv]. Available: <https://www.geeksforgeeks.org/c-sharp/basics-of-file-handling-in-c-sharp/>. [Accesat 03 2026].
- [7] Microsoft, „File Class,” [Interactiv]. Available: <https://learn.microsoft.com/en-us/dotnet/api/system.io.file?view=net-10.0>. [Accesat 03 2026].
- [8] Wikipedia, „Computer network programming,” [Interactiv]. Available: https://en.wikipedia.org/wiki/Computer_network_programming. [Accesat 04 2026].
- [9] guru99.com, „Model TCP/IP: Ce sunt Straturile și Protocolul? Stiva TCP/IP,” guru99, [Interactiv]. Available: <https://www.guru99.com/ro/tcp-ip-model.html>. [Accesat 15 04 2026].
- [10] Wikipedia, „Domain Name System,” [Interactiv]. Available: https://en.wikipedia.org/wiki/Domain_Name_System. [Accesat 04 2026].
- [11] Wikipedia, „Transmission Control Protocol,” [Interactiv]. Available: https://en.wikipedia.org/wiki/Transmission_Control_Protocol. [Accesat 04 2026].

- [12] Wikipedia, „User Datagram Protocol,” [Interactiv]. Available: https://en.wikipedia.org/wiki/User_Datagram_Protocol. [Accesat 04 2026].
- [13] Wikipedia, „HTTP,” [Interactiv]. Available: <https://en.wikipedia.org/wiki/HTTP>. [Accesat 04 2026].
- [14] Wikipedia, „HTTPS,” [Interactiv]. Available: <https://en.wikipedia.org/wiki/HTTPS>. [Accesat 04 2026].